PHẦN 1: NỀN TẢNG CỦA QUẢN LÝ BỘ NHỚ

1.1. Bộ nhớ chính và các yêu cầu đối với hệ điều hành (9.1)

1.1.1. Vị trí của bộ nhớ chính trong quá trình thực thi

Bộ nhớ chính là một mảng lớn các byte, mỗi byte có một địa chỉ riêng. Trong một chu kỳ thực thi lệnh điển hình, CPU nạp lệnh từ bộ nhớ theo giá trị của bộ đếm chương trình, giải mã lệnh, có thể nạp thêm toán hạng từ bộ nhớ và ghi kết quả trở lại bộ nhớ. Đơn vị bộ nhớ chỉ nhận được một dòng các địa chỉ: nó không biết địa chỉ được tạo ra bằng cách nào và dùng cho lệnh hay cho dữ liệu. Vì vậy, khi nghiên cứu quản lý bộ nhớ, điều cần quan tâm là chuỗi địa chỉ mà chương trình sinh ra trong lúc chạy [1, tr. 349–350].

Bộ nhớ chính và các thanh ghi bên trong mỗi lõi xử lý là những vùng lưu trữ đa dụng duy nhất mà CPU truy cập trực tiếp. Có những lệnh máy nhận địa chỉ bộ nhớ làm đối số, nhưng không có lệnh nào nhận địa chỉ trên đĩa. Do đó, mọi lệnh đang thực thi và mọi dữ liệu mà lệnh sử dụng phải nằm ở một trong hai nơi này; dữ liệu chưa có trong bộ nhớ phải được chuyển vào trước khi CPU xử lý [1, tr. 350]. Mệnh đề tưởng như hiển nhiên này là điểm xuất phát của cả Part Four: chương trình muốn chạy phải được đưa vào bộ nhớ, trong khi bộ nhớ có hạn và được nhiều tiến trình dùng chung.

1.1.2. Chênh lệch tốc độ và vai trò của bộ nhớ đệm

Thanh ghi thường được truy cập trong một chu kỳ đồng hồ, còn một lần truy cập bộ nhớ chính phải thực hiện qua bus bộ nhớ và có thể mất nhiều chu kỳ. Trong thời gian đó, bộ xử lý thường phải dừng chờ (*stall*) vì chưa có dữ liệu để hoàn tất lệnh. Do tần suất truy cập bộ nhớ rất cao, tình trạng này không thể chấp nhận; giải pháp là đặt một bộ nhớ đệm (*cache*) tốc độ cao giữa CPU và bộ nhớ chính, thường nằm ngay trên chip. Bộ nhớ đệm do phần cứng tự quản lý, không cần hệ điều hành điều khiển [1, tr. 350].

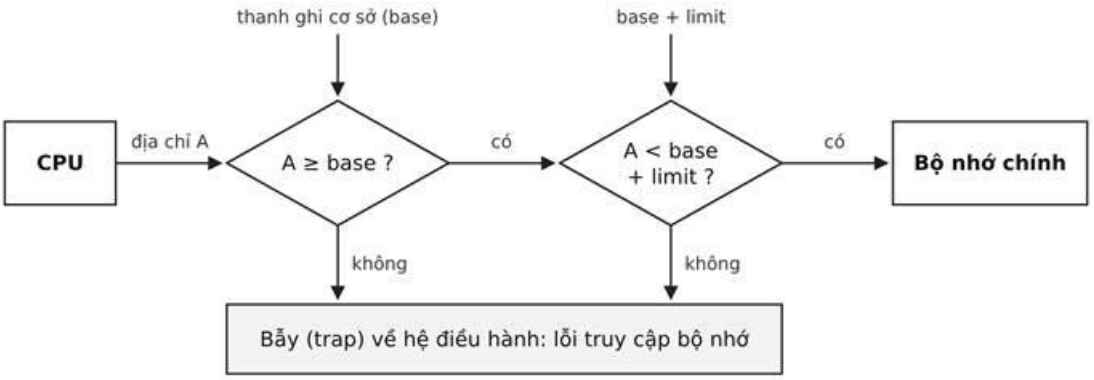
1.2. Bảo vệ bộ nhớ bằng thanh ghi cơ sở và thanh ghi giới hạn (9.1.1)

1.2.1. Nguyên lý

Để nhiều tiến trình cùng nằm trong bộ nhớ, mỗi tiến trình phải có không gian nhớ riêng; điều này vừa bảo vệ các tiến trình khỏi nhau vừa là điều kiện để chúng thực thi đồng thời. Hệ thống cần xác định được miền địa chỉ hợp lệ của từng tiến trình và bảo đảm tiến trình chỉ truy cập trong miền đó. Một cách hiện thực đơn giản dùng hai thanh ghi: thanh ghi cơ sở (*base register*), còn gọi là thanh ghi nền, chứa địa chỉ vật lý hợp lệ nhỏ nhất của tiến trình; thanh ghi giới hạn (*limit register*) chứa kích thước của miền [1, tr. 351]. Gọi A là một địa chỉ do tiến trình sinh ra trong chế độ người dùng, truy cập chỉ hợp lệ khi:

$$base \le A < base + limit \qquad (1.1)$$

Phần cứng CPU so sánh mọi địa chỉ sinh ra trong chế độ người dùng với hai thanh ghi (Hình 1.1). Mọi truy cập vào vùng nhớ của hệ điều hành hoặc của người dùng khác đều gây ra một bẫy (*trap*) chuyển quyền điều khiển cho hệ điều hành, và hệ điều hành xử lý trường hợp này như một lỗi nghiêm trọng [1, tr. 351–352]. Việc kiểm tra phải do phần cứng thực hiện vì hệ điều hành thường không can thiệp vào từng lần CPU truy cập bộ nhớ; nếu làm vậy, chi phí hiệu năng sẽ quá lớn [1, tr. 350].



Hình 1.1. *Kiểm tra địa chỉ bằng thanh ghi cơ sở và thanh ghi giới hạn*

1.2.2. Ví dụ

Giả sử thanh ghi cơ sở chứa 300040 và thanh ghi giới hạn chứa 120900. Khi đó tiến trình được truy cập mọi địa chỉ từ 300040 đến 420939, kể cả hai đầu mút [1, tr. 351]. Bảng 1.2 áp dụng điều kiện cho bốn địa chỉ cụ thể.

Bảng 1.2. *Kiểm tra một số địa chỉ với base = 300040 và limit = 120900*

Địa chỉ A	Đối chiếu với điều kiện (1.1)	Kết quả
300040	A ≥ 300040 và A < 420940	Hợp lệ, ô đầu tiên của miền
420939	A ≥ 300040 và A < 420940	Hợp lệ, ô cuối cùng của miền
420940	Vi phạm vế phải: A = base + limit	Bẫy về hệ điều hành
256000	Vi phạm vế trái: A < base	Bẫy về hệ điều hành

Nguồn: Nhóm tính theo số liệu ví dụ trong [1, tr. 351].

Ví dụ cho thấy base + limit = 420940 là địa chỉ đầu tiên nằm ngoài miền, nên vế phải của (1.1) phải là bất đẳng thức chặt. Nhầm lẫn giữa "<" và "≤" tại biên là lỗi thường gặp khi tính toán bằng tay.

1.2.3. Quyền thay đổi thanh ghi và đặc quyền của hệ điều hành

Hai thanh ghi chỉ có thể được nạp bằng một lệnh đặc quyền. Lệnh đặc quyền chỉ thực thi được trong chế độ nhân, và chỉ hệ điều hành chạy trong chế độ nhân, nên chỉ hệ điều hành mới thay đổi được giá trị của chúng; chương trình người dùng không thể tự mở rộng miền nhớ của mình [1, tr. 351]. Ngược lại, hệ điều hành chạy trong chế độ nhân được truy cập không hạn chế cả bộ nhớ của hệ điều hành lẫn bộ nhớ người dùng. Quyền này cần thiết để hệ điều hành nạp chương trình vào vùng nhớ người dùng, kết xuất chương trình khi có lỗi, đọc và sửa tham số của lời gọi hệ thống, thực hiện vào/ra với vùng nhớ người dùng và chuyển ngữ cảnh: trước khi chạy tiến trình kế tiếp, hệ điều hành lưu trạng thái của tiến trình hiện tại từ các thanh ghi vào bộ nhớ, rồi nạp trạng thái của tiến trình kế tiếp vào các thanh ghi [1, tr. 351–352].

Cơ chế base–limit ở đây kiểm tra trực tiếp địa chỉ vật lý. Khi trình bày cấp phát bộ nhớ liên tục, giáo trình dùng một biến thể kết hợp thanh ghi tái định vị với thanh ghi giới hạn: địa chỉ logic được so với giới hạn trước, sau đó mới được cộng với giá trị tái định vị. Biến thể này được phân tích ở mục 2.1 của Phần 2. Điểm chung của hai cách là phần cứng kiểm tra mọi lần truy cập, còn hệ điều hành chỉ quyết định giá trị của các thanh ghi.

1.3. Gắn kết địa chỉ (9.1.2)

1.3.1. Từ địa chỉ ký hiệu đến địa chỉ tuyệt đối

Chương trình thường nằm trên đĩa dưới dạng tệp thực thi nhị phân. Để chạy, chương trình phải được đưa vào bộ nhớ và đặt trong ngữ cảnh của một tiến trình. Hầu hết hệ thống cho phép tiến trình người dùng nằm ở bất kỳ vị trí nào của bộ nhớ vật lý; vì vậy, dù không gian địa chỉ của máy bắt đầu từ 00000, địa chỉ đầu tiên của tiến trình không nhất thiết là 00000 [1, tr. 352].

Trước khi được thực thi, chương trình đi qua nhiều bước như biên dịch, liên kết và nạp, và ở mỗi bước địa chỉ được biểu diễn theo cách khác nhau. Địa chỉ trong mã nguồn thường mang tính ký hiệu, chẳng hạn tên biến *count*. Trình biên dịch gắn (*bind*) địa chỉ ký hiệu với địa chỉ khả tái định vị, chẳng hạn "cách đầu mô-đun này 14 byte"; trình liên kết hoặc bộ nạp tiếp tục gắn địa chỉ khả tái định vị với địa chỉ tuyệt đối, chẳng hạn 74014. Mỗi lần gắn là một phép ánh xạ từ không gian địa chỉ này sang không gian địa chỉ khác [1, tr. 352].

1.3.2. Ba thời điểm gắn kết

Bảng 1.3. *Ba thời điểm gắn kết địa chỉ*

Thời điểm	Điều kiện áp dụng	Mã được sinh ra	Khi vị trí bắt đầu thay đổi
Lúc biên dịch	Biết trước tiến trình sẽ nằm ở đâu, chẳng hạn bắt đầu tại địa chỉ R	Mã tuyệt đối, bắt đầu từ R	Phải biên dịch lại
Lúc nạp	Chưa biết vị trí khi biên dịch	Mã khả tái định vị	Chỉ cần nạp lại mã người dùng
Lúc thực thi	Tiến trình có thể bị di chuyển giữa các vùng nhớ khi đang chạy	Địa chỉ logic, được dịch lúc chạy	Không cần làm lại; đòi hỏi phần cứng hỗ trợ

Hầu hết hệ điều hành dùng cách gắn kết lúc thực thi [1, tr. 353]. Lựa chọn này là tiền đề của nhiều kỹ thuật ở các phần sau: chỉ khi địa chỉ được dịch lúc chạy, hệ điều hành mới có thể dời tiến trình đi nơi khác để dồn nén bộ nhớ (mục 2.3.4), đặt các trang của tiến trình vào những khung không liền nhau (Phần 3) hay chỉ nạp một trang khi trang đó được tham chiếu (Phần 4).

1.4. Không gian địa chỉ logic, địa chỉ vật lý và MMU (9.1.3)

1.4.1. Địa chỉ logic và địa chỉ vật lý

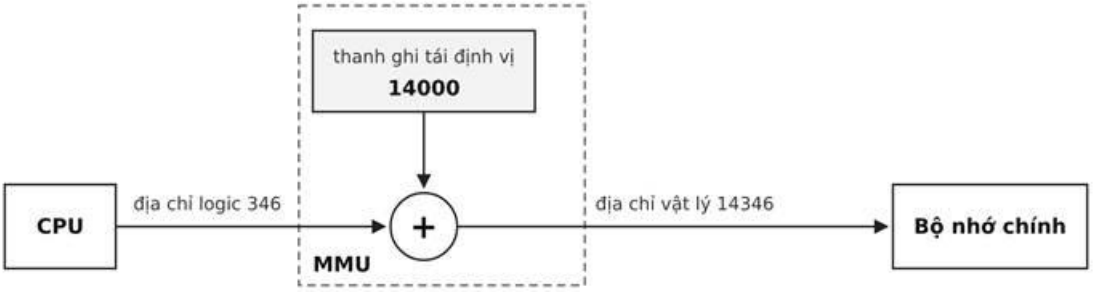
Địa chỉ do CPU sinh ra được gọi là địa chỉ logic (*logical address*), còn địa chỉ mà đơn vị bộ nhớ nhìn thấy, tức giá trị được nạp vào thanh ghi địa chỉ bộ nhớ, được gọi là địa chỉ vật lý (*physical address*). Gắn kết lúc biên dịch hoặc lúc nạp tạo ra địa chỉ logic và địa chỉ vật lý giống nhau; gắn kết lúc thực thi làm hai loại địa chỉ khác nhau, và khi đó địa chỉ logic thường được gọi là địa chỉ ảo (*virtual address*). Tập mọi địa chỉ logic mà chương trình sinh ra là không gian địa chỉ logic; tập các địa chỉ vật lý tương ứng là không gian địa chỉ vật lý [1, tr. 353–354].

1.4.2. MMU và thanh ghi tái định vị

Việc ánh xạ địa chỉ ảo sang địa chỉ vật lý lúc chạy do một thiết bị phần cứng gọi là đơn vị quản lý bộ nhớ (*memory-management unit*, MMU) đảm nhiệm. Lược đồ MMU đơn giản nhất là bản khái quát của cơ chế thanh ghi cơ sở ở mục 1.2: thanh ghi cơ sở giờ được gọi là thanh ghi tái định vị (*relocation register*), và giá trị của nó được cộng vào mọi địa chỉ mà tiến trình người dùng sinh ra tại thời điểm địa chỉ được gửi tới bộ nhớ [1, tr. 354]. Gọi R là giá trị thanh ghi tái định vị, ta có:

$$\text{địa chỉ vật lý} = R + \text{địa chỉ logic} \qquad (1.2)$$

Chẳng hạn, với R = 14000, truy cập tới địa chỉ logic 0 được ánh xạ thành địa chỉ vật lý 14000, còn truy cập tới địa chỉ logic 346 được ánh xạ thành 14346 (Hình 1.2).



Hình 1.2. *Tái định vị động bằng thanh ghi tái định vị trong MMU*

Chương trình người dùng không bao giờ truy cập trực tiếp địa chỉ vật lý. Chương trình có thể tạo một con trỏ tới địa chỉ 346, lưu nó vào bộ nhớ, thao tác trên

nó và so sánh nó với các địa chỉ khác, tất cả đều như với số 346; chỉ khi giá trị đó được dùng làm địa chỉ bộ nhớ, nó mới được tái định vị theo thanh ghi. Như vậy, chương trình làm việc với miền địa chỉ logic từ 0 đến max, còn các địa chỉ vật lý tương ứng nằm trong miền từ R + 0 đến R + max [1, tr. 354].

1.4.3. Hàm ánh xạ địa chỉ: mạch nối giữa các phần

Denning [2] gọi tập các địa chỉ mà chương trình sử dụng là không gian địa chỉ (*address space*), tập các vị trí thực trong bộ nhớ là không gian bộ nhớ (*memory space*), và quan hệ nối hai tập này là hàm ánh xạ địa chỉ (*address map*). Cách nhìn này đặt các kỹ thuật của Part Four trên cùng một trục: mỗi kỹ thuật là một cách hiện thực hàm ánh xạ với chi phí và độ linh hoạt khác nhau, như tóm tắt ở Bảng 1.4.

Bảng 1.4. *Các cách hiện thực hàm ánh xạ địa chỉ được trình bày trong tiểu luận*

Kỹ thuật	Cách ánh xạ địa chỉ logic sang địa chỉ vật lý	Trình bày tại
Thanh ghi tái định vị	Cộng một hằng số R vào mọi địa chỉ	Mục 1.4, 2.1
Phân trang	Tách địa chỉ thành số trang và độ dời; tra bảng trang để thay số trang bằng số khung	Phần 3
Phân trang theo yêu cầu	Như phân trang; trang chưa có trong bộ nhớ gây lỗi trang và được nạp khi cần	Phần 4
Thay thế trang	Thu hồi khung của một trang và sửa lại ánh xạ để nhường chỗ cho trang mới	Phần 5

1.5. Nạp động, liên kết động và thư viện dùng chung (9.1.4–9.1.5)

1.5.1. Nạp động

Nếu toàn bộ chương trình và dữ liệu phải nằm trong bộ nhớ vật lý thì kích thước tiến trình bị giới hạn bởi kích thước bộ nhớ. Nạp động (*dynamic loading*) nới giới hạn này: một thủ tục chỉ được nạp khi được gọi. Mọi thủ tục được lưu trên đĩa ở dạng khả tái định vị; chương trình chính được nạp và thực thi trước. Khi một thủ tục cần gọi thủ tục khác, nó kiểm tra xem thủ tục đó đã có trong bộ nhớ chưa; nếu chưa,

bộ nạp liên kết khả tái định vị được gọi để nạp thủ tục và cập nhật bảng địa chỉ của chương trình, sau đó quyền điều khiển được chuyển cho thủ tục vừa nạp [1, tr. 355].

Lợi ích của nạp động là thủ tục chỉ được nạp khi cần. Kỹ thuật này đặc biệt hữu ích khi chương trình có lượng lớn mã xử lý những tình huống hiếm gặp, chẳng hạn các thủ tục xử lý lỗi: tổng kích thước chương trình có thể lớn, nhưng phần thực sự được nạp có thể nhỏ hơn nhiều. Nạp động không cần hệ điều hành hỗ trợ đặc biệt; trách nhiệm thiết kế chương trình để tận dụng kỹ thuật này thuộc về người lập trình, dù hệ điều hành có thể cung cấp các thủ tục thư viện hỗ trợ [1, tr. 355].

1.5.2. Liên kết động và thư viện dùng chung

Thư viện liên kết động (*dynamically linked library*, DLL) là thư viện hệ thống được liên kết với chương trình người dùng khi chương trình chạy. Một số hệ điều hành chỉ hỗ trợ liên kết tĩnh, trong đó thư viện hệ thống được bộ nạp gộp vào ảnh nhị phân như mọi mô-đun đối tượng khác. Liên kết động tương tự nạp động, nhưng việc được hoãn đến lúc chạy là liên kết chứ không phải nạp. Kỹ thuật này thường dùng cho thư viện hệ thống như thư viện chuẩn của ngôn ngữ C. Nếu không có liên kết động, mỗi chương trình phải chứa một bản sao thư viện trong ảnh thực thi, vừa làm tăng kích thước tệp vừa lãng phí bộ nhớ chính [1, tr. 355–356].

Lợi ích thứ hai là thư viện có thể được nhiều tiến trình dùng chung, nên bộ nhớ chính chỉ cần giữ một bản; vì vậy DLL còn được gọi là thư viện dùng chung (*shared library*) và được dùng rộng rãi trên Windows và Linux. Khi chương trình tham chiếu một thủ tục trong thư viện động, bộ nạp tìm thư viện, nạp vào bộ nhớ nếu cần, rồi điều chỉnh các địa chỉ tham chiếu tới thủ tục đó theo vị trí của thư viện trong bộ nhớ [1, tr. 356].

Liên kết động còn thuận lợi cho việc cập nhật thư viện: khi thư viện được thay bằng phiên bản mới, mọi chương trình tham chiếu tới nó tự động dùng phiên bản mới mà không cần liên kết lại. Để chương trình không vô tình chạy với phiên bản không tương thích, thông tin phiên bản được lưu trong cả chương trình lẫn thư viện. Nhiều phiên bản của một thư viện có thể cùng được nạp vào bộ nhớ, và mỗi chương trình dựa vào thông tin phiên bản của mình để chọn bản thích hợp; thay đổi nhỏ giữ nguyên số phiên bản, còn thay đổi lớn làm tăng số này [1, tr. 356]. Quy ước này có thể quan

sát trên Linux: lệnh *ldd /bin/ls* liệt kê các thư viện động mà chương trình *ls* cần, trong đó thư viện C chuẩn mang tên *libc.so.6*, với 6 là số phiên bản chính.

Khác với nạp động, liên kết động và thư viện dùng chung thường cần hệ điều hành hỗ trợ. Vì các tiến trình được bảo vệ khỏi nhau, chỉ hệ điều hành mới kiểm tra được thủ tục cần dùng đã nằm trong vùng nhớ của tiến trình khác hay chưa, và mới có thể cho phép nhiều tiến trình truy cập cùng một vùng nhớ [1, tr. 356]. Cách nhiều tiến trình dùng chung một bản thư viện được giáo trình trình bày cùng với phân trang ở mục 9.3.4, tương ứng với Phần 3 của tiểu luận. Bảng 1.5 tóm tắt sự khác biệt giữa hai kỹ thuật.

Bảng 1.5. *So sánh nạp động và liên kết động*

Tiêu chí	Nạp động	Liên kết động
Công việc được hoãn đến lúc chạy	Nạp thủ tục vào bộ nhớ	Liên kết chương trình với thư viện hệ thống
Đối tượng điển hình	Thủ tục ít dùng, như xử lý lỗi	Thư viện hệ thống, như thư viện C chuẩn
Dùng chung giữa các tiến trình	Không	Có, bộ nhớ chỉ giữ một bản thư viện
Hỗ trợ của hệ điều hành	Không bắt buộc	Thường cần

1.6. Hoán đổi (9.5)

Lệnh và dữ liệu phải nằm trong bộ nhớ khi được thực thi, nhưng một tiến trình, hoặc một phần của nó, có thể tạm thời được chuyển ra vùng lưu trữ phụ (*backing store*) rồi đưa trở lại để tiếp tục chạy. Hoán đổi (*swapping*) cho phép tổng không gian địa chỉ vật lý của mọi tiến trình vượt quá bộ nhớ vật lý thực có, qua đó tăng mức đa chương của hệ thống [1, tr. 376].

1.6.1. Hoán đổi chuẩn

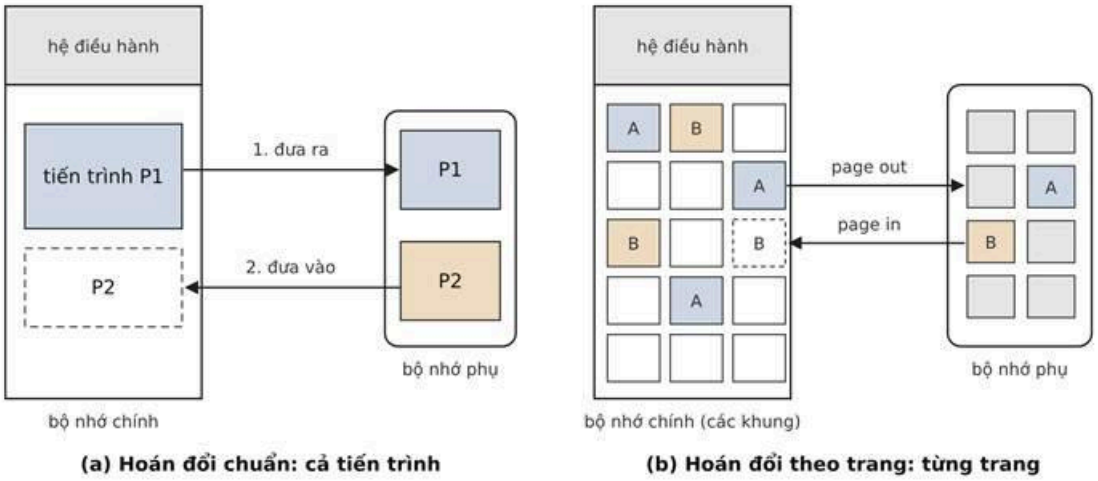
Hoán đổi chuẩn (*standard swapping*) chuyển toàn bộ tiến trình giữa bộ nhớ chính và vùng lưu trữ phụ. Vùng lưu trữ phụ thường là thiết bị lưu trữ thứ cấp tốc độ cao, phải đủ lớn để chứa các phần tiến trình cần lưu và cho phép truy cập trực tiếp.

Khi chuyển một tiến trình ra, các cấu trúc dữ liệu đi kèm cũng phải được ghi ra; với tiến trình đa luồng, cấu trúc dữ liệu của từng luồng cũng phải được hoán đổi, và hệ điều hành phải lưu siêu dữ liệu để khôi phục tiến trình khi đưa trở lại. Ưu điểm của cách này là cho phép cấp quá bộ nhớ vật lý: các tiến trình nhàn rỗi hoặc gần như nhàn rỗi được chuyển ra, nhường bộ nhớ cho tiến trình đang hoạt động [1, tr. 377].

Nhược điểm là thời gian di chuyển cả tiến trình quá lớn. Vì vậy, hoán đổi chuẩn từng được dùng trong các hệ UNIX truyền thống nhưng nhìn chung không còn được dùng trong các hệ điều hành hiện nay; ngoại lệ là Solaris, vẫn dùng hoán đổi chuẩn nhưng chỉ khi bộ nhớ trống còn cực kỳ ít [1, tr. 377]. Để hình dung quy mô chi phí, xét một phép tính minh họa với số liệu giả định: tiến trình có kích thước 100 MB, thiết bị lưu trữ truyền dữ liệu với tốc độ 50 MB/s. Chỉ riêng thời gian truyền khi đưa tiến trình ra đã là 100 / 50 = 2 giây, đưa vào thêm 2 giây nữa, tổng cộng khoảng 4 giây, chưa kể các độ trễ khác. Nếu chỉ cần chuyển một trang 4 KB, lượng dữ liệu phải chép giảm 25.600 lần (100 MB = 102.400 KB).

1.6.2. Hoán đổi theo trang

Hầu hết hệ điều hành, trong đó có Linux và Windows, dùng một biến thể: hoán đổi các trang của tiến trình thay vì toàn bộ tiến trình. Cách này vẫn cho phép cấp quá bộ nhớ vật lý nhưng tránh được chi phí chuyển cả tiến trình, vì thường chỉ một số ít trang tham gia. Thao tác chuyển một trang từ bộ nhớ ra vùng lưu trữ phụ gọi là *page out*, thao tác ngược lại gọi là *page in*. Hiện nay, thuật ngữ "hoán đổi" thường chỉ hoán đổi chuẩn, còn "phân trang" (*paging*) chỉ hoán đổi theo trang [1, tr. 377]. Hình 1.3 so sánh hai cách.



Hình 1.3. *Hoán đổi chuẩn (a) và hoán đổi theo trang (b)*

Hoán đổi theo trang là cầu nối giữa Chương 9 và Chương 10. Giáo trình nhận xét rằng hoán đổi theo trang kết hợp tốt với bộ nhớ ảo [1, tr. 377], và ở Chương 10 mô tả hệ thống phân trang theo yêu cầu tương tự một hệ thống phân trang có hoán đổi, trong đó tiến trình nằm ở bộ nhớ thứ cấp và trang chỉ được nạp khi được dùng tới [1, tr. 393]. Cơ chế này được phân tích ở Phần 4.

1.6.3. Hoán đổi trên thiết bị di động

Khác với máy tính cá nhân và máy chủ, các hệ thống di động thường không hỗ trợ hoán đổi dưới bất kỳ hình thức nào. Thiết bị di động dùng bộ nhớ flash thay cho ổ đĩa dung lượng lớn; dung lượng hạn chế, số lần ghi mà flash chịu được trước khi trở nên kém tin cậy có hạn, và thông lượng giữa bộ nhớ chính với flash thấp là những lý do khiến người thiết kế tránh hoán đổi [1, tr. 377].

Thay vào đó, khi bộ nhớ trống giảm dưới một ngưỡng, iOS yêu cầu các ứng dụng tự nguyện giải phóng bộ nhớ đã được cấp. Dữ liệu chỉ đọc như mã lệnh có thể bị loại khỏi bộ nhớ rồi nạp lại từ flash khi cần, còn dữ liệu đã bị sửa đổi như ngăn xếp không bao giờ bị loại; ứng dụng không giải phóng đủ bộ nhớ có thể bị hệ điều hành kết thúc. Android áp dụng chiến lược tương tự, nhưng trước khi kết thúc một tiến trình sẽ ghi trạng thái ứng dụng xuống flash để có thể khởi động lại nhanh. Vì vậy, lập trình viên ứng dụng di động phải cấp phát và giải phóng bộ nhớ thận trọng, tránh dùng quá nhiều bộ nhớ hoặc để rò rỉ bộ nhớ [1, tr. 378]. Ngoài ra, nén bộ nhớ là thành phần

không thể thiếu trong chiến lược quản lý bộ nhớ của hầu hết hệ điều hành di động, kể cả Android và iOS [1, tr. 426]; kỹ thuật này được trình bày ở mục 2.4.

Tuy vậy, nhận định "thiết bị di động không hoán đổi" nên được hiểu là một lựa chọn thiết kế hơn là quy luật cố định. Zhu và cộng sự [3] ghi nhận các thiết bị di động, đặc biệt là thiết bị cấu hình thấp, đang đưa hoán đổi trở lại như một cách mở rộng bộ nhớ mà không cần bổ sung DRAM, theo ba hướng: hoán đổi truyền thống, hoán đổi dựa trên nén và hoán đổi dựa trên phần cứng mới. Nhóm tác giả cũng đề xuất đánh giá hoán đổi trên thiết bị di động bằng hai thước đo gắn trực tiếp với trải nghiệm người dùng là thời gian khởi động ứng dụng và độ trễ khi chuyển đổi giữa các ứng dụng.

1.6.4. Hoán đổi như một dấu hiệu quá tải

Dù hoán đổi theo trang hiệu quả hơn hoán đổi cả tiến trình, việc hệ thống phải hoán đổi thường là dấu hiệu có nhiều tiến trình hoạt động hơn mức bộ nhớ vật lý đáp ứng được. Giáo trình nêu hai hướng xử lý: kết thúc bớt một số tiến trình, hoặc bổ sung bộ nhớ vật lý [1, tr. 378]. Khi tình trạng thiếu bộ nhớ trở nên nghiêm trọng, hệ thống có thể rơi vào thrashing, hiện tượng được phân tích ở mục 4.4. Bảng 1.6 tổng hợp các hình thức hoán đổi đã trình bày.

Bảng 1.6. *So sánh các hình thức hoán đổi*

Tiêu chí	Hoán đổi chuẩn	Hoán đổi theo trang	Thiết bị di động
Đơn vị chuyển	Toàn bộ tiến trình	Từng trang	Thường không hoán đổi
Chi phí di chuyển	Rất lớn	Nhỏ hơn nhiều	Không phát sinh
Ví dụ hệ điều hành	Solaris, chỉ khi bộ nhớ trống cực kỳ ít	Linux, Windows	iOS, Android
Cách xử lý khi thiếu bộ nhớ	Chuyển tiến trình nhàn rỗi ra bộ nhớ phụ	Đưa trang ra (page out), đưa trang vào (page in)	Ứng dụng tự giải phóng bộ nhớ, kết thúc ứng dụng, nén bộ nhớ

1.7. Bộ nhớ ảo: ý tưởng nền tảng (10.1)

1.7.1. Chương trình không cần nằm hoàn toàn trong bộ nhớ

Các kỹ thuật ở mục 1.2 đến 1.6 đều xuất phát từ yêu cầu lệnh đang thực thi phải nằm trong bộ nhớ vật lý. Cách đáp ứng đầu tiên là đặt toàn bộ không gian địa chỉ logic vào bộ nhớ vật lý; liên kết động có thể nới bớt yêu cầu này nhưng thường đòi hỏi người lập trình phải cẩn trọng và làm thêm việc. Yêu cầu đó hợp lý nhưng bất lợi, vì nó giới hạn kích thước chương trình bởi kích thước bộ nhớ vật lý [1, tr. 389–390].

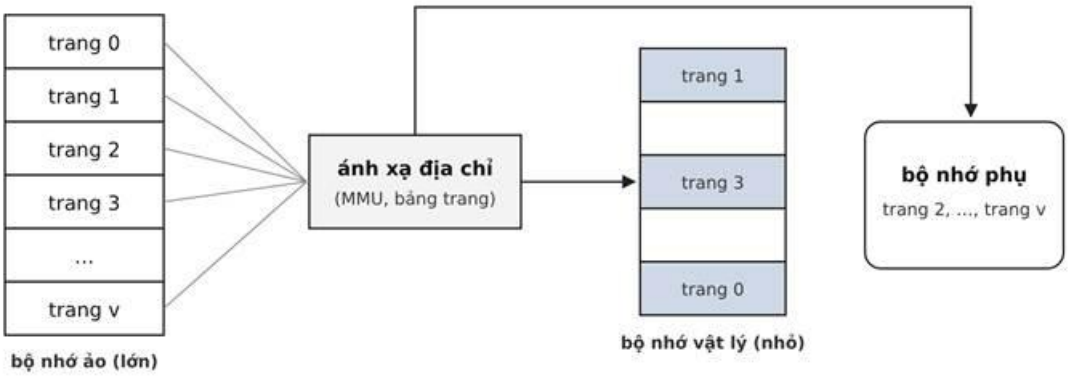
Khảo sát các chương trình thực tế cho thấy trong nhiều trường hợp không cần đến toàn bộ chương trình. Mã xử lý các lỗi hiếm khi xảy ra gần như không bao giờ được thực thi; mảng, danh sách và bảng thường được cấp nhiều bộ nhớ hơn mức cần, chẳng hạn một mảng khai báo 100 × 100 phần tử nhưng hiếm khi dùng quá 10 × 10; một số tùy chọn và tính năng của chương trình rất ít khi được dùng. Ngay cả khi cần toàn bộ chương trình, các phần của nó cũng không nhất thiết được dùng cùng lúc [1, tr. 390]. Khả năng thực thi một chương trình chỉ nằm một phần trong bộ nhớ vì thế mang lại ba lợi ích [1, tr. 390]:

- Chương trình không còn bị giới hạn bởi dung lượng bộ nhớ vật lý; người lập trình có thể viết chương trình cho một không gian địa chỉ ảo cực lớn.
- Mỗi chương trình chiếm ít bộ nhớ vật lý hơn nên nhiều chương trình có thể chạy đồng thời, làm tăng mức sử dụng CPU và thông lượng mà không làm tăng thời gian đáp ứng hay thời gian hoàn thành.
- Cần ít thao tác vào/ra hơn để nạp hoặc hoán đổi các phần của chương trình, nên mỗi chương trình chạy nhanh hơn.

1.7.2. Tách bộ nhớ logic khỏi bộ nhớ vật lý

Bộ nhớ ảo (*virtual memory*) là sự tách biệt giữa bộ nhớ logic mà người lập trình nhìn thấy với bộ nhớ vật lý. Sự tách biệt này cho phép cung cấp một bộ nhớ ảo rất lớn trong khi bộ nhớ vật lý nhỏ hơn nhiều (Hình 1.4), nhờ đó người lập trình không phải lo về dung lượng bộ nhớ vật lý [1, tr. 390]. Ý tưởng này xuất hiện từ rất sớm:

máy Atlas do Đại học Manchester và hãng Ferranti phát triển trong giai đoạn 1957–1962 được IEEE ghi nhận là nơi bộ nhớ ảo ra đời. Trên Atlas, bộ nhớ lõi tốc độ cao và trống từ tốc độ thấp được tổ chức để hoạt động như một hệ lưu trữ một mức duy nhất [4], [5].



Hình 1.4. *Bộ nhớ ảo lớn hơn bộ nhớ vật lý*
Nguồn: Nhóm vẽ lại theo [1, tr. 391, Hình 10.1].

Không gian địa chỉ ảo của tiến trình là cách nhìn logic về việc tiến trình được lưu trong bộ nhớ: tiến trình bắt đầu tại một địa chỉ logic, chẳng hạn 0, và nằm liên tục trong bộ nhớ. Trên thực tế, bộ nhớ vật lý được tổ chức thành các khung trang, và các khung cấp cho một tiến trình có thể không liền nhau; MMU chịu trách nhiệm ánh xạ các trang logic vào khung vật lý [1, tr. 390–391]. Trong không gian địa chỉ ảo, vùng heap tăng dần lên khi cấp phát động, còn vùng stack tăng dần xuống qua các lời gọi hàm. Khoảng trống lớn giữa heap và stack thuộc không gian địa chỉ ảo nhưng chỉ cần trang vật lý khi heap hoặc stack phát triển tới. Không gian địa chỉ có những khoảng trống như vậy được gọi là không gian địa chỉ thưa (*sparse address space*); các khoảng trống có thể được lấp khi stack, heap tăng hoặc khi liên kết động thư viện trong lúc chạy [1, tr. 391].

1.7.3. Chia sẻ trang nhờ bộ nhớ ảo

Ngoài việc tách bộ nhớ logic khỏi bộ nhớ vật lý, bộ nhớ ảo cho phép hai hay nhiều tiến trình dùng chung tệp và vùng nhớ thông qua chia sẻ trang (mục 9.3.4 của giáo trình, thuộc Phần 3). Thư viện hệ thống như thư viện C chuẩn được ánh xạ vào không gian địa chỉ ảo của nhiều tiến trình, thường ở chế độ chỉ đọc; mỗi tiến trình coi

thư viện là một phần không gian địa chỉ của mình, nhưng các trang vật lý chứa thư viện được dùng chung. Theo cách tương tự, một tiến trình có thể tạo vùng nhớ dùng chung với tiến trình khác, và các trang có thể được dùng chung khi tạo tiến trình bằng lời gọi hệ thống *fork()*, giúp tăng tốc việc tạo tiến trình [1, tr. 392].

Đây là chỗ liên kết động ở mục 1.5 gặp bộ nhớ ảo: thư viện dùng chung không còn chỉ là kỹ thuật tiết kiệm bộ nhớ ở mức chương trình mà trở thành hệ quả tự nhiên của việc ánh xạ trang. Để hiện thực bộ nhớ ảo, giáo trình bắt đầu với phân trang theo yêu cầu, nội dung của Phần 4; việc chia sẻ trang khi gọi *fork()* được khai thác bởi kỹ thuật copy-on-write ở Phần 5.

Tài liệu tham khảo của Phần 1

[1] A. Silberschatz, P. B. Galvin, and G. Gagne, *Operating System Concepts*, 10th ed. Hoboken, NJ, USA: Wiley, 2018.

[2] P. J. Denning, "Virtual memory," *ACM Computing Surveys*, vol. 2, no. 3, pp. 153–189, 1970, doi: 10.1145/356571.356573.

[3] X. Zhu, D. Liu, L. Liang, K. Zhong, L. Long, M. Qiu, Z. Shao, and E. H.-M. Sha, "Revisiting swapping in mobile systems with SwapBench," *Future Generation Computer Systems*, vol. 74, pp. 265–275, 2017, doi: 10.1016/j.future.2016.05.026.

[4] T. Kilburn, D. B. G. Edwards, M. J. Lanigan, and F. H. Sumner, "One-level storage system," *IRE Transactions on Electronic Computers*, vol. EC-11, pp. 223–235, 1962.

[5] IEEE, "Milestones: Atlas computer and the invention of virtual memory, 1957–1962," *Engineering and Technology History Wiki*, 2022. Accessed: Oct. 21, 2026. [Online]. Available: https://ethw.org/Milestones:Atlas_Computer_and_the_Invention_of_Virtual_Memory,_1957-1962

🖿 PHẦN 2_ Hệ điều hành(Các hình draw.io ở trong link)

PHẦN 2: CẤP PHÁT BỘ NHỚ

MỤC LỤC

2.1. Bảo vệ bộ nhớ: 2
2.1.1. Nguyên lý bảo vệ bộ nhớ 2
2.1.2. Ví dụ minh họa 3
2.1.3. Ý nghĩa và giới hạn của cơ chế 3
2.2. Cấp phát bộ nhớ 4
2.2.1. Nguyên tắc cấp phát và thu hồi bộ nhớ 4
2.2.2. Chiến lược First-fit 5
2.2.3. Chiến lược Best-fit 6
2.2.4. Chiến lược Worst-fit 6
2.2.5. Ví dụ minh họa 7
2.2.6. So sánh và đánh giá 8
2.3. Phân mảnh bộ nhớ và dồn nén 8
2.3.1. Phân mảnh ngoài 9
2.3.2. Phân mảnh trong 10
2.3.3. Ví dụ phân tích quá trình hình thành phân mảnh ngoài 11
2.3.4. Dồn nén bộ nhớ và điều kiện thực hiện 12
2.3.5. Chi phí dồn nén và lựa chọn biện pháp khắc phục 13
2.4. Nén bộ nhớ 15
2.4.1. Nguyên lý hoạt động 15
2.4.2. Lợi ích và hạn chế 15
2.5. Cấp phát bộ nhớ cho nhân hệ điều hành 16
2.5.1. Hệ thống Buddy 16
2.5.2. Cấp phát Slab 17
2.5.3. So sánh hai phương pháp 18
2.6. Một số vấn đề khác trong quản lý bộ nhớ 19
2.6.1. Tải trước trang 19
2.6.2. Kích thước trang 20
2.6.3. Phạm vi bao phủ của TLB 21
2.6.4. Bảng trang đảo ngược 21
2.6.5. Cấu trúc chương trình 22
2.6.6. Phối hợp I/O và khóa trang 23
TÀI LIỆU THAM KHẢO: 24

2.1. Bảo vệ bộ nhớ:

Trong hệ thống đa chương trình, bộ nhớ chính đồng thời chứa hệ điều hành và nhiều tiến trình của người dùng. Mỗi tiến trình được cấp một vùng nhớ để lưu trữ mã lệnh, dữ liệu và các thông tin phục vụ quá trình thực thi. Nếu không kiểm soát phạm vi truy cập, một tiến trình có thể đọc hoặc ghi đè lên vùng nhớ của tiến trình khác, gây sai lệch dữ liệu hoặc ảnh hưởng đến hoạt động của hệ thống. Vì vậy, để bảo vệ bộ nhớ nhằm bảo đảm mỗi tiến trình chỉ truy cập những địa chỉ thuộc phạm vi được phép, duy trì sự cô lập giữa các tiến trình và bảo vệ hệ điều hành (Silberschatz et al., 2018).

2.1.1. Nguyên lý bảo vệ bộ nhớ

Trong phương pháp cấp phát bộ nhớ liên tục, một cơ chế bảo vệ cơ bản là kết hợp *thanh ghi tái định vị* (relocation register) và *thanh ghi giới hạn* (limit register). Thanh ghi tái định vị, còn gọi là thanh ghi nền trong mô hình này, lưu địa chỉ vật lý bắt đầu của vùng nhớ được cấp cho tiến trình. Thanh ghi giới hạn xác định kích thích phạm vi địa chỉ logic hợp lệ. Hai thanh ghi được sử dụng để kiểm tra và chuyển đổi địa chỉ trước khi truy cập bộ nhớ (Silberschatz et al., 2018).

Gọi **B** là giá trị thanh ghi tái định vị, **L** là giá trị thanh ghi giới hạn và **d** là địa chỉ logic do CPU tạo ra. Địa chỉ được chấp nhận khi thỏa mãn:

$$0 \leq d < L.$$

Nếu điều kiện trên được đáp ứng, bộ quản lý bộ nhớ – MMU (Memory – Management Unit) – xác định địa chỉ vật lý theo công thức:

$$A_{vật\ lý} = B + d.$$

Như vậy, phạm vi địa chỉ vật lý và tiến trình được phép truy cập:

$$B \leq A_{vật\ lý} < B + L.$$

Nếu địa chỉ logic nằm ngoài giới hạn, phần cứng phát sinh ngoại lệ truy cập bộ nhớ và chuyển quyền xử lý cho hệ điều hành. Hệ điều hành sau đó thực hiện biện pháp phù hợp, chẳng hạn kết thúc tiến trình vi phạm. Nhờ việc kiểm tra bằng phần

cứng, yêu cầu truy cập ngoài phạm vi bị ngăn chặn trong quá trình thực thi (Silberschatz et al., 2018).

Cơ chế bảo vệ chỉ có hiệu lực khi tiến trình người dùng không được tự ý thay đổi hai thanh ghi. Vì vậy, các lệnh cập nhật thanh ghi nền và giới hạn phải là lệnh đặc quyền, chỉ được thực hiện trong chế độ nhân. Nếu một tiến trình có thể tự thay đổi các giá trị này, nó có thể mở rộng phạm vi truy cập và vượt qua giới hạn bảo vệ đã thiết lập (Arpaci-Dusseau & Arpaci-Dusseau, 2023).

Khi chuyển ngữ cảnh, hệ điều hành lưu các giá trị thanh ghi của tiến trình đang chạy và khôi phục các giá trị tương ứng của tiến trình được chọn. Nhờ đó, việc kiểm tra địa chỉ luôn phù hợp với vị trí và kích thước vùng nhớ của tiến trình hiện hành (Arpaci-Dusseau & Arpaci-Dusseau, 2023).

2.1.2. Ví dụ minh họa

Giả sử tiến trình P được cấp vùng nhớ bắt đầu tại địa chỉ vật lý 40 000, có kích thước 12000 byte. Thanh ghi tái định vị và thanh ghi giới hạn lần lượt có giá trị B =40 000 và L= 12000.

- Với địa chỉ logic d = 3 200, do $0 \leq 3\,200 < 12\,000$truy cập hợp lệ. Địa chỉ vật lý là: A = 40 000 + 3 200 = 43 200

- Với địa chỉ logic d = 12 000, truy cập bị từ chối vì $d \geq L$. Phần cứng phát sinh ngoại lệ và chuyển quyền xử lý cho hệ điều hành.

2.1.3. Ý nghĩa và giới hạn của cơ chế

Cơ chế sử dụng thanh ghi tái định vị và thanh ghi giới hạn giúp bảo vệ vùng nhớ của hệ điều hành và các tiến trình khác trước những truy cập ngoài phạm vi cho phép. Đồng thời, chương trình có thể sử dụng địa chỉ logic mà không cần biết vị trí vật lý cụ thể của mình, vì phần cứng đảm nhiệm việc chuyển đổi địa chỉ trong quá trình thực thi (Silberschatz et al., 2018).

Cơ chế này còn hỗ trợ việc bố trí bộ nhớ linh hoạt. Trong hệ thống hỗ trợ tái định vị động, hệ điều hành có thể tạm dừng tiến trình, di chuyển vùng nhớ của nó đến

vị trí khác và cập nhật giá trị thanh ghi nền trước khi cho tiến trình tiếp tục chạy. Các địa chỉ logic của chương trình được giữ nguyên, còn phép chuyển đổi địa chỉ sẽ phản ánh vị trí vật lý mới (Arpaci-Dusseau & Arpaci-Dusseau, 2023).

Tuy nhiên, hai thanh ghi chỉ kiểm soát ranh giới chung của vùng nhớ tiến trình. Từ nguyên lý này có thể thấy rằng một thao tác ghi sai vị trí nhưng vẫn nằm trong vùng được cấp sẽ không bị phát hiện chỉ bằng phép kiểm tra giới hạn. Cơ chế cũng chưa phân biệt quyền đọc, ghi và thực thi đối với từng phần bên trong vùng nhớ. Vì vậy, đây là phương thức bảo vệ cơ bản trong cấp phát bộ nhớ liên tục; những yêu cầu bảo vệ chi tiết hơn cần có các cơ chế bổ sung.

2.2. Cấp phát bộ nhớ

Trong phương pháp cấp phát bộ nhớ liên tục, mỗi tiến trình được bố trí trong một vùng nhớ vật lý liền nhau. Khi nhiều tiến trình cùng tồn tại, hệ điều hành cần theo dõi những vùng đã được sử dụng và những vùng còn trống để đáp ứng các yêu cầu cấp phát. Với mô hình phân vùng có kích thước thay đổi, kích thước vùng cấp cho mỗi tiến trình được xác định dựa trên nhu cầu bộ nhớ của tiến trình đó (Silberschatz et al., 2018).

Quá trình các tiến trình được nạp vào và rời khỏi bộ nhớ tạo ra những vùng trống có kích thước khác nhau. Vì vậy, vấn đề cấp phát không chỉ là xác định tổng dung lượng còn lại, mà còn phải lựa chọn một vùng trống đủ lớn cho từng yêu cầu. Ba chiến lược thường được sử dụng là First-fit, Best-fit và Worst-fit.

2.2.1. Nguyên tắc cấp phát và thu hồi bộ nhớ

Hệ điều hành duy trì thông tin về các vùng nhớ đang được sử dụng và các vùng còn trống. Một vùng nhớ trống liên tục được gọi là *hole*. Khi có yêu cầu cấp phát, hệ thống tìm một vùng trống có kích thước không nhỏ hơn dung lượng được yêu cầu (Silberschatz et al., 2018).

Gọi R là kích thước bộ nhớ tiến trình yêu cầu và H là kích thước vùng trống được xem xét. Điều kiện để cấp phát là:

$$H \geq R.$$

Nếu H = R, toàn bộ vùng trống được cấp cho tiến trình. Nếu H > R, vùng trống có thể được tách thành một phần cấp phát và một phần còn lại tiếp tục được quản lý như vùng nhớ trống:

$$H_{còn\ lại} = H - R.$$

Khi tiến trình kết thúc, vùng nhớ của nó được thu hồi. Nếu vùng vừa giải phóng nằm liền kề các vùng trống khác, hệ điều hành có thể gộp chúng thành một vùng lớn hơn. Việc gộp các vùng trống liền kề giúp đáp ứng những yêu cầu có kích thước lớn hơn mà từng vùng riêng lẻ không thể đáp ứng.

Nếu không có vùng trống nào đủ lớn, hệ điều hành có thể từ chối yêu cầu hoặc đưa tiến trình vào hàng đợi để chờ bộ nhớ được giải phóng. Vì vậy, lựa chọn vùng trống phù hợp có ảnh hưởng đến cả tốc độ cấp phát và khả năng phục vụ các yêu cầu tiếp theo (Silberschatz et al., 2018).

2.2.2. Chiến lược First-fit

First-fit lựa chọn vùng trống đầu tiên có kích thước đủ đáp ứng yêu cầu. Hệ thống duyệt danh sách theo thứ tự tìm kiếm và dừng ngay khi gặp một vùng phù hợp, không cần tiếp tục kiểm tra các vùng còn lại.

Ưu điểm của First-fit là cách thực hiện đơn giản và thường có chi phí tìm kiếm thấp. Nếu vùng phù hợp xuất hiện sớm trong danh sách, việc cấp phát có thể được quyết định sau một số ít lần kiểm tra.

Tuy nhiên, kết quả lựa chọn phụ thuộc vào thứ tự quản lý các vùng trống. Vùng được chọn đầu tiên chưa chắc là vùng có kích thước gần nhất với yêu cầu. Sau nhiều lần cấp phát và thu hồi, phần đầu danh sách có thể xuất hiện nhiều vùng trống nhỏ, khiến những lần tìm kiếm tiếp theo phải kiểm tra nhiều vùng hơn.

Do đó, First-fit ưu tiên tìm được vùng đủ dùng nhanh chóng, thay vì tìm vùng có phần dư nhỏ nhất. Hiệu quả thực tế của chiến lược phụ thuộc vào trạng thái bộ nhớ và cách tổ chức danh sách vùng trống (Silberschatz et al., 2018).

2.2.3. Chiến lược Best-fit

Best-fit lựa chọn vùng trống nhỏ nhất trong số những vùng đủ đáp ứng yêu cầu. Với yêu cầu kích thước **R**, chiến lược tìm vùng **H** thỏa mãn:

$$H \geq R$$

và làm cho phần dư **H - R** nhỏ nhất.

Cách lựa chọn này hạn chế việc sử dụng một vùng quá lớn cho một yêu cầu nhỏ, qua đó giữ lại các vùng lớn cho những yêu cầu có thể xuất hiện sau đó.

Tuy nhiên, để tìm vùng phù hợp nhất, hệ thống thường phải kiểm tra toàn bộ danh sách, trừ khi sử dụng cách tổ chức dữ liệu hỗ trợ tìm kiếm hiệu quả hơn. Nếu danh sách được sắp xếp theo kích thước tăng dần, vùng đầu tiên đủ lớn có thể được chọn, nhưng hệ thống vẫn phải chịu chi phí duy trì thứ tự khi cấp phát và thu hồi.

Một hạn chế khác là Best-fit có thể để lại những phần dư rất nhỏ, khó đáp ứng các yêu cầu tiếp theo. Vì vậy, tên gọi "phù hợp nhất" chỉ thể hiện tiêu chí phần dư nhỏ nhất trong lần cấp phát hiện tại, không có nghĩa chiến lược luôn sử dụng bộ nhớ hiệu quả nhất trong suốt quá trình hoạt động (Silberschatz et al., 2018).

2.2.4. Chiến lược Worst-fit

Worst-fit lựa chọn vùng trống lớn nhất để cấp phát. Ý tưởng của chiến lược là sau khi đáp ứng yêu cầu, phần dư vẫn còn tương đối lớn và có thể tiếp tục phục vụ các tiến trình khác.

Để xác định vùng lớn nhất, hệ thống thường phải duyệt toàn bộ danh sách nếu chưa có cấu trúc dữ liệu hỗ trợ phù hợp. Do đó, chi phí tìm kiếm có thể cao hơn First-fit.

Hạn chế của Worst-fit là thường xuyên chia nhỏ các vùng trống lớn nhất. Điều này có thể làm giảm khả năng đáp ứng những yêu cầu lớn xuất hiện về sau. Mặc dù phần dư của từng lần cấp phát tương đối lớn, lợi ích đó không bảo đảm hiệu quả sử dụng bộ nhớ trên toàn bộ chuỗi yêu cầu. Các kết quả mô phỏng được trình bày trong

giáo trình cho thấy First-fit và Best-fit thường có kết quả tốt hơn Worst-fit khi xét thời gian và mức độ sử dụng bộ nhớ (Silberschatz et al., 2018).

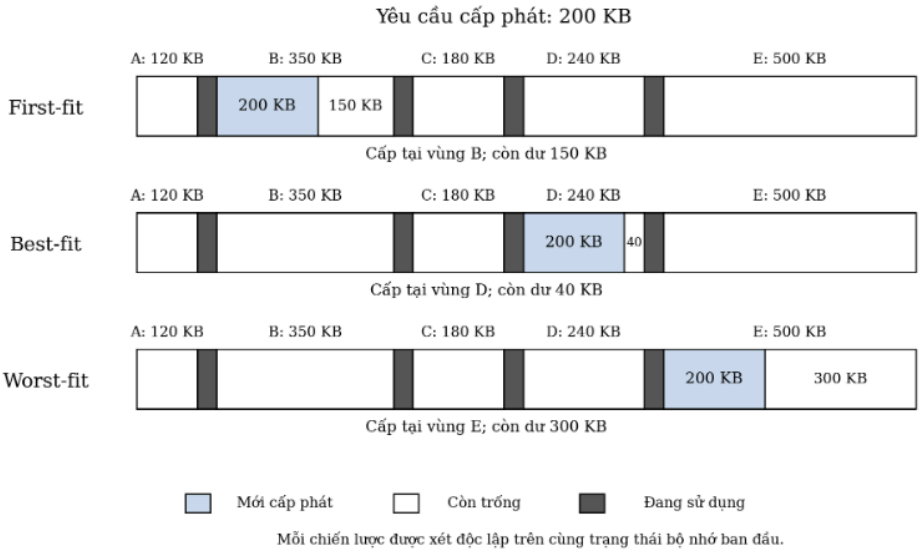
2.2.5. Ví dụ minh họa

Giả sử bộ nhớ có năm vùng trống theo thứ tự như sau:

Vùng trống	A	B	C	D	E
Kích thước (KB)	120	350	180	240	500

Một tiến trình yêu cầu 200 KB. Xét từng chiến lược độc lập trên cùng trạng thái bộ nhớ ban đầu, kết quả là:

Chiến lược	Vùng được chọn	Phần dư	Giải thích
First-fit	B: 350 KB	150 KB	B là vùng đầu tiên đủ chứa 200 KB.
Best-fit	D: 240 KB	40 KB	D là vùng nhỏ nhất trong các vùng đủ lớn.
Worst-fit	E: 500 KB	300 KB	E là vùng trống lớn nhất.



Hình ảnh minh họa so sánh lựa chọn vùng nhớ của First-fit, Best-fit và Worst-fit xây dựng bằng diagrams.net (draw.io), dựa trên Silberschatz et al. (2018).

Ví dụ cho thấy cùng một yêu cầu có thể dẫn đến những lựa chọn khác nhau. First-fit dừng tìm kiếm sớm, Best-fit để lại phần dư nhỏ nhất, còn Worst-fit tạo ra phần dư lớn nhất.

Để thấy rõ ảnh hưởng đến yêu cầu tiếp theo, giả sử ngay sau đó có một tiến trình cần 450 KB, không có hoạt động giải phóng bộ nhớ xen giữa. Với First-fit và Best-fit, vùng E có kích thước 500 KB vẫn còn nguyên nên đáp ứng được yêu cầu. Với Worst-fit, vùng E đã được chia thành phần cấp phát 200 KB và phần dư 300 KB; các vùng còn lại cũng đều nhỏ hơn 450 KB nên yêu cầu mới chưa thể được đáp ứng. Kết quả này minh họa một hạn chế của Worst-fit trong tình huống cụ thể, không chứng minh chiến lược đó luôn kém hơn với mọi chuỗi yêu cầu.

2.2.6. So sánh và đánh giá

Ba chiến lược khác nhau chủ yếu ở tiêu chí lựa chọn vùng trống và mức độ tìm kiếm cần thực hiện.

Tiêu chí	First-fit	Best-fit	Worst-fit
Vùng được chọn	Vùng đầu tiên đủ lớn	Vùng nhỏ nhất đủ lớn	Vùng lớn nhất
Tìm kiếm trên danh sách chưa sắp xếp	Dừng khi tìm được vùng phù hợp	Thường phải duyệt toàn bộ	Thường phải duyệt toàn bộ
Đặc điểm phần dư	Phụ thuộc vùng gặp đầu tiên	Nhỏ nhất trong lần cấp phát	Lớn nhất trong lần cấp phát
Hạn chế đáng chú ý	Phụ thuộc thứ tự tìm kiếm	Có thể tạo nhiều phần dư rất nhỏ	Có thể làm mất các vùng lớn cần cho yêu cầu sau

Không có chiến lược nào bảo đảm tối ưu cho mọi chuỗi yêu cầu. Hiệu quả còn phụ thuộc vào kích thước yêu cầu, thứ tự xuất hiện, thời điểm thu hồi và cách quản lý vùng trống. Theo Silberschatz et al. (2018), First-fit thường nhanh hơn Best-fit, trong khi chưa thể khẳng định một trong hai luôn tốt hơn về mức độ sử dụng bộ nhớ.

Cả ba chiến lược đều lựa chọn dựa trên trạng thái bộ nhớ hiện tại, không dự đoán đầy đủ các yêu cầu tương lai. Vì vậy, việc đánh giá cần xem xét cả chi phí tìm kiếm lẫn khả năng đáp ứng yêu cầu về sau. Bên cạnh đó, cấp phát liên tục với phân vùng có kích thước thay đổi vẫn có thể làm các vùng trống bị chia nhỏ và phân tán, dẫn đến vấn đề phân mảnh được trình bày ở mục tiếp theo.

2.3. Phân mảnh bộ nhớ và dồn nén

Trong quá trình cấp phát và thu hồi bộ nhớ, dung lượng còn lại không phải lúc nào cũng được sử dụng hiệu quả. Các vùng trống có thể bị chia nhỏ và nằm xen giữa

những vùng đang sử dụng; đồng thời, một phần dung lượng bên trong vùng đã cấp có thể vượt quá nhu cầu thực tế của tiến trình. Hai hiện tượng này lần lượt được gọi là phân mảnh ngoài và phân mảnh trong. Chúng đều làm giảm hiệu quả sử dụng bộ nhớ nhưng khác nhau về nguyên nhân, vị trí phát sinh và phương pháp khắc phục (Silberschatz et al., 2018).

2.3.1. Phân mảnh ngoài

Phân mảnh ngoài (*external fragmentation*) là tình trạng bộ nhớ trống bị phân tán thành các vùng không liên tục. Hệ quả là một yêu cầu cấp phát liên tục có thể không được đáp ứng dù tổng dung lượng trống vẫn đủ.

Hiện tượng này hình thành từ sự xen kẽ giữa các hoạt động cấp phát và thu hồi. Khi một tiến trình kết thúc, vùng nhớ của nó được giải phóng. Tuy nhiên, nếu vùng vừa giải phóng bị ngăn cách với những vùng trống khác bởi các tiến trình đang hoạt động, chúng không thể được gộp ngay. Các yêu cầu tiếp theo tiếp tục chia nhỏ những vùng trống hiện có, khiến bộ nhớ dần xuất hiện nhiều khoảng trống với kích thước khác nhau (Silberschatz et al., 2018).

Gọi **R** là dung lượng yêu cầu, H_i là kích thước vùng trống thứ **i** và **n** là số vùng trống. Trường hợp không đáp ứng được yêu cầu do phân mảnh ngoài được biểu diễn như sau:

$$\sum_{i=1}^{n} H_i \geq R \qquad nhưng \qquad \left(H_1, H_2, \ldots, H_n\right) < R.$$

Biểu thức cho thấy cần phân biệt tổng dung lượng trống với kích thước vùng trống liên tục lớn nhất. Một hệ thống có nhiều bộ nhớ trống vẫn có thể phải trì hoãn yêu cầu nếu không tồn tại vùng liên tục đủ lớn.

Các chiến lược lựa chọn vùng trống ảnh hưởng đến cách phân mảnh hình thành. First-fit ưu tiên vùng đầu tiên đủ lớn, còn Best-fit chọn vùng nhỏ nhất đáp ứng yêu cầu. Best-fit có thể giữ lại các vùng lớn nhưng cũng dễ để lại những phần dư quá nhỏ; First-fit thường tìm kiếm nhanh hơn nhưng không bảo đảm phân bố vùng trống

thuận lợi cho các yêu cầu sau. Vì vậy, cả hai đều có thể gặp phân mảnh ngoài và không có chiến lược nào luôn tốt hơn trong mọi tình huống (Silberschatz et al., 2018).

Sách còn đề cập quy tắc 50% đối với First-fit: dưới những giả định thống kê nhất định, với N khối đã cấp phát, có khoảng 0,5N khối trống do phân mảnh. Đây là kết quả phân tích thống kê trong những điều kiện nhất định, không phải bảo đảm cho mọi trạng thái bộ nhớ. Nhận xét này cho thấy số khoảng trống phát sinh có thể đáng kể ngay cả khi quá trình cấp phát được tổ chức tương đối hiệu quả. Tuy nhiên, không thể trực tiếp suy ra tỉ lệ dung lượng bị lãng phí chỉ từ số lượng khối nếu chưa xét kích thước của chúng.

2.3.2. Phân mảnh trong

Phân mảnh trong (*internal fragmentation*) là phần dung lượng không được sử dụng nằm bên trong vùng đã cấp phát. Nguyên nhân trực tiếp là kích thước vùng cấp lớn hơn nhu cầu thực tế của tiến trình.

Gọi **A** là dung lượng được cấp và **R** là dung lượng cần sử dụng. Lượng phân mảnh trong là:

$$F_{trong} = A - R.$$

Ví dụ, nếu hệ thống cấp phát theo bội số của 8 KB, một yêu cầu 29 KB sẽ được làm tròn lên 32 KB. Phần dư bên trong vùng cấp phát là:

$$F_{trong} = 32 - 29 = 3\,KB.$$

Phần dư này không thuộc danh sách vùng nhớ trống nên chưa thể cấp cho tiến trình khác.

Phân mảnh trong không chỉ xuất hiện do đơn vị cấp phát cố định. Trong mô hình phân vùng có kích thước thay đổi, hệ thống có thể cấp toàn bộ một vùng trống nếu phần dư sau khi tách quá nhỏ. Chẳng hạn, nếu một vùng 1.024 byte được cấp toàn bộ cho yêu cầu 1.020 byte thì có 4 byte dư. Khi chi phí quản lý một vùng trống riêng lớn hơn giá trị sử dụng của phần dư, việc cấp cả vùng có thể hợp lý hơn (Silberschatz et al., 2018).

Điều đó thể hiện sự đánh đổi giữa tiết kiệm dung lượng và chi phí quản lý. Cấp sát nhu cầu làm giảm phần dư trong từng vùng nhưng có thể tạo thêm nhiều vùng nhỏ cần theo dõi. Ngược lại, làm tròn hoặc bỏ qua việc tách phần dư quá nhỏ giúp đơn giản hóa quản lý nhưng làm tăng phân mảnh trong. Vì vậy, mục tiêu thực tế là hạn chế lãng phí ở mức phù hợp với chi phí xử lý, thay vì luôn cố gắng loại bỏ mọi byte dư.

2.3.3. Ví dụ phân tích quá trình hình thành phân mảnh ngoài

Giả sử một vùng nhớ 1.000 KB dành cho các tiến trình có địa chỉ tương đối từ 0 đến dưới 1.000 KB. Các giá trị địa chỉ dưới đây đều tính theo KB. Bỏ qua chi phí quản lý và yêu cầu căn chỉnh để tập trung phân tích cơ chế cấp phát. Ban đầu, các tiến trình được nạp liên tiếp như sau:

Thành phần	Phạm vi địa chỉ tương đối (KB)	Kích thước
Tiến trình A	$0 \leq x < 200$	200 KB
Tiến trình B	$200 \leq x < 350$	150 KB
Tiến trình C	$350 \leq x < 600$	250 KB
Tiến trình D	$600 \leq x < 800$	200 KB
Vùng trống	$800 \leq x < 1\,000$	200 KB

Sau đó, B kết thúc và giải phóng 150 KB. Giả sử hệ thống sử dụng First-fit, duyệt theo địa chỉ tăng dần, một tiến trình E yêu cầu 100 KB sẽ được cấp vào phần đầu vùng vừa giải phóng. Vùng này còn dư 50 KB. Tiếp theo, D kết thúc; vùng 200 KB của D được gộp với vùng trống 200 KB ở cuối bộ nhớ.

Trạng thái bộ nhớ lúc này là:

Thành phần	Phạm vi địa chỉ tương đối (KB)	Kích thước
Tiến trình A	$0 \leq x < 200$	200 KB
Tiến trình E	$200 \leq x < 300$	100 KB
Vùng trống thứ nhất	$300 \leq x < 350$	50 KB
Tiến trình C	$350 \leq x < 600$	250 KB
Vùng trống thứ hai	$600 \leq x < 1\,000$	400 KB

Tổng dung lượng trống là:

$$H_{tổng} = 50 + 400 = 450\ KB.$$

Nếu tiến trình F yêu cầu 430 KB, hệ thống chưa thể cấp phát liên tục vì:

$$450 \geq 430 \quad nhưng \quad 400 < 430.$$

Như vậy, tổng dung lượng trống đủ đáp ứng yêu cầu, nhưng cách bố trí hiện tại chưa cho phép cấp phát liên tục.

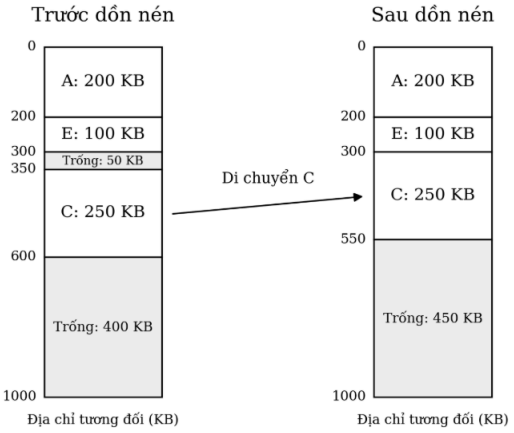
Ví dụ cũng cho thấy việc gộp vùng trống liền kề đã được thực hiện khi D kết thúc nhưng chưa giải quyết hoàn toàn phân mảnh. Hai vùng 50 KB và 400 KB vẫn bị tiến trình C ngăn cách nên không thể kết hợp nếu giữ nguyên vị trí các tiến trình.

2.3.4. Dồn nén bộ nhớ và điều kiện thực hiện

Dồn nén bộ nhớ (*compaction*) là quá trình di chuyển các vùng đang sử dụng để tập hợp những khoảng trống rải rác thành một vùng liên tục lớn hơn. Dồn nén làm thay đổi cách bố trí bộ nhớ, không làm tăng tổng dung lượng trống (Silberschatz et al., 2018).

Trong ví dụ trên, hệ điều hành có thể chuyển tiến trình C từ vị trí bắt đầu 350 KB xuống vị trí bắt đầu 300 KB. Sau khi di chuyển, bộ nhớ được bố trí như sau:

Thành phần	Phạm vi địa chỉ tương đối (KB)	Kích thước
Tiến trình A	$0 \leq x < 200$	200 KB
Tiến trình E	$200 \leq x < 300$	100 KB
Tiến trình C	$300 \leq x < 550$	250 KB
Vùng trống liên tục	$550 \leq x < 1\,000$	450 KB



Hình ảnh minh họa trạng thái bộ nhớ trước và sau dồn nén được xây dựng bằng diagrams.net (draw.io), dựa trên Silberschatz et al. (2018).

Hai khoảng trống ban đầu được tập hợp thành một vùng 450 KB. Tiến trình F cần 430 KB có thể được cấp phát, để lại:

$$450 – 430 = 20 \text{ KB}$$

Để thực hiện việc di chuyển này, hệ thống phải hỗ trợ tái định vị động trong quá trình thực thi. Trong mô hình sử dụng thanh ghi tái định vị, địa chỉ logic của tiến trình C được giữ nguyên, còn địa chỉ nền được cập nhật theo vị trí mới. Nếu địa chỉ bắt đầu thực tế của toàn vùng nhớ đang xét là M, giá trị nền của C thay đổi từ M + 350 KB thành M + 300 KB.

Hệ điều hành cần tạm dừng tiến trình liên quan, di chuyển nội dung bộ nhớ và cập nhật thông tin tái định vị trước khi cho tiến trình tiếp tục chạy. Nếu địa chỉ chỉ được cố định tại thời điểm biên dịch hoặc nạp chương trình và không có hỗ trợ tái định vị động, hệ thống không thể trực tiếp áp dụng cơ chế dồn nén này trong lúc chạy (Silberschatz et al., 2018).

Cần phân biệt dồn nén với hai thao tác khác:

- Gộp vùng trống liền kề: kết hợp các vùng trống vốn nằm cạnh nhau, không di chuyển tiến trình.
- Nén dữ liệu trong bộ nhớ: thay đổi cách biểu diễn dữ liệu để giảm dung lượng lưu trữ, thay vì di chuyển các vùng để gom khoảng trống.

2.3.5. Chi phí dồn nén và lựa chọn biện pháp khắc phục

Dồn nén có thể tạo ra vùng liên tục đủ lớn nhưng phải trả chi phí di chuyển dữ liệu. Chi phí này phụ thuộc vào lượng bộ nhớ cần sao chép, tốc độ hệ thống nhớ và công việc cập nhật trạng thái quản lý.

Trong ví dụ, tiến trình C chỉ dịch xuống 50 KB nhưng có kích thước 250 KB. Vì vậy, khoảng cách di chuyển không đồng nghĩa với lượng dữ liệu cần di chuyển: hệ thống vẫn phải di chuyển nội dung vùng nhớ 250 KB của C. Do vùng nguồn và vùng đích chồng lấn, thao tác sao chép còn phải bảo đảm dữ liệu chưa được chuyển không bị ghi đè.

Nếu gọi V là lượng dữ liệu phải di chuyển và S là tốc độ di chuyển dữ liệu hiệu dụng, thời gian sao chép có thể ước lượng sơ bộ bằng:

$$T_{sao\ chép} \approx \frac{V}{S}.$$

Đây là mô hình ước lượng đơn giản; thời gian thực tế còn bao gồm việc tạm dừng tiến trình, cập nhật thông tin địa chỉ và các chi phí phối hợp khác. Vì vậy, dồn nén quá thường xuyên có thể tiêu tốn thời gian xử lý ngay cả khi giúp bố trí bộ nhớ gọn hơn. Nếu tổng dung lượng trống vẫn nhỏ hơn yêu cầu, việc dồn nén cũng không thể đáp ứng yêu cầu đó.

Việc lựa chọn biện pháp cần dựa trên nguyên nhân cụ thể:

Tình trạng	Biện pháp phù hợp	Giới hạn
Các vùng trống nằm liền kề	Gộp vùng trống	Không kết hợp được các vùng bị tiến trình ngăn cách
Tổng dung lượng đủ nhưng vùng liên tục lớn nhất quá nhỏ	Cân nhắc dồn nén	Cần hỗ trợ tái định vị động và chịu chi phí di chuyển
Tổng dung lượng trống không đủ	Chờ giải phóng bộ nhớ hoặc sử dụng cơ chế quản lý khác	Dồn nén không tạo thêm dung lượng
Vùng cấp phát lớn hơn nhu cầu	Điều chỉnh đơn vị hoặc chính sách cấp phát	Có thể làm tăng chi phí quản lý

Một hướng xử lý khác đối với phân mảnh ngoài là cho phép tiến trình sử dụng những vùng nhớ vật lý không liên tục. Phân trang thực hiện điều này bằng cách bố trí các trang vào những khung còn trống. Cơ chế đó loại bỏ phân mảnh ngoài ở cấp cấp phát từng khung trang, nhưng vẫn có thể gây phân mảnh trong khi khung được cấp không được sử dụng hết (Silberschatz et al., 2018).

Đối với phân mảnh trong, giảm đơn vị cấp phát có thể hạn chế phần dư nhưng thường làm tăng số lượng đơn vị cần theo dõi. Do đó, hiệu quả quản lý bộ nhớ cần được đánh giá đồng thời theo dung lượng sử dụng, khả năng đáp ứng yêu cầu và chi phí xử lý, thay vì chỉ dựa vào tổng bộ nhớ còn trống.

2.4. Nén bộ nhớ

Nén bộ nhớ (*memory compression*) là kỹ thuật giảm dung lượng RAM mà dữ liệu chiếm dụng bằng các thuật toán nén. Khi số khung trang trống giảm, thay vì chuyển ngay các trang ra vùng hoán đổi trên thiết bị lưu trữ, hệ điều hành có thể nén một số trang và giữ chúng trong RAM. Cách tiếp cận này giúp giải phóng khung trang để đáp ứng những yêu cầu khác, đồng thời giảm nhu cầu truy cập bộ nhớ phụ (Silberschatz et al., 2018).

2.4.1. Nguyên lý hoạt động

Hệ điều hành lựa chọn các trang phù hợp để nén và lưu dữ liệu đã nén trong một vùng được quản lý trong RAM. Do dữ liệu sau nén có kích thước nhỏ hơn, nội dung của nhiều trang có thể được lưu trong số khung ít hơn ban đầu. Những khung được giải phóng sẽ được trả về danh sách khung trống.

Khi tiến trình truy cập lại một trang đã nén, hệ thống phải khôi phục nội dung trang trước khi tiếp tục phục vụ truy cập. Trong cơ chế được giáo trình mô tả, truy cập này gây ra lỗi trang; hệ điều hành tìm dữ liệu đã nén, giải nén và khôi phục trang vào bộ nhớ (Silberschatz et al., 2018).

Nén bộ nhớ khác với dồn nén đã trình bày ở mục 2.3. Dồn nén di chuyển các vùng đang sử dụng để gom khoảng trống liên tục, còn nén bộ nhớ thay đổi cách biểu diễn dữ liệu để giảm dung lượng chiếm dụng. Kỹ thuật này không làm tăng dung lượng RAM vật lý mà giúp RAM hiện có lưu được nhiều dữ liệu hơn. Chẳng hạn, nếu dữ liệu của bốn trang được nén để lưu trong hai khung trang cùng kích thước, hệ thống giải phóng ròng hai khung, không tính chi phí quản lý.

2.4.2. Lợi ích và hạn chế

Lợi ích chính của nén bộ nhớ là giảm lượng dữ liệu phải chuyển ra thiết bị lưu trữ khi RAM chịu áp lực. Nếu chi phí nén và giải nén thấp hơn chi phí thực hiện I/O tương ứng, kỹ thuật này có thể cải thiện thời gian phản hồi và giúp hệ thống duy trì nhiều dữ liệu trong RAM hơn (Silberschatz et al., 2018).

Tuy nhiên, hiệu quả nén phụ thuộc vào đặc điểm dữ liệu. Dữ liệu có nhiều mẫu lặp thường có khả năng nén tốt hơn dữ liệu vốn đã được nén. Quá trình nén và giải nén cũng sử dụng thời gian CPU, trong khi dữ liệu nén và thông tin quản lý vẫn chiếm một phần RAM. Nếu các trang thường xuyên bị nén rồi được truy cập lại, chi phí xử lý có thể làm giảm lợi ích thu được.

Vì vậy, nén bộ nhớ là biện pháp bổ trợ cho quản lý bộ nhớ ảo, không thay thế hoàn toàn các cơ chế khác. Hiệu quả của nó phụ thuộc vào khả năng nén dữ liệu, tần suất truy cập lại và sự cân bằng giữa chi phí CPU với chi phí truy cập thiết bị lưu trữ.

2.5. Cấp phát bộ nhớ cho nhân hệ điều hành

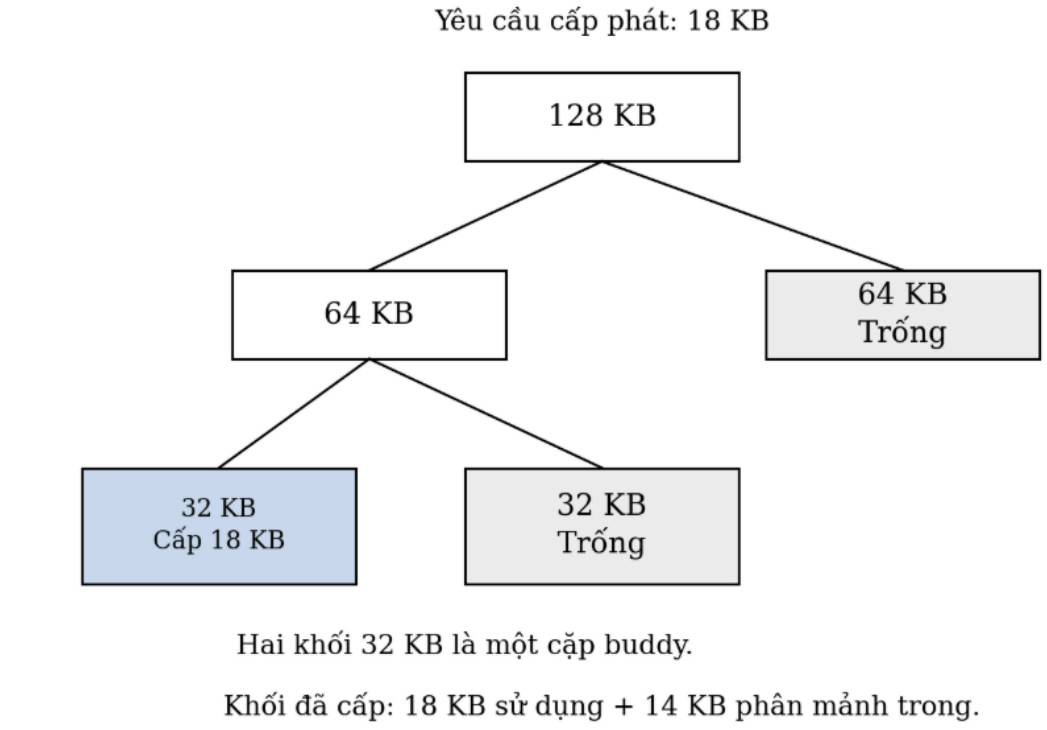
Bộ nhớ dành cho nhân hệ điều hành có những yêu cầu khác với bộ nhớ dành cho tiến trình người dùng. Nhân thường cần cấp phát các cấu trúc dữ liệu có kích thước khác nhau, trong đó nhiều cấu trúc nhỏ hơn một trang. Bên cạnh đó, một số thiết bị phần cứng yêu cầu vùng nhớ vật lý liên tục để thực hiện trao đổi dữ liệu. Vì vậy, cơ chế cấp phát cần vừa hạn chế lãng phí, vừa đáp ứng nhanh các yêu cầu này. Hai phương pháp được trình bày trong phần này là Buddy system và Slab allocation (Silberschatz et al., 2018).

2.5.1. Hệ thống Buddy

Hệ thống Buddy (*Buddy system*) quản lý bộ nhớ bằng các khối có kích thước là lũy thừa của 2. Khi có yêu cầu, hệ thống chọn kích thước khối nhỏ nhất trong các mức được hỗ trợ nhưng vẫn đủ chứa dung lượng cần cấp. Nếu chưa có khối phù hợp, một khối lớn hơn được chia đôi liên tiếp cho đến khi đạt kích thước cần thiết (Silberschatz et al., 2018).

Hai khối được tạo ra từ cùng một lần chia đôi được gọi là hai khối *buddy*. Khi cả hai đều được giải phóng, chúng có thể được gộp lại thành khối lớn ban đầu. Quá trình gộp tiếp tục ở các mức lớn hơn nếu khối buddy tương ứng cũng đang trống. Không phải hai khối bất kỳ có cùng kích thước và nằm cạnh nhau đều có thể gộp; chúng phải là cặp được tạo từ cùng một khối cha.

Chẳng hạn, với vùng nhớ 128 KB và yêu cầu 18 KB, hệ thống chia khối 128 KB thành hai khối 64 KB, sau đó chia một khối 64 KB thành hai khối 32 KB. Một khối 32 KB được cấp cho yêu cầu, để lại 14 KB phân mảnh trong. Khi khối này được giải phóng, nó có thể gộp với khối buddy 32 KB nếu khối đó cũng trống.



Hình ảnh chia khối bộ nhớ theo hệ thống Buddy, xây dựng bằng diagrams.net (draw.io), dựa trên Silberschatz et al. (2018).

Ưu điểm của Buddy system là cách tổ chức có quy luật, thuận tiện cho việc chia và gộp khối. Tuy nhiên, yêu cầu phải làm tròn lên kích thước lũy thừa của 2 có thể gây phân mảnh trong đáng kể, đặc biệt khi dung lượng yêu cầu chỉ lớn hơn một chút so với mức kích thước nhỏ hơn.

2.5.2. Cấp phát Slab

Cấp phát Slab (*Slab allocation*) tổ chức bộ nhớ theo các loại đối tượng của nhân hệ điều hành, chẳng hạn đối tượng mô tả tiến trình, đối tượng tệp hoặc semaphore. Thay vì cấp một khối lớn cho từng yêu cầu nhỏ, phương pháp này chuẩn

bị các ô nhớ phù hợp với kích thước của từng loại đối tượng để sử dụng và tái sử dụng (Silberschatz et al., 2018).

Cơ chế này gồm ba thành phần:

- Đối tượng (*object*): một thể hiện của cấu trúc dữ liệu nhân.
- Slab: một hoặc nhiều trang vật lý liên tục, được chia thành các ô chứa đối tượng.
- Cache: tập hợp một hoặc nhiều slab phục vụ một loại đối tượng nhất định. Cache ở đây là vùng quản lý đối tượng, không phải bộ nhớ đệm phần cứng của CPU.

Khi nhân cần một đối tượng, bộ cấp phát lấy một ô trống từ cache tương ứng và đánh dấu đang sử dụng. Khi đối tượng được giải phóng, ô nhớ được đánh dấu trống để phục vụ yêu cầu tiếp theo. Việc tái sử dụng giúp giảm chi phí cấp phát và khởi tạo lặp lại.

Theo trạng thái sử dụng các ô đối tượng, slab được chia thành ba loại:

Trạng thái	Đặc điểm
Đầy (*full*)	Tất cả các ô đối tượng đang được sử dụng.
Trống (*empty*)	Tất cả các ô đối tượng chưa được sử dụng.
Một phần (*partial*)	Có cả ô đang sử dụng và ô còn trống.

Bộ cấp phát ưu tiên lấy đối tượng từ slab đang sử dụng một phần. Nếu không có, hệ thống sử dụng slab trống; khi cần thiết mới cấp thêm slab cho cache.

Slab phù hợp với những đối tượng được tạo và giải phóng thường xuyên. Việc chia ô theo kích thước đối tượng giúp hạn chế lãng phí do làm tròn như trong Buddy system. Tuy nhiên, không nên hiểu rằng Slab loại bỏ mọi chi phí bộ nhớ: thông tin quản lý, yêu cầu căn chỉnh và các ô chưa sử dụng trong slab vẫn có thể chiếm dung lượng.

2.5.3. So sánh hai phương pháp

Buddy system và Slab khác nhau về đơn vị quản lý và cách đáp ứng yêu cầu.

Tiêu chí	Buddy system	Slab allocation
Đơn vị cấp phát	Khối có kích thước là lũy thừa của 2	Ô nhớ dành cho một loại đối tượng
Cách tổ chức	Chia đôi khối lớn, gộp các cặp buddy trống	Quản lý đối tượng trong cache và slab
Ưu điểm	Thuận tiện cho việc chia và gộp khối	Cấp phát nhanh, tái sử dụng đối tượng hiệu quả
Hạn chế	Có thể gây phân mảnh trong do làm tròn	Có chi phí quản lý và dung lượng dành cho các ô chưa dùng
Nhu cầu phù hợp	Cấp phát các khối nhớ vật lý liên tục	Cấp phát thường xuyên các đối tượng nhân có kích thước xác định

Hai phương pháp có thể bổ trợ cho nhau ở các mức quản lý khác nhau. Buddy system đáp ứng nhu cầu về các khối trang, còn Slab tổ chức bộ nhớ thành các ô phù hợp cho đối tượng nhân. Sự kết hợp này giúp cân bằng giữa tốc độ cấp phát, khả năng tái sử dụng và hiệu quả sử dụng bộ nhớ.

2.6. Một số vấn đề khác trong quản lý bộ nhớ

Bên cạnh thuật toán thay thế trang và chính sách phân phối khung trang, hoạt động quản lý bộ nhớ còn phụ thuộc vào cách nạp trang, kích thước trang, khả năng hỗ trợ chuyển đổi địa chỉ và đặc điểm truy cập của chương trình. Ngoài ra, hệ điều hành phải bảo đảm các trang phục vụ thao tác nhập/xuất không bị thay thế vào thời điểm không phù hợp. Những yếu tố này ảnh hưởng đến số lỗi trang, chi phí xử lý và tính đúng đắn của hệ thống (Silberschatz et al., 2018).

2.6.1. Tải trước trang

Tải trước trang (*prepaging*) là việc đưa vào bộ nhớ một số trang được dự đoán sẽ cần dùng trước khi tiến trình thực sự yêu cầu chúng. Mục đích là giảm số lỗi trang, đặc biệt khi tiến trình mới bắt đầu hoặc tiếp tục thực thi sau thời gian tạm dừng.

Trong phân trang theo yêu cầu, những trang chưa có trong RAM phải được nạp khi được truy cập. Nếu nhiều trang cần thiết đều chưa hiện diện, tiến trình có thể liên tục gặp lỗi trang ở giai đoạn đầu. Tải trước hạn chế tình trạng này bằng cách nạp một nhóm trang trong cùng đợt. Chẳng hạn, hệ điều hành có thể ghi nhớ tập làm việc của

tiến trình trước khi tạm dừng và nạp lại các trang đó trước khi cho tiến trình tiếp tục chạy (Silberschatz et al., 2018).

Hiệu quả của phương pháp phụ thuộc vào độ chính xác của dự đoán. Nếu phần lớn các trang nạp trước được sử dụng, hệ thống tránh được nhiều lỗi trang. Ngược lại, những trang không được dùng sẽ chiếm RAM và gây ra hoạt động I/O không cần thiết. Vì vậy, tải trước cần được cân nhắc dựa trên chi phí nạp thêm so với chi phí xử lý lỗi trang có thể tránh được.

2.6.2. Kích thước trang

Kích thước trang (*page size*) ảnh hưởng đồng thời đến dung lượng bảng trang, phân mảnh trong, hiệu quả I/O và mức độ khai thác tính cục bộ. Không có một kích thước phù hợp nhất cho mọi hệ thống vì các yếu tố này có thể dẫn đến những lựa chọn khác nhau (Silberschatz et al., 2018).

Với cùng một không gian địa chỉ, trang nhỏ làm tăng số trang và số mục cần quản lý trong bảng trang. Ngược lại, trang lớn giúp giảm kích thước bảng trang nhưng có thể làm tăng phần dung lượng không sử dụng trong trang cuối của vùng được cấp.

Trang nhỏ cho phép nạp sát với phần dữ liệu thực sự cần, hạn chế đưa dữ liệu không sử dụng vào RAM. Tuy nhiên, hệ thống có thể phải xử lý nhiều lỗi trang hơn. Trong khi đó, trang lớn đưa được nhiều dữ liệu vào sau mỗi lần nạp, nhưng lợi ích phụ thuộc vào việc chương trình có sử dụng phần dữ liệu đó hay không.

Đối với HDD, chi phí tìm kiếm và độ trễ thường đáng kể so với thời gian truyền dữ liệu. Vì vậy, truyền một lượng dữ liệu lớn trong một lần có thể hiệu quả hơn chia thành nhiều lần truyền nhỏ. Đây là một yếu tố ủng hộ việc sử dụng trang lớn, song cần cân đối với phân mảnh trong và đặc điểm truy cập của chương trình.

Yếu tố	Trang nhỏ	Trang lớn
Kích thước bảng trang	Thường lớn hơn	Thường nhỏ hơn
Phân mảnh trong	Thường ít hơn	Thường nhiều hơn
Mức độ nạp sát dữ liệu cần dùng	Cao hơn	Có thể nạp thêm dữ liệu không cần
Số lần xử lý lỗi trang	Có thể nhiều hơn	Có thể giảm nếu dữ liệu được sử dụng

Hiệu quả truyền dữ liệu trên HDD	Chi phí tìm kiếm và độ trễ được phân bổ cho ít dữ liệu hơn	Có thể tận dụng tốt hơn mỗi lần I/O

2.6.3. Phạm vi bao phủ của TLB

TLB (*Translation Lookaside Buffer*) là bộ đệm lưu một số ánh xạ từ trang ảo sang khung trang vật lý, giúp giảm thời gian chuyển đổi địa chỉ. Phạm vi bao phủ của TLB (*TLB reach*) là dung lượng bộ nhớ mà các mục ánh xạ trong TLB có thể bao phủ.

Khi các mục TLB sử dụng cùng kích thước trang, phạm vi này được xác định bằng:

TLB reach = Số mục TLB x Kích thước trang.

Ví dụ, TLB có 64 mục, mỗi mục ánh xạ một trang 8 KB, thì phạm vi bao phủ là:

64 x 8 = 512 KB.

Nếu các ánh xạ của những trang thuộc tập làm việc được duy trì trong TLB, nhiều yêu cầu chuyển đổi địa chỉ có thể được xử lý mà không cần tra bảng trang. Tuy nhiên, phạm vi bao phủ đủ lớn chưa tự bảo đảm mọi truy cập đều trúng TLB vì còn phụ thuộc vào cách tổ chức, thứ tự truy cập và chính sách thay thế các mục.

Có thể tăng TLB reach bằng cách tăng số mục TLB hoặc tăng kích thước trang. Cách thứ nhất làm tăng chi phí phần cứng và năng lượng; cách thứ hai có thể làm tăng phân mảnh trong. Hỗ trợ nhiều kích thước trang là một hướng dung hòa, cho phép sử dụng trang lớn đối với những vùng nhớ phù hợp (Silberschatz et al., 2018).

2.6.4. Bảng trang đảo ngược

Bảng trang đảo ngược (*inverted page table*) tổ chức thông tin theo các khung trang vật lý. Mỗi mục tương ứng với một khung và ghi nhận trang ảo của tiến trình nào đang được lưu trong khung đó. Khác với bảng trang thông thường của từng tiến trình, số mục của bảng đảo ngược gắn với số khung trang vật lý, giúp giảm dung lượng cần để quản lý ánh xạ (Silberschatz et al., 2018).

Tuy nhiên, bảng này chủ yếu mô tả các trang đang có trong RAM. Khi xảy ra lỗi trang, hệ điều hành còn cần xác định vị trí của trang chưa hiện diện trong bộ nhớ. Vì vậy, hệ thống phải duy trì thông tin bổ sung về các trang của từng tiến trình và vị trí lưu trữ của chúng.

Theo cách tổ chức được giáo trình trình bày, các bảng bổ sung có thể được đưa ra bộ nhớ phụ khi chưa cần dùng. Điều này tiết kiệm RAM nhưng có thể làm quá trình xử lý lỗi trang phức tạp hơn: để tìm một trang, hệ thống có thể phải nạp trước phần bảng chứa thông tin về trang đó. Do đó, lợi ích giảm dung lượng bảng ánh xạ đi kèm yêu cầu quản lý thông tin bổ sung và chi phí tra cứu.

2.6.5. Cấu trúc chương trình

Phân trang theo yêu cầu thường hoạt động mà chương trình không cần trực tiếp quản lý việc nạp và thay thế trang. Tuy nhiên, cách tổ chức dữ liệu và thứ tự truy cập vẫn ảnh hưởng đến số lỗi trang. Chương trình có tính cục bộ tham chiếu tốt sẽ tập trung truy cập một nhóm vị trí trong một khoảng thời gian nhất định, giúp khai thác hiệu quả các trang đang có trong RAM (Silberschatz et al., 2018).

Chẳng hạn, mảng hai chiều trong C được lưu theo hàng. Duyệt hết các phần tử trên một hàng trước khi chuyển sang hàng tiếp theo thường tận dụng tốt những vị trí nhớ liền nhau. Ngược lại, duyệt theo cột có thể khiến chương trình liên tục chuyển giữa nhiều trang. Khi số khung được cấp không đủ giữ những trang cần thiết, số lỗi trang có thể tăng.

Ngoài thứ tự duyệt, việc lựa chọn cấu trúc dữ liệu cũng ảnh hưởng đến tính cục bộ. Tuy nhiên, không thể chỉ dựa vào tiêu chí này để đánh giá một cấu trúc dữ liệu; cần xem xét thêm tốc độ tìm kiếm, số lần truy cập và dung lượng sử dụng.

Trình biên dịch và bộ nạp có thể hỗ trợ bằng cách bố trí các thủ tục thường xuyên gọi nhau gần nhau, hạn chế để một thủ tục trải qua nhiều trang và tách mã lệnh khỏi dữ liệu. Các trang mã chỉ đọc không bị sửa đổi có thể được loại khỏi RAM mà không cần ghi lại nội dung nếu đã có bản sao hợp lệ trên thiết bị lưu trữ.

2.6.6. Phối hợp I/O và khóa trang

Khi thiết bị thực hiện nhập/xuất trực tiếp với vùng nhớ của tiến trình, hệ thống phải bảo đảm khung chứa vùng đệm không bị thay thế trong lúc thao tác còn đang chờ hoặc đang diễn ra.

Sau khi tiến trình yêu cầu I/O và chuyển sang trạng thái chờ, CPU có thể chạy các tiến trình khác. Trong thời gian đó, trang chứa vùng đệm có thể bị chọn để thay thế và khung của nó được dùng cho dữ liệu khác. Nếu thiết bị vẫn thực hiện I/O theo địa chỉ vật lý đã nhận trước đó, dữ liệu có thể bị đọc hoặc ghi sai vị trí (Silberschatz et al., 2018).

Một giải pháp là thực hiện I/O thông qua vùng đệm do hệ điều hành quản lý và được bảo đảm ổn định trong bộ nhớ, sau đó sao chép dữ liệu giữa vùng đệm này và bộ nhớ của tiến trình. Tuy nhiên, việc sao chép bổ sung làm tăng chi phí xử lý.

Giải pháp khác là khóa trang trong RAM (*page locking* hoặc *pinning*). Những khung liên quan đến thao tác I/O được đánh dấu để không bị chọn thay thế. Khi thao tác hoàn tất, hệ thống mở khóa và cho phép chúng tham gia quá trình thay thế bình thường.

Khóa trang giúp bảo đảm tính đúng đắn của thao tác I/O nhưng làm giảm số khung có thể thu hồi cho các nhu cầu khác. Vì vậy, hệ điều hành cần kiểm soát phạm vi và thời gian khóa, tránh giữ quá nhiều trang trong trạng thái không thể thay thế.

TÀI LIỆU THAM KHẢO:

Arpaci-Dusseau, R. H., & Arpaci-Dusseau, A. C. (2023). *Operating systems: Three easy pieces* (Version 1.10). Arpaci-Dusseau Books. https://pages.cs.wisc.edu/~remzi/OSTEP/

Silberschatz, A., Galvin, P. B., & Gagne, G. (2018). *Operating system concepts* (10th ed.). Wiley.

MỤC LỤC

Phần 3: Phân trang và phần cứng.....2
3.1. Phương pháp phân trang cơ bản.....2
3.1.1. Khái niệm và mục đích của phân trang..... 2
3.1.2. Cách tổ chức Page và Frame..... 2
3.1.3. Cấu trúc địa chỉ logic và phép ánh xạ.....3
3.1.4. Ví dụ minh họa quá trình phân trang.....4
3.1.5. Phân mảnh trong và ảnh hưởng của kích thước trang..... 6
3.1.6. Quản lý và cấp phát các Frame trong bộ nhớ vật lý..... 7
3.2. Hỗ trợ phần cứng cho phân trang và bộ nhớ đệm TLB (Hardware Support).... 8

3.2.1. MMU, lưu trữ bảng trang và thanh ghi PTBR....8
3.2.2. Hạn chế của việc lưu bảng trang trong bộ nhớ chính....9
3.2.3. Khái niệm và cấu trúc của TLB....10
3.2.4. Nguyên lý hoạt động của TLB....10
3.2.5. Hiệu quả của TLB đối với thời gian truy cập bộ nhớ....12
3.2.6. Quản lý TLB và ảnh hưởng đến hiệu suất hệ thống....13
3.3. Bảo vệ bộ nhớ và chia sẻ trang....14
3.3.1. Cơ chế bảo vệ bộ nhớ trong phân trang....14
3.3.2. Cơ chế chia sẻ trang giữa các tiến trình....15
3.4. Cấu trúc của bảng trang....16
3.4.1. Bảng trang nhiều cấp (Hierarchical Paging)....16
3.4.2. Bảng trang băm (Hashed Page Tables)....17
3.4.3. Bảng trang đảo ngược (Inverted Page Tables)....17
3.4.4. Cơ chế quản lý bảng trang trong Oracle SPARC Solaris....18
3.5. Cơ chế phân trang trên kiến trúc Intel IA-32 và x86-64....19
3.5.1. Kiến trúc Intel IA-32....19
3.5.2. Kiến trúc Intel x86-64....20
3.6. Cơ chế phân trang trên kiến trúc ARMv8....21
3.6.1. Đặc điểm quản lý bộ nhớ của kiến trúc ARMv8....21
3.6.2. Cấu trúc bảng trang và hỗ trợ TLB trên ARMv8....21

Phần 3: Phân trang và phần cứng

3.1. Phương pháp phân trang cơ bản

3.1.1. Khái niệm và mục đích của phân trang

Phân trang (Paging) là phương pháp quản lý bộ nhớ cho phép các phần của tiến trình nằm ở nhiều vị trí không liên tiếp trong RAM.

Trong phương pháp cấp phát liên tục (Contiguous Memory Allocation), mỗi tiến trình phải nằm trong một vùng RAM liền kề. Khi các tiến trình được nạp và giải

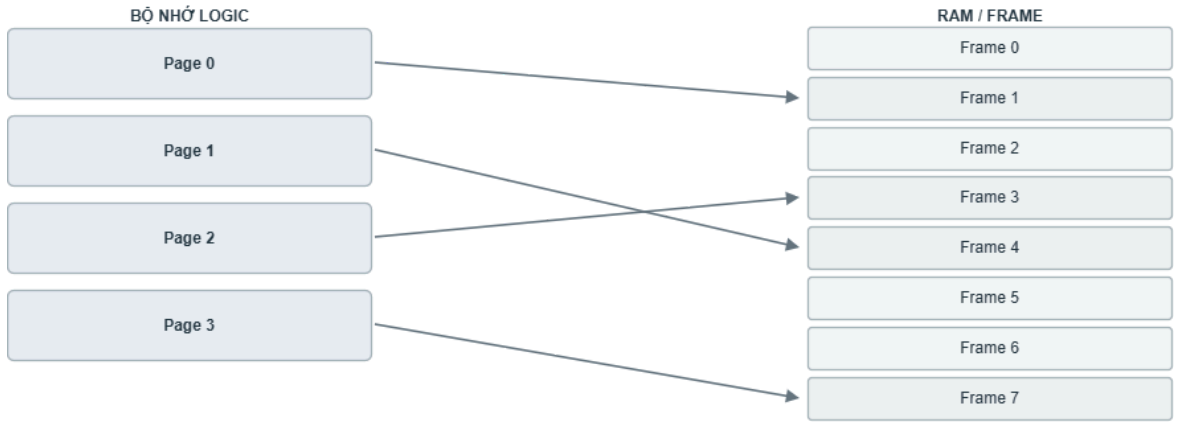
phóng, những khoảng trống nhỏ có thể xuất hiện rải rác. Dù tổng bộ nhớ trống còn đủ, hệ thống vẫn không cấp phát được nếu thiếu một vùng liên tục đủ lớn. Đây là phân mảnh ngoài (External Fragmentation). Hệ điều hành có thể dồn bộ nhớ (Compaction) để gom các khoảng trống, nhưng thao tác này tốn thời gian và tài nguyên.

Phân trang khắc phục hạn chế đó bằng cách chia bộ nhớ thành các khối có kích thước cố định. Hệ điều hành cấp phát những Frame đang trống cho các Page của tiến trình, không yêu cầu các Frame nằm cạnh nhau. Bảng trang (Page Table) lưu thông tin ánh xạ Page–Frame để phần cứng tìm đúng dữ liệu. Cách này loại bỏ phân mảnh ngoài trong mô hình cấp phát theo Frame, nhưng vẫn có thể gây phân mảnh trong nếu Frame cuối chưa được sử dụng hết. [1, Sec. 9.3.1, pp. 359-360]

3.1.2. Cách tổ chức Page và Frame

Bộ nhớ logic của tiến trình được chia thành các trang (Page), còn RAM được chia thành các khung trang (Frame). Page và Frame có cùng kích thước S byte, thường là lũy thừa của 2. Khi được nạp vào RAM, một Page chiếm một Frame. Các Frame chứa những Page liên tiếp không nhất thiết nằm cạnh nhau.

Thứ tự logic không quyết định vị trí vật lý. Ví dụ, Page 0, 1, 2, 3 có thể nằm ở Frame 1, 4, 3, 7 như Hình 3.1. Phần cứng vẫn truy cập đúng dữ liệu nhờ thông tin ánh xạ trong bảng trang. [1, Sec. 9.3.1, pp. 360-362]



Hình 3.1. Page có thể nằm ở những Frame không liên tiếp

3.1.3. Cấu trúc địa chỉ logic và phép ánh xạ

Địa chỉ logic do CPU tạo ra gồm số hiệu trang (Page Number – p) và độ lệch trong trang (Page Offset – d). Giá trị p xác định trang cần truy cập, còn d cho biết vị trí dữ liệu trong trang đó.

Địa chỉ logic = [p | d]

Với không gian địa chỉ logic có kích thước 2^m byte và kích thước trang $S = 2^n$ byte, số hiệu trang p chiếm $m - n$ bit, còn độ lệch d chiếm n bit. Ví dụ, với địa chỉ logic 16 bit và trang 1 KB (2^{10} byte), d chiếm 10 bit và p chiếm 6 bit. Vì vậy, không gian địa chỉ này có tối đa 2^6 = 64 Page.

Vì các Page không cần nằm liên tiếp trong RAM, mỗi tiến trình có một bảng trang (Page Table) để ghi lại vị trí của chúng. MMU dùng số hiệu trang p để tìm mục bảng trang tương ứng và lấy số hiệu khung trang vật lý f.

Quá trình chuyển đổi địa chỉ được thực hiện bởi đơn vị quản lý bộ nhớ (Memory Management Unit – MMU), gồm các bước:

Bước 1. CPU tạo địa chỉ logic và phần cứng xác định hai thành phần p và d.

Bước 2. MMU sử dụng số hiệu trang p để tra cứu mục tương ứng trong bảng trang của tiến trình đang thực thi.

Bước 3. Nếu ánh xạ hợp lệ, mục bảng trang cung cấp số hiệu khung trang vật lý f.

Bước 4. MMU kết hợp số hiệu khung trang f với độ lệch d để hình thành địa chỉ vật lý và thực hiện truy cập bộ nhớ.

Trong phép dịch địa chỉ, p được thay bằng f, còn d giữ nguyên vì Page và Frame có cùng kích thước. Địa chỉ vật lý có dạng:

Địa chỉ vật lý = [f | d]

Hoặc tính theo công thức:

$$\mathbf{PA = f \times S + d}$$

Trong đó, PA là địa chỉ vật lý, f là số hiệu khung trang, S là kích thước trang (byte) và d là độ lệch bên trong trang. [1, Sec. 9.3.1, pp. 360-362]



Hình 3.2. MMU sử dụng Page Table để dịch địa chỉ.

3.1.4. Ví dụ minh họa quá trình phân trang

Xét hệ thống có bộ nhớ logic 4 KB, RAM 8 KB và kích thước trang 1 KB (1.024 byte). Bộ nhớ logic gồm 4 Page, RAM gồm 8 Frame. Bảng sau cho biết Frame tương ứng với từng Page.

Bảng 3.1. Ánh xạ Page-Frame

Page Number (p)	Frame Number (f)
0	6
1	1
2	4
3	7

Với địa chỉ logic 2.500 và kích thước trang S = 1.024 byte, MMU tính số hiệu trang và độ lệch:

$$\mathbf{p = \lfloor 2.500 / 1.024 \rfloor = 2}$$

$$\mathbf{d = 2.500 \bmod 1.024 = 452}$$

Vậy địa chỉ logic 2.500 thuộc Page 2, với độ lệch 452 byte. Tra bảng trang, Page 2 nằm ở Frame 4.

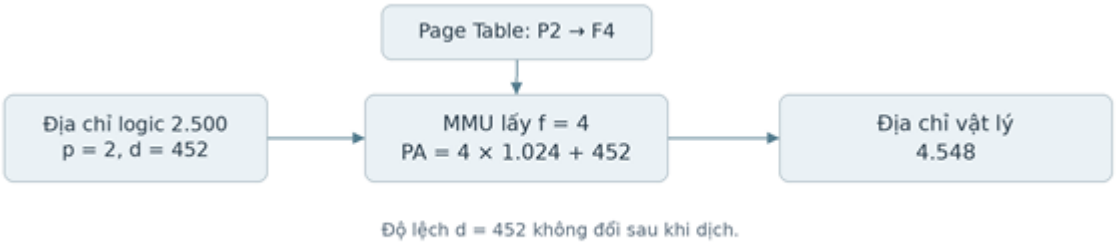
Địa chỉ vật lý được xác định theo công thức:

$$\mathbf{PA = f \times S + d}$$

$$\mathbf{PA = 4 \times 1.024 + 452 = 4.548}$$

Frame 4 bắt đầu tại địa chỉ vật lý 4.096 và kết thúc tại địa chỉ 5.119. Với độ lệch 452 byte, vị trí dữ liệu cần truy cập là địa chỉ vật lý 4.548.

Như vậy, địa chỉ logic 2.500 được dịch thành địa chỉ vật lý 4.548. Ví dụ cho thấy Page 2 không phải nằm ở Frame 2. Bảng trang giúp MMU tìm đúng Frame dù các Page được lưu ở những vị trí không liên tiếp. [1, Sec. 9.3.1, pp. 361-363]



Hình 3.3. Ví dụ minh họa dịch địa chỉ logic sang địa chỉ vật lý

3.1.5. Phân mảnh trong và ảnh hưởng của kích thước trang

Phân trang loại bỏ phân mảnh ngoài nhưng vẫn có thể gây phân mảnh trong (Internal Fragmentation). Lý do là hệ thống cấp phát bộ nhớ theo Frame có kích thước

cố định. Nếu dung lượng tiến trình không chia hết cho kích thước trang, Frame cuối sẽ còn phần chưa sử dụng.

Ví dụ, tiến trình cần 72.766 byte và kích thước trang là 2.048 byte. Tiến trình dùng trọn 35 trang, tương ứng 71.680 byte, và thêm 1.086 byte ở trang thứ 36. Vì phải cấp phát cả Frame cuối, tổng bộ nhớ được cấp là:

$$36 \times 2.048 = 73.728 \text{ byte}$$

Phần bộ nhớ bị lãng phí do phân mảnh trong:

$$73.728 - 72.766 = 962 \text{ byte}$$

Frame cuối còn dư 962 byte. Nếu kích thước tiến trình phân bố tương đối đều so với kích thước trang, lượng phân mảnh trong trung bình xấp xỉ nửa trang cho mỗi tiến trình.

Kích thước trang ảnh hưởng đến cả phân mảnh trong và dung lượng bảng trang. Page nhỏ thường ít lãng phí ở trang cuối, nhưng một tiến trình sẽ có nhiều Page hơn nên bảng trang lớn hơn. Ngược lại, Page lớn giảm số mục bảng trang nhưng có thể làm tăng lượng bộ nhớ dư.

Bảng 3.2. Ảnh hưởng của kích thước trang đến phân mảnh trong và bảng trang

Kích thước trang	Phân mảnh trong	Bảng trang
Page nhỏ	Thường ít lãng phí	Nhiều mục ánh xạ
Page lớn	Có thể lãng phí nhiều	Ít mục ánh xạ

Vì vậy, không có kích thước trang tối ưu cho mọi trường hợp. Hệ thống cần cân bằng giữa dung lượng bị lãng phí và chi phí quản lý bảng trang. Trên Linux x86-64, ngoài trang cơ bản 4 KB, một số cấu hình phần cứng còn hỗ trợ Huge Pages 2 MB hoặc 1 GB. [1, Sec. 9.3.1, pp. 362-363]; [2]

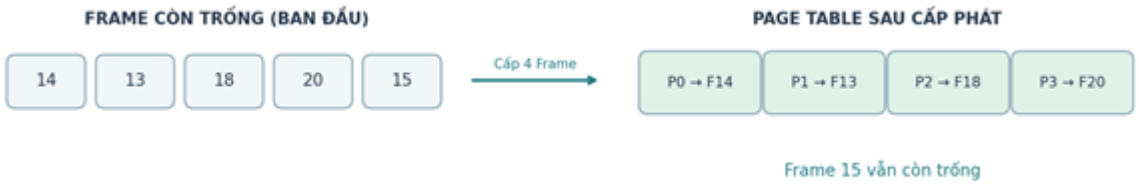
3.1.6. Quản lý và cấp phát các Frame trong bộ nhớ vật lý

Hệ điều hành quản lý các Frame chưa sử dụng bằng danh sách khung trang trống (Free-frame List). Nếu cần nạp toàn bộ một tiến trình có n Page vào RAM, hệ thống phải có ít nhất n Frame trống. Sau khi chọn Frame, hệ điều hành nạp từng Page vào RAM và cập nhật bảng trang để ghi lại ánh xạ Page–Frame.

Một tiến trình có bốn Page được cấp phát lần lượt các Frame 14, 13, 18 và 20. Bảng trang ghi nhận các ánh xạ 0 → 14, 1 → 13, 2 → 18, 3 → 20. Frame 15 vẫn còn trống.

Ngoài Free-frame List, hệ điều hành sử dụng bảng khung trang (Frame Table) để theo dõi tình trạng RAM. Mỗi mục ứng với một Frame, cho biết Frame đang trống hay đã được cấp phát và những thông tin sở hữu liên quan. Điểm khác biệt là Page Table dùng để xác định Page của một tiến trình nằm ở đâu, còn Frame Table theo dõi các Frame vật lý trên toàn hệ thống.

Khi cấp phát hoặc thu hồi Frame, hệ điều hành cập nhật các cấu trúc liên quan. Việc này giúp tránh cấp phát nhầm Frame đang được sử dụng và bảo đảm ánh xạ Page–Frame chính xác. [1, Sec. 9.3.1, pp. 364-365]



Hình 3.4. Cấp phát Frame còn trống và cập nhật bảng trang

3.2. Hỗ trợ phần cứng cho phân trang và bộ nhớ đệm TLB (Hardware Support)

3.2.1. MMU, lưu trữ bảng trang và thanh ghi PTBR

Đơn vị quản lý bộ nhớ (Memory Management Unit – MMU) thực hiện phép dịch từ địa chỉ logic sang địa chỉ vật lý. Để tìm Frame tương ứng với một Page, MMU

cần thông tin trong bảng trang. Vì việc dịch địa chỉ diễn ra thường xuyên, vị trí lưu bảng trang ảnh hưởng trực tiếp đến tốc độ truy cập bộ nhớ.

Một cách tổ chức là lưu bảng trang trong các thanh ghi phần cứng tốc độ cao. Cách này tra cứu nhanh nhưng chỉ phù hợp với bảng trang nhỏ, chẳng hạn 256 mục. Nếu bảng lớn, số thanh ghi cần dùng sẽ nhiều và tốn kém. Việc thay đổi hàng loạt thanh ghi khi chuyển tiến trình cũng làm tăng chi phí chuyển ngữ cảnh (Context Switch).

Vì vậy, bảng trang lớn thường được lưu trong RAM. Thanh ghi cơ sở bảng trang (Page-Table Base Register – PTBR) giữ địa chỉ bắt đầu của bảng trang đang sử dụng. Với bảng trang một cấp, MMU kết hợp PTBR và số hiệu trang p để xác định mục cần đọc. Nếu mỗi mục có kích thước E byte, địa chỉ mục bảng trang được xác định bằng PTBR + p × E. Từ mục này, MMU lấy số hiệu Frame f.

Mỗi tiến trình có bảng trang riêng. Khi chuyển sang không gian địa chỉ khác, hệ điều hành cập nhật PTBR để trỏ đến bảng trang mới. Thông tin tham chiếu bảng trang được lưu trong trạng thái quản lý tiến trình, chẳng hạn PCB. Nhờ PTBR, hệ thống không cần nạp toàn bộ bảng trang vào các thanh ghi khi chuyển tiến trình.

PTBR chỉ cho biết bảng trang nào đang được dùng; nó không lưu toàn bộ ánh xạ Page–Frame. Cách lưu bảng trang trong RAM thuận tiện hơn về dung lượng và chuyển ngữ cảnh, nhưng làm phát sinh chi phí tra cứu bộ nhớ. [1, Sec. 9.3.2, pp. 365-366]

3.2.2. Hạn chế của việc lưu bảng trang trong bộ nhớ chính

Khi bảng trang nằm trong RAM, phép dịch địa chỉ cần thêm một lần truy cập bộ nhớ. Trong mô hình bảng trang một cấp, không có TLB, một lần truy cập dữ liệu cần hai lần truy cập RAM.

Đầu tiên, MMU dùng p và PTBR để tìm mục bảng trang trong RAM, rồi lấy số hiệu Frame f. Sau đó, MMU ghép f với độ lệch d để tạo địa chỉ vật lý PA = f × S + d. Lần truy cập RAM thứ hai mới được dùng để đọc hoặc ghi dữ liệu.

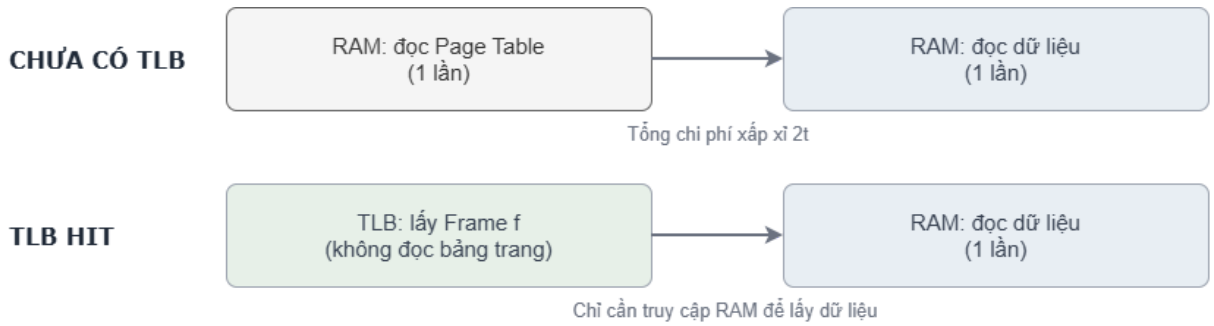
Nếu mỗi lần truy cập RAM mất t và bỏ qua các chi phí khác, tổng thời gian là:

T = t (tra bảng trang) + t (truy cập dữ liệu) = 2t

Như vậy, so với việc truy cập trực tiếp dữ liệu, thời gian có thể gần gấp đôi. Chi phí tăng thêm đến từ lần đọc bảng trang, không phải từ lượng dữ liệu lớn hơn.

Kết quả 2t chỉ đúng với bảng trang một cấp và mỗi lần tra bảng chỉ cần một lần đọc RAM. Bảng trang nhiều cấp có thể cần thêm nhiều lần truy cập nếu không có bộ nhớ đệm ánh xạ.

Để giảm chi phí này, phần cứng sử dụng Translation Lookaside Buffer (TLB). Đây là bộ nhớ đệm giữ một số ánh xạ Page–Frame thường dùng, giúp hạn chế việc đọc bảng trang trong RAM. [1, Sec. 9.3.2, pp. 365-366]



Hình 3.5. So sánh truy cập RAM khi chưa có TLB và khi xảy ra TLB Hit.

3.2.3. Khái niệm và cấu trúc của TLB

Translation Lookaside Buffer (TLB) là bộ nhớ đệm phần cứng tốc độ cao, lưu một số ánh xạ giữa Page và Frame. MMU tra TLB trước khi đọc bảng trang trong RAM. Nếu tìm được ánh xạ, phần cứng có thể xác định Frame nhanh hơn. TLB hỗ trợ chứ không thay thế bảng trang.

TLB thường sử dụng bộ nhớ kết hợp (Associative Memory), cho phép tìm bằng nội dung khóa. Trong mô hình cơ bản, mỗi mục có khóa (Key/Tag) là số hiệu trang p và giá trị (Value) là số hiệu Frame f. Khi MMU đưa p vào TLB, phần cứng so sánh p

với các khóa đang lưu. Nếu khớp, TLB trả về f mà không cần tra bảng trang trong RAM.

Vì cần tra cứu rất nhanh, TLB có dung lượng giới hạn. Giáo trình nêu kích thước điển hình khoảng 32–1.024 mục, nhỏ hơn nhiều so với bảng trang của các không gian địa chỉ lớn. Do đó, TLB thường chỉ giữ một phần ánh xạ. Các mục chưa có trong TLB vẫn được lấy từ bảng trang trong RAM. [1, Sec. 9.3.2, p. 366]

3.2.4. Nguyên lý hoạt động của TLB

Khi CPU tạo địa chỉ logic, MMU tra TLB để tìm ánh xạ Page–Frame. Kết quả được chia thành hai trường hợp: TLB Hit và TLB Miss.

a. TLB Hit – Tìm thấy thông tin ánh xạ

TLB Hit xảy ra khi ánh xạ của Page p đã có trong TLB. MMU lấy Frame f, ghép với độ lệch d để tạo địa chỉ vật lý và truy cập dữ liệu.

Trong trường hợp này, hệ thống không phải đọc bảng trang trong RAM, nên chi phí dịch địa chỉ thấp hơn.

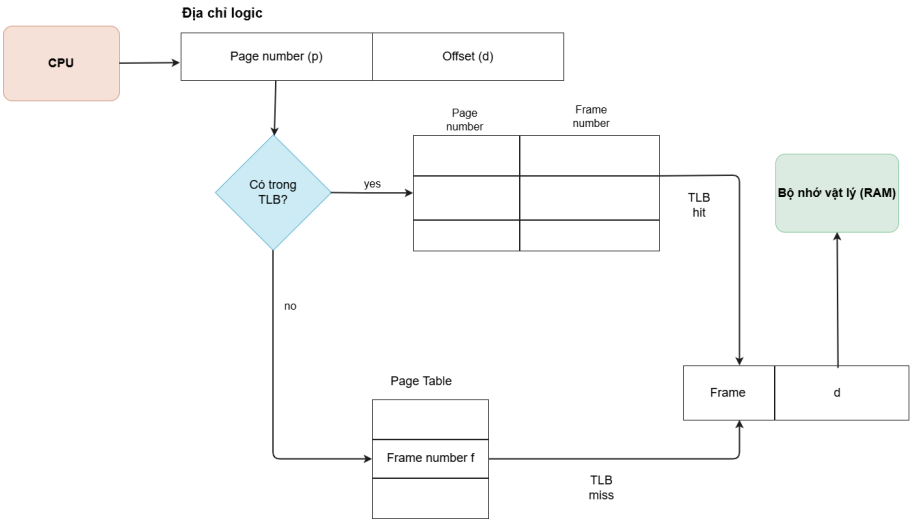
b. TLB Miss – Không tìm thấy thông tin ánh xạ

TLB Miss xảy ra khi TLB không có ánh xạ của Page p. MMU phải tra bảng trang trong RAM để lấy Frame f. Nếu ánh xạ hợp lệ và Page đang ở RAM, phần cứng tạo địa chỉ vật lý. Ánh xạ mới cũng có thể được lưu vào TLB để sử dụng lần sau.

Nếu TLB đầy, hệ thống chọn một mục cũ để thay thế theo chính sách quản lý TLB. Sau khi có Frame f, MMU ghép f với d rồi truy cập dữ liệu.

TLB Miss tốn thêm thời gian do phải tra bảng trang. Tuy nhiên, TLB Miss không đồng nghĩa với Page Fault. TLB Miss chỉ cho biết ánh xạ không có trong TLB; Page vẫn có thể ở RAM. Trong cơ chế Demand Paging, Page Fault có thể xảy ra khi Page cần truy cập chưa được nạp vào RAM. Hệ điều hành sẽ kiểm tra truy cập và xử lý lỗi; nếu hợp lệ, trang có thể được nạp vào bộ nhớ. Vì thế, một TLB Miss có thể dẫn

tới việc phát hiện Page Fault, nhưng không phải lúc nào cũng vậy. [1, Sec. 9.3.2, pp. 366-367]; [1, Sec. 10.2.1, pp. 393–394]



Hình 3.6. Quá trình chuyển đổi địa chỉ thông qua TLB Hit và TLB Miss

3.2.5. Hiệu quả của TLB đối với thời gian truy cập bộ nhớ

Hiệu quả của TLB được thể hiện qua tỷ lệ trúng (TLB Hit Ratio), ký hiệu α. Đây là tỷ lệ số lần tìm thấy ánh xạ trong TLB so với tổng số lần tra cứu. Tỷ lệ trúng càng cao, hệ thống càng ít phải đọc bảng trang trong RAM.

Để định lượng lợi ích này, ta dùng thời gian truy cập hiệu dụng (Effective Access Time – EAT). Xét mô hình đơn giản: thời gian tra TLB không đáng kể, mỗi lần truy cập RAM mất t và trang cần truy cập đã nằm trong RAM. Khi TLB Miss, việc tra bảng trang chỉ cần một lần đọc RAM.

Với TLB Hit, hệ thống chỉ cần một lần đọc RAM để lấy dữ liệu, mất t. Với TLB Miss, hệ thống đọc bảng trang rồi đọc dữ liệu, mất 2t. Do đó:

$$\mathbf{EAT = \alpha \times t + (1 - \alpha) \times 2t}$$

Rút gọn:

$$\mathbf{EAT = (2 - \alpha) \times t}$$

Trong đó, α là tỷ lệ TLB Hit, (1 − α) là tỷ lệ TLB Miss và t là thời gian của một lần truy cập bộ nhớ chính. Công thức cho thấy khi tỷ lệ TLB Hit tăng, thời gian truy cập hiệu dụng giảm và tiến gần đến t.

Với t = 10 ns và tỷ lệ TLB Hit là 80%:

$$EAT = 0{,}80 \times 10 + 0{,}20 \times 20 = 12 \text{ ns}$$

Nếu tỷ lệ TLB Hit đạt 99%:

$$EAT = 0{,}99 \times 10 + 0{,}01 \times 20 = 10{,}1 \text{ ns}$$

Kết quả được tổng hợp trong bảng sau:

TLB Hit Ratio	TLB Miss Ratio	EAT (t = 10ns)	Chi phí tăng so với 10ns
80%	20%	12 ns	20%
99%	1%	10,1 ns	1%

Khi tỷ lệ TLB Hit tăng từ 80% lên 99%, TLB Miss giảm từ 20% xuống 1%. EAT nhờ đó giảm từ 12 ns xuống 10,1 ns. Ví dụ cho thấy TLB có tỷ lệ trúng cao làm giảm đáng kể chi phí đọc bảng trang.

Công thức EAT = (2 − α)t chỉ áp dụng cho mô hình đơn giản nêu trên. Trên CPU có TLB nhiều cấp, thời gian tra cứu và chi phí xử lý TLB Miss phụ thuộc vào từng cấp. Vì vậy, không nên dùng công thức này để dự đoán chính xác hiệu suất của mọi bộ xử lý. [1, Sec. 9.3.2, pp. 367-368]

3.2.6. Quản lý TLB và ảnh hưởng đến hiệu suất hệ thống

Vì TLB có dung lượng giới hạn, khi đầy và cần thêm ánh xạ, hệ thống phải thay thế một mục cũ. Các chính sách phổ biến gồm Least Recently Used (LRU), Round-Robin và Random. LRU thay mục ít dùng gần đây nhất; Round-Robin thay theo thứ tự luân phiên; Random chọn ngẫu nhiên. Mỗi chính sách có ưu, nhược điểm

riêng và không bảo đảm tỷ lệ Hit cao nhất với mọi kiểu truy cập. Một số thiết kế còn cho phép giữ cố định các mục quan trọng.

TLB cũng phải phân biệt ánh xạ của những không gian địa chỉ khác nhau. Cùng Page 3 có thể thuộc Frame 2 ở tiến trình A, nhưng thuộc Frame 7 ở tiến trình B. Nếu không nhận biết được tiến trình sở hữu ánh xạ, TLB có thể trả về Frame sai.

Một số kiến trúc sử dụng Address-Space Identifier (ASID) để nhận diện không gian địa chỉ. Khi có ASID, mỗi mục TLB lưu cả mã không gian địa chỉ và số hiệu Page. Phần cứng chỉ chấp nhận mục có cả hai thông tin khớp. Nhờ vậy, TLB có thể chứa ánh xạ của nhiều tiến trình cùng lúc.

ASID đặc biệt hữu ích khi chuyển ngữ cảnh (Context Switch). Nếu TLB không phân biệt được không gian địa chỉ, hệ thống phải xóa những mục không còn phù hợp khi đổi bảng trang. Các truy cập sau đó dễ gặp TLB Miss hơn. Khi có ASID, các ánh xạ hợp lệ của tiến trình cũ có thể được giữ lại để dùng khi tiến trình được chạy tiếp, giảm số lần phải nạp lại TLB.

Bộ xử lý cũng có thể dùng TLB riêng cho lệnh (Instruction TLB) và dữ liệu (Data TLB), hoặc chia TLB thành nhiều cấp. Ví dụ, Intel Core i7 trong giáo trình có L1 TLB riêng cho lệnh và dữ liệu, cùng L2 TLB lớn hơn. Nếu L1 không có ánh xạ, phần cứng tiếp tục tra L2 trước khi đọc bảng trang. Chi phí thực tế phụ thuộc thời gian tra cứu và tỷ lệ trúng ở từng cấp.

Như vậy, hiệu quả của TLB không chỉ phụ thuộc vào Hit Ratio mà còn vào chính sách thay thế, ASID và cách tổ chức các cấp TLB. Hệ điều hành cần phối hợp với những cơ chế phần cứng này để giảm chi phí dịch địa chỉ. [1, Sec. 9.3.2, pp. 366-368]

Bảng 3.3. Minh họa các mục TLB có sử dụng ASID

ASID	Page	Frame
1	3	2

2	3	7

3.3. Bảo vệ bộ nhớ và chia sẻ trang

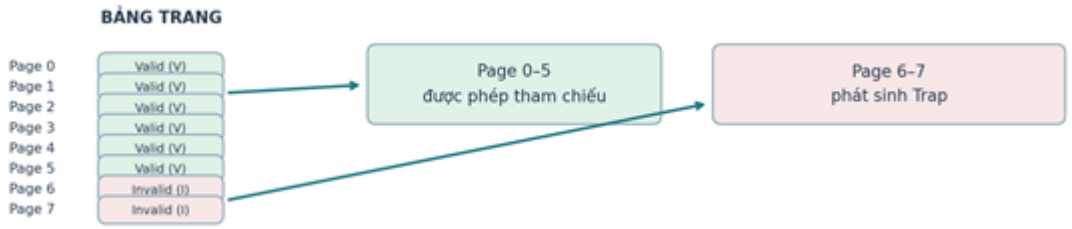
3.3.1. Cơ chế bảo vệ bộ nhớ trong phân trang

Bảo vệ bộ nhớ giúp ngăn tiến trình truy cập hoặc sửa dữ liệu ngoài phạm vi được cấp phép. Trong hệ thống phân trang, phần cứng dựa vào thông tin điều khiển trong mỗi mục bảng trang (Page Table Entry) để kiểm soát truy cập.

Các bit bảo vệ (Protection Bits) quy định quyền đọc (Read), ghi (Write) và thực thi (Execute), tùy kiến trúc. Ví dụ, trang mã chương trình có thể được phép đọc và thực thi nhưng không được ghi. Nếu tiến trình vi phạm quyền, phần cứng phát sinh ngoại lệ (Trap) để hệ điều hành xử lý.

Bit Valid–Invalid cho biết trang có thuộc không gian địa chỉ hợp lệ của tiến trình hay không. Trong ví dụ bảo vệ tại mục 9.3.3, Valid nghĩa là trang hợp lệ, còn Invalid nghĩa là trang nằm ngoài phạm vi được phép truy cập. Khi phát hiện địa chỉ không hợp lệ, phần cứng phát sinh Trap.

Xét hệ thống có địa chỉ logic 14 bit và kích thước trang 2 KB. Trong ví dụ này, Page 0–5 được đánh dấu Valid, còn Page 6–7 được đánh dấu Invalid. Tuy nhiên, bit này chỉ kiểm tra theo đơn vị trang. Nó không phân biệt phần dữ liệu đã dùng và phần còn dư trong một Page hợp lệ. Vì vậy, một số hệ thống cần thêm thông tin giới hạn địa chỉ để kiểm soát chính xác hơn. [1, Sec. 9.3.3, pp. 368-369]



Hình 3.7. Bit Valid - Invalid xác định trang hợp lệ của tiến trình

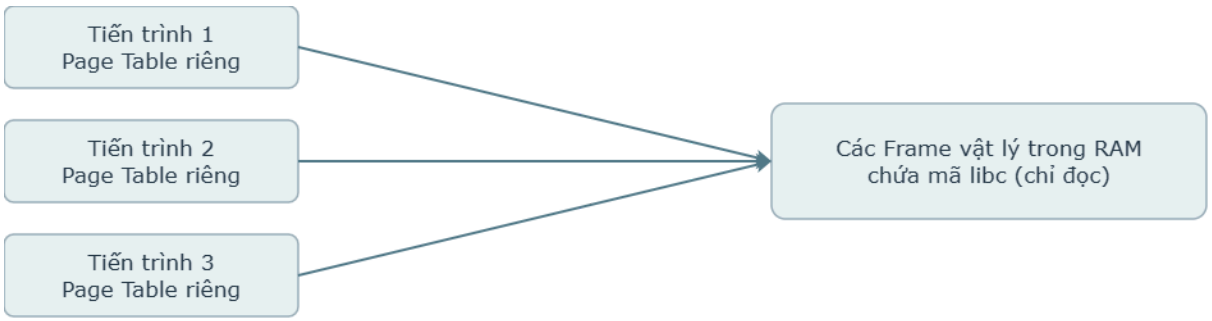
3.3.2. Cơ chế chia sẻ trang giữa các tiến trình

Chia sẻ trang (Shared Pages) cho phép nhiều tiến trình cùng ánh xạ đến một Frame vật lý. Mỗi tiến trình vẫn có bảng trang riêng, nhưng các mục bảng trang có thể trỏ đến cùng Frame chứa nội dung dùng chung. Nhờ đó, RAM không phải lưu nhiều bản sao giống nhau.

Cơ chế này phù hợp với mã có thể sử dụng lại (Reentrant Code): mã không tự thay đổi khi thực thi nên nhiều tiến trình có thể dùng chung. Các trang mã chia sẻ thường chỉ cho phép đọc, trong khi mỗi tiến trình vẫn có vùng dữ liệu và trạng thái thực thi riêng.

Ví dụ, nếu 40 tiến trình cùng dùng phần mã thư viện libc dung lượng 2 MB mà mỗi tiến trình giữ một bản riêng, RAM cần khoảng 80 MB. Với Shared Pages, chúng có thể cùng ánh xạ đến một bản mã khoảng 2 MB. Con số này chưa tính dữ liệu riêng và chi phí quản lý.

Ba tiến trình có bảng trang riêng nhưng cùng trỏ đến các Frame chứa mã libc. Cách tổ chức này giúp tiết kiệm RAM và tránh lưu trữ trùng lặp mã thư viện. [1, Sec. 9.3.4, pp. 369-370]



Hình 3.8. Nhiều tiến trình cùng ánh xạ đến Frame chứa mã libc

3.4. Cấu trúc của bảng trang

3.4.1. Bảng trang nhiều cấp (Hierarchical Paging)

Khi không gian địa chỉ ảo lớn, bảng trang một cấp cũng có thể rất lớn. Ví dụ, địa chỉ logic 32 bit với trang 4 KB có tối đa 2^{20} Page. Nếu mỗi mục bảng trang chiếm 4 byte, một bảng trang đầy đủ cần 4 MB cho một tiến trình. Trong khi đó, tiến trình thường không dùng hết không gian địa chỉ, nên nhiều mục bảng trang có thể không cần thiết.

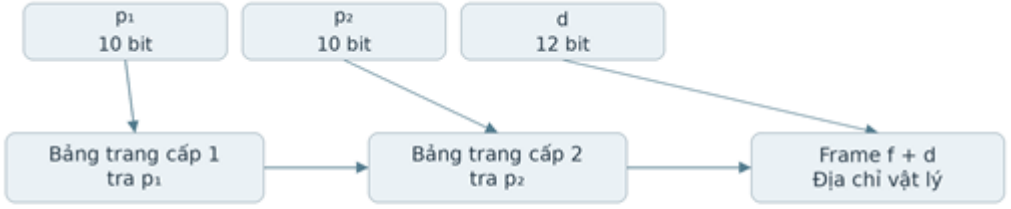
Bảng trang nhiều cấp (Hierarchical Paging) chia bảng trang thành các bảng nhỏ. Ở mô hình hai cấp, bảng trang ngoài (Outer Page Table) chỉ đến bảng trang trong (Inner Page Table). Bảng trang trong mới lưu ánh xạ đến Frame vật lý.

Với địa chỉ logic 32 bit và kích thước trang 4 KB, địa chỉ có cấu trúc:

Địa chỉ logic = [p_1 (10 bit) | p_2 (10 bit) | d (12 bit)]

Trong đó, p_1 là chỉ số tra cứu bảng trang ngoài, p_2 xác định mục trong bảng trang cấp hai và d là độ lệch bên trong trang. Khi chuyển đổi địa chỉ, hệ thống sử dụng p_1 để xác định bảng trang trong, tiếp tục dùng p_2 để tìm số hiệu Frame, rồi kết hợp Frame với d để tạo địa chỉ vật lý.

Ưu điểm là chỉ cần tạo các bảng trang cấp dưới cho những vùng địa chỉ đang dùng, giúp giảm bộ nhớ quản lý. Hạn chế là khi TLB Miss, hệ thống có thể phải đọc nhiều cấp bảng trang. Số cấp được lựa chọn theo độ rộng địa chỉ và thiết kế phần cứng. [1, Sec. 9.4.1, pp. 371-373]



Hình 3.9. Bảng trang hai cấp với địa chỉ 32 bit và trang 4KB

3.4.2. Bảng trang băm (Hashed Page Tables)

Bảng trang băm (Hashed Page Tables) dùng hàm băm (Hash Function) trên số hiệu trang ảo để tìm vị trí lưu ánh xạ. Cách này không cần bố trí một dãy mục liên tục cho toàn bộ không gian địa chỉ.

Khi dịch địa chỉ, hệ thống tính giá trị băm của số trang ảo để tìm Bucket tương ứng. Mỗi Bucket có thể trỏ đến một danh sách mục gồm số trang ảo, Frame vật lý và con trỏ mục tiếp theo. Hệ thống so sánh số trang cần tìm với các mục để lấy Frame phù hợp.

Nếu nhiều trang cho cùng giá trị băm, sẽ xảy ra va chạm băm (Hash Collision). Các mục được nối thành danh sách liên kết và được kiểm tra lần lượt đến khi tìm được trang phù hợp.

Clustered Page Tables là một biến thể cho phép một mục băm lưu ánh xạ của nhiều trang, chẳng hạn 16 trang. Cách này hữu ích cho không gian địa chỉ lớn nhưng các vùng sử dụng nằm rải rác. [1, Sec. 9.4.2, pp. 373-374]

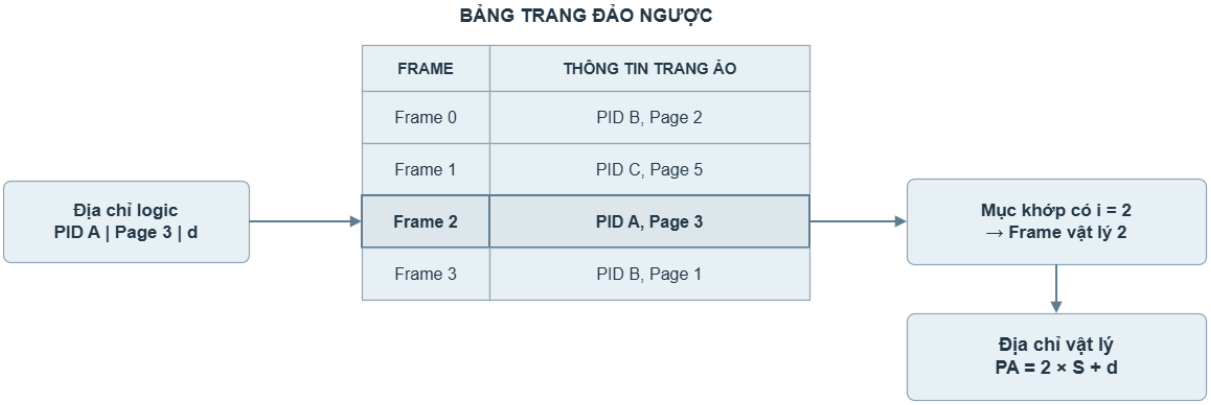
3.4.3. Bảng trang đảo ngược (Inverted Page Tables)

Bảng trang đảo ngược (Inverted Page Table) tổ chức các mục theo Frame vật lý thay vì theo Page của từng tiến trình. Mỗi Frame tương ứng với một mục lưu thông tin về tiến trình và trang ảo đang sử dụng Frame đó.

Khi dịch địa chỉ, hệ thống tìm mục khớp với mã tiến trình và số trang ảo. Nếu mục khớp nằm ở vị trí i, Frame vật lý cũng có số hiệu i. MMU ghép i với độ lệch d để tạo địa chỉ vật lý. Mã tiến trình giúp phân biệt những Page có cùng số hiệu ở các tiến trình khác nhau.

Ưu điểm là số mục phụ thuộc vào số Frame trong RAM, thay vì số trang ảo của tất cả tiến trình. Vì vậy, bảng trang đảo ngược thường tiết kiệm bộ nhớ hơn trong hệ thống có không gian địa chỉ ảo lớn.

Hạn chế là việc tra cứu từ địa chỉ ảo khó hơn, vì bảng được sắp theo Frame vật lý. Duyệt tuần tự sẽ chậm, nên hệ thống thường kết hợp cơ chế băm. Việc chia sẻ một Frame cho nhiều trang ảo cũng phức tạp hơn. [1, Sec. 9.4.3, pp. 374-375]



Hình 3.10. Bảng trang đảo ngược

3.4.4. Cơ chế quản lý bảng trang trong Oracle SPARC Solaris

Oracle Solaris trên SPARC là ví dụ kết hợp nhiều cơ chế quản lý địa chỉ trong hệ thống 64 bit. Solaris dùng bảng băm để lưu ánh xạ địa chỉ ảo sang vật lý, với cấu trúc dành cho nhân hệ điều hành và tiến trình người dùng. Một mục băm có thể mô tả cả vùng địa chỉ liên tiếp.

TLB giữ các mục dịch địa chỉ gọi là Translation Table Entries (TTE). Solaris còn dùng Translation Storage Buffer (TSB), một bộ đệm TTE trong bộ nhớ, để hỗ trợ việc nạp ánh xạ vào TLB.

Khi dịch địa chỉ, phần cứng tra TLB trước. Nếu TLB Miss, hệ thống kiểm tra TSB; nếu thấy TTE phù hợp thì nạp vào TLB. Nếu TSB cũng không có, hệ điều hành tra bảng băm, tạo TTE và cập nhật TSB trước khi tiếp tục.

Bảng băm đảm nhiệm lưu trữ và quản lý ánh xạ; TLB và TSB giúp giảm thời gian tra cứu. Đây là ví dụ thực tế về sự kết hợp giữa cấu trúc bảng trang và bộ nhớ đệm dịch địa chỉ. [1, Sec. 9.4.4, pp. 375-376]

3.5. Cơ chế phân trang trên kiến trúc Intel IA-32 và x86-64

3.5.1. Kiến trúc Intel IA-32

Trong Intel IA-32, địa chỉ được chuyển qua hai cơ chế: phân đoạn (Segmentation) và phân trang (Paging). CPU tạo địa chỉ logic, bộ phận phân đoạn chuyển nó thành địa chỉ tuyến tính (Linear Address), rồi bộ phận phân trang dịch sang địa chỉ vật lý để truy cập RAM.

Địa chỉ logic → Segmentation → Địa chỉ tuyến tính → Paging → Địa chỉ vật lý

Với kích thước trang 4 KB, IA-32 sử dụng bảng trang hai cấp và địa chỉ tuyến tính 32 bit được chia thành ba phần:

Địa chỉ tuyến tính = [p_1 (10 bit) | p_2 (10 bit) | d (12 bit)]

Trong đó, p_1 chọn mục trong Page Directory, p_2 chọn mục trong Page Table và d là độ lệch của trang 4 KB. Thanh ghi CR3 giữ địa chỉ cơ sở của Page Directory. Phần cứng lần lượt tra hai cấp bảng trang để tìm Frame tương ứng.

IA-32 còn hỗ trợ trang lớn 4 MB. Với trang lớn, Page Directory có thể ánh xạ trực tiếp đến vùng 4 MB mà không cần bảng trang cấp hai.

Page Address Extension (PAE) giúp IA-32 mở rộng địa chỉ vật lý lên 36 bit trong mô hình của giáo trình, tương ứng khả năng định địa chỉ tối đa 64 GB RAM. Khi bật PAE, cấu trúc phân trang chuyển từ hai cấp sang ba cấp. Việc sử dụng còn phụ thuộc hệ điều hành và phần cứng. [1, Sec. 9.6.1, pp. 379-382]

3.5.2. Kiến trúc Intel x86-64

Kiến trúc x86-64 được phát triển nhằm mở rộng khả năng định địa chỉ so với các hệ thống 32 bit. Dù dùng thanh ghi 64 bit, không phải mọi thiết kế đều sử dụng đủ 64 bit địa chỉ ảo. Trong mô hình của giáo trình, địa chỉ ảo dài 48 bit và bảng trang có bốn cấp.

Với kích thước trang 4 KB, địa chỉ ảo được chia thành bốn trường chỉ số, mỗi trường 9 bit, và một trường độ lệch 12 bit:

Địa chỉ ảo = [9 bit | 9 bit | 9 bit | 9 bit | 12 bit]

Bốn chỉ số lần lượt tra Page Map Level 4, Page Directory Pointer Table, Page Directory và Page Table. Mục cuối cung cấp Frame vật lý; kết hợp Frame với d sẽ tạo địa chỉ vật lý.

Ngoài trang 4 KB, x86-64 hỗ trợ trang lớn 2 MB và 1 GB trong mô hình giáo trình. Với trang lớn, việc dịch địa chỉ có thể kết thúc sớm ở cấp bảng trang phía trên, giúp giảm số cấp phải tra và số mục cần quản lý.

Bảng trang nhiều cấp giúp x86-64 quản lý không gian địa chỉ lớn mà không phải cấp phát toàn bộ bảng trang ngay từ đầu. Chỉ những vùng được dùng mới cần bảng cấp dưới. Đổi lại, TLB Miss có thể khiến phần cứng phải tra qua nhiều cấp, làm tăng chi phí. [1, Sec. 9.6.2, pp. 382-383]

3.6. Cơ chế phân trang trên kiến trúc ARMv8

3.6.1. Đặc điểm quản lý bộ nhớ của kiến trúc ARMv8

ARM là kiến trúc bộ xử lý được sử dụng phổ biến trong các thiết bị di động và hệ thống nhúng. Với phiên bản 64 bit ARMv8, cơ chế phân trang được thiết kế để hỗ trợ không gian địa chỉ lớn và cho phép tổ chức bộ nhớ với nhiều kích thước trang khác nhau.

ARMv8 sử dụng đơn vị dịch địa chỉ (Translation Granule) để xác định kích thước trang cơ bản. Giáo trình giới thiệu ba lựa chọn: 4 KB, 16 KB và 64 KB. Tùy Granule, kiến trúc còn hỗ trợ những vùng ánh xạ lớn hơn (Region), như bảng dưới.

Translation Granule	Page Size	Region Size
4KB	4KB	2MB, 1GB

16KB	16KB	32MB
64KB	64KB	512MB

Với Granule 4 KB, hệ thống có thể ánh xạ từng trang 4 KB hoặc vùng lớn 2 MB và 1 GB. Trang nhỏ phù hợp với quản lý chi tiết; vùng lớn giúp giảm số mục bảng trang cho bộ nhớ liên tiếp.

Nhờ nhiều lựa chọn Granule, hệ điều hành có thể tổ chức bộ nhớ phù hợp với thiết kế phần cứng và nhu cầu sử dụng. [1, Sec. 9.7, p. 383]

3.6.2. Cấu trúc bảng trang và hỗ trợ TLB trên ARMv8

Với Translation Granule 4 KB, ARMv8 có thể dùng bảng trang bốn cấp, từ Level 0 đến Level 3. Trong mô hình địa chỉ ảo 48 bit của giáo trình, mỗi cấp dùng chỉ số 9 bit và phần d chiếm 12 bit:

Địa chỉ ảo = [L0 (9 bit) | L1 (9 bit) | L2 (9 bit) | L3 (9 bit) | d (12 bit)]

Thanh ghi Translation Table Base Register (TTBR) giữ địa chỉ cơ sở của bảng trang cấp đầu. Phần cứng dùng các trường chỉ số để lần lượt tìm mục ánh xạ qua các cấp.

Việc dịch địa chỉ không luôn đi hết bốn cấp. Với Granule 4 KB, Level 1 có thể ánh xạ vùng 1 GB và Level 2 ánh xạ vùng 2 MB. Trang 4 KB được xác định ở Level 3. Nhờ vậy, ARMv8 hỗ trợ cả trang nhỏ và vùng nhớ lớn. Để tăng tốc, mô hình ARM trong giáo trình sử dụng Micro TLB riêng cho lệnh và dữ liệu, cùng một Main TLB ở cấp tiếp theo. Các mục TLB có thể gắn ASID để phân biệt không gian địa chỉ.

Nếu Micro TLB không có ánh xạ, phần cứng tra Main TLB. Nếu vẫn không tìm thấy, hệ thống thực hiện Page Table Walk để lấy địa chỉ vật lý từ bảng trang. Như vậy, ARMv8 kết hợp bảng trang nhiều cấp và TLB để quản lý không gian địa chỉ lớn mà vẫn hạn chế chi phí dịch địa chỉ. [1, Sec. 9.7, pp. 383-384]

TÀI LIỆU THAM KHẢO

[1] A. Silberschatz, P. B. Galvin, and G. Gagne, *Operating System Concepts*, 10th ed. Hoboken, NJ, USA: Wiley, 2018.

[2] The Linux Kernel, "HugeTLB Pages," The Linux Kernel documentation. [Online]. Available: https://docs.kernel.org/admin-guide/mm/hugetlbpage.html. [Accessed: Oct. 21, 2026].

Bao_cao_Phan_4_Bo_Nho_Ao_Thrashing_DONG_BO_GIAO_DIEN - Google Tài liệu

MỤC LỤC

TÓM TẮT NGHIÊN CỨU 2
QUY ƯỚC TRÍCH DẪN VÀ GIỚI HẠN 4
4.0. Dẫn nhập: vấn đề và phương pháp nghiên cứu 4
4.1. Nền tảng bộ nhớ ảo (10.1) 4
4.2. Phân trang theo yêu cầu (10.2.1) 6
4.3. Danh sách khung trang trống (10.2.2) 8
4.4. Hiệu năng và EAT (10.2.3) 9
4.5. Copy-on-Write (10.3) 12
4.6. Phân bổ khung trang (10.5.1–10.5.4) 13
4.7. Nguyên nhân hình thành thrashing (10.6.1) 15
4.8. Mô hình working-set (10.6.2) 17
4.9. Tần suất lỗi trang PFF (10.6.3) 19
4.10. Thực tiễn xử lý thrashing (10.6.4) 22
4.11. Tổng hợp, phản biện và kết luận 22
TÀI LIỆU THAM KHẢO 24

QUY ƯỚC TRÍCH DẪN VÀ GIỚI HẠN NGHIÊN CỨU

Ký hiệu "tr." chỉ số trang in trong giáo trình; số trang PDF là vị trí trong tệp điện tử và có thể khác số trang in. Trích dẫn trong thân bài theo tác giả – năm – trang (APA); ví dụ: (Silberschatz et al., 2018, tr. 399–400).

Báo cáo dùng hệ thống mục 4.x để thống nhất với các phần 2, 3, 5 của nhóm; ký hiệu 10.x trong ngoặc tiêu đề chỉ mục tương ứng của Operating System Concepts. Mục 10.4 thuộc Phần 5 nên không trình bày lại ở đây. Các số EAT, NUMA, tỷ lệ lỗi trang là tính toán minh họa với giả định công khai, không phải dữ liệu đo thực nghiệm.

4.0. DẪN NHẬP: VẤN ĐỀ VÀ PHƯƠNG PHÁP NGHIÊN CỨU

Trong hệ điều hành đa chương, nhiều tiến trình cùng hoạt động trong khi bộ nhớ vật lý hữu hạn. Nhu cầu lưu toàn bộ mã lệnh và dữ liệu của mọi tiến trình vào RAM không phải điều kiện cần cho sự thực thi: tại mỗi giai đoạn, tiến trình thường chỉ sử dụng một phần không gian địa chỉ. Phân trang theo yêu cầu khai thác đặc điểm này bằng cách chuyển trang từ bộ nhớ phụ vào RAM tại thời điểm có truy cập đến một trang đang vắng mặt. Nhờ đó, nhiều không gian địa chỉ có thể cùng tồn tại trên một hệ thống có RAM nhỏ hơn tổng kích thước ảo của chúng (Silberschatz et al., 2018, tr. 392–395; Nông Minh Ngọc & Nguyễn Văn Huy, 2016, tr. 128–131).

Tuy nhiên, mỗi lần CPU đòi hỏi một trang chưa có trong RAM, bộ xử lý không thể hoàn tất tham chiếu ngay. Hệ điều hành phải kiểm tra địa chỉ, cấp khung trang, yêu cầu thiết bị lưu trữ đọc dữ liệu, cập nhật bảng trang và cho phép lệnh tiếp tục. Vì hoạt động I/O có thể chậm hơn truy cập RAM rất nhiều trong mô hình giáo trình, chỉ một xác suất page fault tương đối thấp cũng làm tăng đáng kể thời gian truy cập trung bình. Đây là nghịch lý vận hành cần giải thích: chính cơ chế tiết kiệm bộ nhớ có thể làm suy giảm hiệu năng nếu các trang cần thiết không được giữ lại đúng thời điểm (Silberschatz et al., 2018, tr. 397–398).

Vấn đề trở nên hệ thống hơn khi nhiều tiến trình cùng thiếu khung trang. Một tiến trình thay trang mà các tiến trình khác vẫn cần; những tiến trình này tiếp tục fault để lấy lại trang, khiến nhu cầu I/O tăng, thời gian đợi kéo dài và thời gian CPU làm việc có ích giảm. Giáo trình gọi trạng thái hệ thống dành nhiều thời gian cho phân trang hơn cho thực thi là thrashing. Hệ quả cho thấy cơ chế demand paging và chính sách phân phối khung không thể nghiên cứu rời nhau (Silberschatz et al., 2018, tr. 419–421).

Báo cáo giải quyết bốn nhóm câu hỏi: (i) không gian địa chỉ ảo thưa thớt và COW giảm nhu cầu RAM bằng cơ chế nào; (ii) một lỗi trang hợp lệ và lỗi ghi COW được xử lý khác nhau ra sao; (iii) EAT phụ thuộc tỷ lệ lỗi và chi phí phục vụ đến mức nào; (iv) chính sách phân bổ khung tương tác với locality, working-set và PFF như thế nào để hạn chế rủi ro thrashing.

4.1. NỀN TẢNG BỘ NHỚ ẢO (giáo trình: 10.1)

4.1.1. Tách biệt không gian logic và không gian vật lý

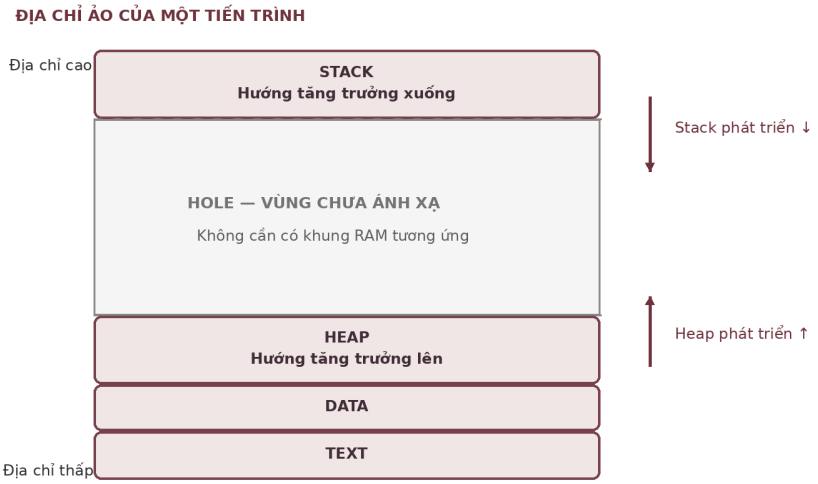
Bộ nhớ ảo (virtual memory) tách không gian địa chỉ logic mà tiến trình sử dụng khỏi các khung trang vật lý trong RAM. Nhờ đó, tiến trình có thể sở hữu không gian địa chỉ lớn hơn phần đang thường trú; chỉ những trang thực sự cần thiết mới phải có nơi lưu trú trong bộ nhớ chính. Đây là nền tảng của việc thực thi chương trình khi không cần nạp toàn bộ mã và dữ liệu ngay từ đầu (Silberschatz et al., 2018, tr. 389–391; Nông Minh Ngọc & Nguyễn Văn Huy, 2016, tr. 128–130).

Về cơ chế, CPU phát sinh địa chỉ ảo; MMU dịch địa chỉ dựa trên bảng trang và trạng thái ánh xạ do hệ điều hành quản lý. Sự tồn tại của một địa chỉ trong không gian ảo không đồng nghĩa

trang đó đang chiếm một khung RAM. Đánh đổi là hệ điều hành phải duy trì cấu trúc ánh xạ, kiểm tra quyền truy cập và xử lý khi trang hợp lệ chưa thường trú. Không nên diễn giải "RAM ảo" như dung lượng RAM vật lý được tăng thêm (Silberschatz et al., 2018, tr. 390–392).

4.1.2. Không gian địa chỉ thưa thớt và sự phát triển heap–stack

Trong mô hình không gian địa chỉ ảo thưa thớt (sparse address space), vùng text và data thường ở địa chỉ thấp, heap mở rộng về phía địa chỉ cao, còn stack phát triển về phía địa chỉ thấp. Khoảng giữa hai vùng là "hole" chưa dùng; nó không đòi hỏi khung vật lý cho đến khi vùng địa chỉ hợp lệ được mở rộng và có nhu cầu hiện thực hóa trang. Khoảng trống cũng thuận lợi cho liên kết động thư viện (Silberschatz et al., 2018, tr. 391, Hình 10.2).



Hình 4.1. Không gian địa chỉ ảo thưa thớt, stack phát triển xuống và heap phát triển lên

Nguồn: Tác giả dựng lại theo Silberschatz et al. (2018, tr. 391, Hình 10.2); mô hình, không biểu diễn vị trí RAM.

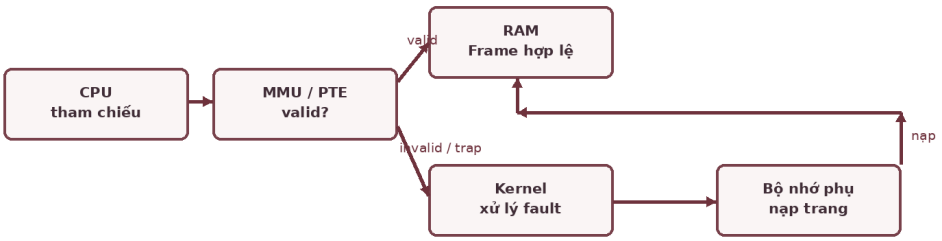
Cần phân biệt vùng chưa ánh xạ với trang hợp lệ nhưng đang vắng mặt: truy cập một vùng không hợp lệ không tự động được hệ điều hành xử lý như page fault có thể nạp. Khoảng trống ảo cũng không tự bảo đảm giảm kích thước bảng trang; hiệu quả phụ thuộc vào cấu trúc bảng trang đã phân tích ở Phần 3 của nhóm (Silberschatz et al., 2018, tr. 391–392; liên hệ Sec. 9.4).

4.1.3. Trang dùng chung, thư viện động và giới hạn bảo vệ

Thông qua nhiều ánh xạ ảo cùng trỏ đến một khung vật lý, các tiến trình có thể chia sẻ trang mã của thư viện hệ thống, chẳng hạn thư viện C chuẩn (libc), hoặc thiết lập vùng nhớ chia sẻ có chủ đích. Mỗi tiến trình xem thư viện là thành phần của không gian địa chỉ riêng, dù dữ liệu trang vật lý được dùng chung. Mã thư viện thường được ánh xạ chỉ đọc để bảo toàn nội dung. Cách chia sẻ này giảm các bản sao dư thừa nhưng đòi hỏi kiểm soát quyền truy cập và sự nhất quán ánh xạ (Silberschatz et al., 2018, tr. 392, Hình 10.3).

Cơ chế fork() cũng có thể dùng chung các khung trang ban đầu và trì hoãn sao chép bằng Copy-on-Write. Do đó, chia sẻ trang là cầu nối giữa phân trang cơ bản của Phần 3 với nhu cầu cấp khung vật lý ở Phần 4. Chia sẻ mã chỉ đọc cần được phân biệt với vùng nhớ chia sẻ có khả năng ghi; mọi phép ánh xạ không mặc nhiên có cùng ngữ nghĩa chia sẻ (Silberschatz et al., 2018, tr. 392; tr. 399–400).

4.2. PHÂN TRANG THEO YÊU CẦU (giáo trình: 10.2.1)



Hình 4.2. Mối liên hệ giữa truy cập trang, bảng trang và nạp trang theo yêu cầu

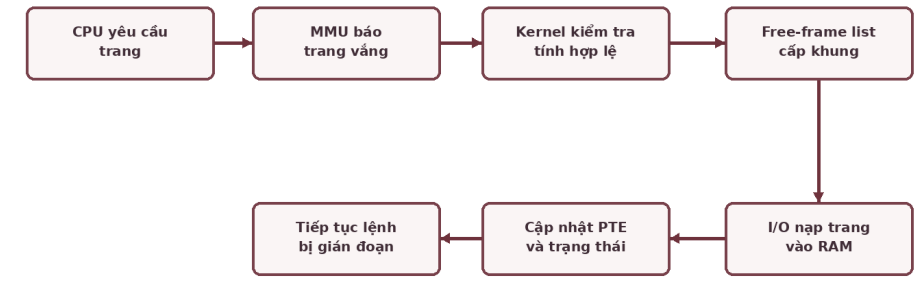
Nguồn: Tác giả dựng lại từ Silberschatz et al. (2018, tr. 393–395), hình mô hình hóa.

4.2.1. Nguyên lý và điều kiện hình thành lỗi trang

Tên nguyên bản của tiểu mục 10.2.1 trong giáo trình là "Basic Concepts"; tên tiếng Việt trong báo cáo làm rõ đó là các khái niệm cơ bản của phân trang theo yêu cầu. Khác với cách nạp toàn bộ chương trình trước khi chạy, demand paging chỉ mang vào RAM những trang thực sự được tham chiếu. Trang không được sử dụng sẽ không tạo ra hoạt động nạp chỉ vì nó thuộc về không gian địa chỉ của tiến trình. Để thực hiện được điều này, phần cứng và hệ điều hành cần một cơ chế nhận biết trang đã thường trú trong RAM hay còn nằm ở bộ nhớ phụ (Silberschatz et al., 2018, tr. 393; Nông Minh Ngọc & Nguyễn Văn Huy, 2016, tr. 130).

Sách sử dụng lược đồ valid–invalid bit để chỉ ra trạng thái. Với trang hợp lệ và đang có trong RAM, giá trị valid cho phép tiếp tục tiến trình dịch địa chỉ. Với giá trị invalid, cần phân biệt hai trường hợp không giống nhau: một trang thực sự không thuộc không gian địa chỉ hợp lệ của tiến trình; hoặc trang hợp lệ nhưng chưa được nạp vào RAM. Bit trạng thái vì thế không tự cung cấp toàn bộ quyết định xử lý: phải kết hợp thông tin quản lý nội bộ của hệ điều hành để phân loại nguyên nhân. Nếu nhầm hai trường hợp, báo cáo sẽ sai ở bước xử lý lỗi, vì truy cập bất hợp lệ không được "cứu" bằng cách cấp một khung RAM mới (Silberschatz et al., 2018, tr. 394).

Khi CPU tham chiếu một địa chỉ có bảng trang đánh dấu invalid, cơ chế phần cứng tạo trap để chuyển quyền điều khiển về hệ điều hành. Page fault ở đây là sự kiện thay đổi đường đi xử lý, không đồng nghĩa lỗi lập trình. Sau khi hệ điều hành xác nhận trang thuộc không gian địa chỉ hợp lệ và hiện còn ở bộ nhớ phụ, nó mới thực hiện quy trình nạp. Ngược lại, tham chiếu vô hiệu dẫn tới xử lý lỗi truy cập và có thể kết thúc tiến trình theo tình huống giáo trình mô tả. Sự phân đôi này được phản ánh ở lưu đồ của Hình 4.3.



Hình 4.3. Luồng quyết định khi phát sinh page fault từ một tham chiếu trang

Nguồn: Tác giả chuyển thể cơ chế của Silberschatz et al. (2018, tr. 394–395, Hình 10.5); giản lược nhánh quản lý I/O.

4.2.2. Sáu bước phục vụ một page fault hợp lệ

Hình 10.5 của giáo trình chính trình bày một chu trình sáu bước. Điểm bắt đầu là CPU tạo tham chiếu tới trang chưa có trong RAM; hệ điều hành kiểm tra thông tin tiến trình để quyết định tham chiếu có hợp lệ; sau đó tìm khung trang tự do, yêu cầu nạp từ bộ nhớ phụ, cập nhật thông tin thường trú và khởi động lại lệnh bị gián đoạn. Giá trị của mô hình không chỉ ở thứ tự mà ở ranh giới trách nhiệm: phần cứng phát hiện điều kiện trap, hệ điều hành kiểm tra hợp lệ và điều phối I/O, bảng trang lưu kết quả cập nhật để lần tham chiếu tiếp theo không còn gặp trạng thái cũ (Silberschatz et al., 2018, tr. 394–395).

Bảng 4.1. Các giai đoạn xử lý lỗi trang hợp lệ và ý nghĩa kỹ thuật

Giai đoạn	Hoạt động chính	Kết quả/điểm kiểm soát
1	Kiểm tra bảng quản lý nội bộ và địa chỉ tiến trình	Phân loại hợp lệ/bất hợp lệ
2	Xác định trang hợp lệ chưa thường trú	Chuẩn bị yêu cầu nạp
3	Lấy một khung trống từ free-frame list	Có vị trí đích trong RAM
4	Yêu cầu đọc trang từ bộ nhớ phụ	Tiến trình có thể phải chờ I/O
5	Cập nhật bảng trang và thông tin nội bộ	Trang trở thành thường trú và ánh xạ hợp lệ
6	Khởi động lại lệnh gây fault	Tiếp tục đường thực thi của tiến trình

Đối chiếu: Silberschatz et al. (2018, tr. 394–395); bảng diễn giải, không sao chép nguyên văn.

Việc nói "sáu bước" không có nghĩa toàn bộ xử lý diễn ra nguyên tử hay không có thời gian chờ. Ở bước đọc dữ liệu, hệ điều hành có thể đưa tiến trình sang trạng thái chờ; CPU được trao cho một tiến trình khác và sau đó cần lập lịch để quay lại. Khi trang đã được tải, thông tin ánh xạ phải được làm nhất quán trước khi lệnh tiếp tục. Đây chính là cầu nối trực tiếp tới mô hình EAT ở mục 4.4: thời gian phục vụ fault bao gồm chi phí xử lý ngoại lệ, I/O và thời gian tái khởi động, chứ không chỉ thời gian đọc dữ liệu vật lý (Silberschatz et al., 2018, tr. 397–398).

Khi trang hợp lệ vắng mặt trong RAM, hệ điều hành có thể phải lấy dữ liệu từ bộ nhớ phụ. SSD là ví dụ về phần cứng có thể chứa dữ liệu swap/page file, nhưng không phải mọi page fault

đều dẫn đến đọc SSD; thời gian 8 ms trong mục 4.4 vẫn là giả định của giáo trình, không phải số liệu đo thực nghiệm trên phần cứng SSD.

Ở mức lý tưởng hóa, pure demand paging cho phép tiến trình bắt đầu khi chưa có trang nào trong RAM, rồi nạp từng trang theo sự kiện truy cập. Cách mô tả này giúp nhận ra tính chất theo nhu cầu nhưng cũng bộc lộ giới hạn: các fault ban đầu có thể xuất hiện dồn dập. Giáo trình giải thích việc thực thi thực tế vẫn thường thuận lợi nhờ locality of reference – các tham chiếu có xu hướng tập trung vào một số trang trong một giai đoạn. Tính cục bộ sẽ trở thành giả định quan trọng của mô hình working-set ở phần sau, chứ không phải một bảo đảm rằng mọi chương trình đều có tỷ lệ fault thấp (Silberschatz et al., 2018, tr. 395, 421–423).

4.2.3. Vấn đề khởi động lại lệnh và giới hạn kiến trúc

Một khía cạnh dễ bị bỏ qua là khả năng khởi động lại đúng lệnh tại thời điểm fault. Với lệnh đơn giản, việc tiếp tục từ trạng thái đã lưu tương đối dễ hình dung. Tuy nhiên giáo trình đưa ví dụ về lệnh di chuyển khối dữ liệu của IBM System/360–370: nếu một phần thao tác đã ghi vào bộ nhớ trước khi trang còn lại gây fault, khởi động lại không kiểm soát có thể tạo ra trạng thái sai. Sách nêu các giải pháp kiến trúc như kiểm tra địa chỉ trước khi thay đổi hoặc lưu giá trị trung gian để phục hồi trạng thái trước trap (Silberschatz et al., 2018, tr. 395–396).

Dẫn chứng này tạo ra một nhận định quan trọng: demand paging không đơn giản là thuật toán nạp trễ được cài riêng trong phần mềm. Nó đòi hỏi phần cứng, cấu trúc trạng thái và cách thực thi lệnh tương thích với việc gián đoạn rồi khởi động lại. Không nên từ cơ chế minh họa trên một kiến trúc suy rộng rằng mọi ISA có cùng phương thức phục hồi lệnh. Trong phạm vi báo cáo, nguyên lý được giữ ở mức hệ điều hành tổng quát; chi tiết kiến trúc thuộc phần thảo luận phần cứng của các thành viên khác.

Kết quả của mục 4.2 là điều kiện kép: page fault phải được xác nhận là hợp lệ và phải có phương thức đưa đúng nội dung trang vào một khung RAM để cho phép tiếp tục lệnh. Việc sở hữu một khung trống là yêu cầu vật lý trực tiếp, nên mục kế tiếp đi sâu cơ chế quản lý free-frame list – thành phần biến yêu cầu nạp thành một địa chỉ lưu trú cụ thể.

4.3. DANH SÁCH KHUNG TRANG TRỐNG (giáo trình: 10.2.2)

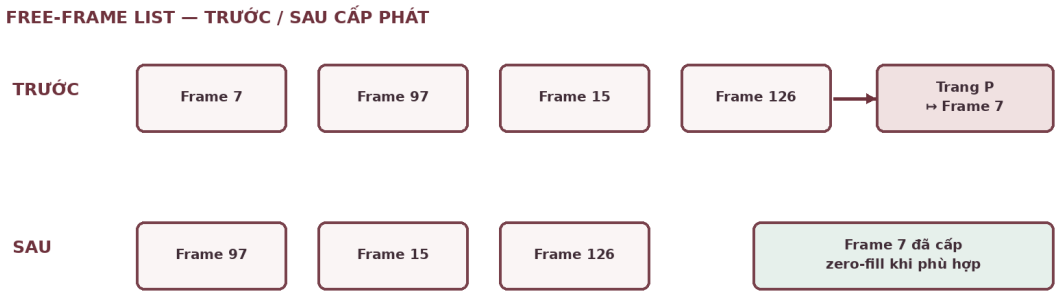
4.3.1. Vai trò free-frame list trong tiến trình nạp trang

Mỗi trang ảo cần một khung trang vật lý (frame) để có thể nằm trong RAM khi được tham chiếu. “Trang” là đơn vị của không gian địa chỉ ảo; “khung” là đơn vị lưu giữ trong bộ nhớ vật lý, hai khái niệm có cùng kích thước trong mô hình phân trang nhưng không đồng nhất về vai trò. Phần mềm có thể quản lý hàng loạt trang ảo đang vắng mặt, trong khi số khung khả dụng ở một thời điểm là hữu hạn. Free-frame list chính là cấu trúc quản lý tập khung vật lý đang sẵn sàng được cấp phát khi cần phục vụ page fault (Silberschatz et al., 2018, tr. 396–397).

Theo trình bày của giáo trình, khi một page fault hợp lệ cần được giải quyết, hệ điều hành có thể lấy khung từ free-frame list, nạp trang cần dùng vào đó, rồi ghi nhận số khung tương ứng trong bảng trang. Danh sách này cũng hỗ trợ yêu cầu mở rộng stack hoặc heap. Cách tổ chức danh sách không quyết định trang ảo nào cần nạp; nó thực hiện trách nhiệm cấp chỗ lưu trú cho trang mà các bước trước đã xác nhận. Do đó, free-frame list nằm ở giao điểm giữa hai bài toán: quản lý tài nguyên RAM tự do và xử lý lỗi trang theo yêu cầu (Silberschatz et al., 2018, tr. 396).

Hình 10.6 của sách minh họa chuỗi chỉ số khung trống. Trên phương diện trừu tượng, một phần tử trong danh sách đại diện cho một khung có thể cấp, không phải một “trang tự do” của chương trình. Sau khi một khung được lấy để chứa trang của tiến trình, khung đó không còn được xem là thành viên tự do cho tới khi hệ điều hành thu hồi theo chính sách thích hợp. Sự thay đổi quy mô danh sách giúp

nhận biết áp lực bộ nhớ vật lý, nhưng không thể tự nó cho biết tính cục bộ truy cập hoặc nguy cơ thrashing của từng tiến trình.



Hình 4.4. Tổ chức danh sách khung tự do và thao tác cấp một khung cho trang

Nguồn: Tác giả mô hình hóa theo Silberschatz et al. (2018, tr. 396–397, Hình 10.6); số khung chỉ minh họa.

4.3.2. Zero-fill-on-demand và tính đúng đắn của cấp phát

Một chi tiết có ý nghĩa hệ thống là zero-fill-on-demand: giáo trình mô tả nhiều hệ điều hành khởi tạo nội dung khung bằng giá trị zero trước khi giao khung cho yêu cầu mới. Nếu dữ liệu cũ trong khung vẫn còn và trở nên quan sát được bởi tiến trình khác, tính cô lập bộ nhớ có thể bị phá vỡ. Bởi vậy, thao tác khởi tạo không chỉ là lựa chọn về hiệu năng mà còn là điều kiện để ngăn dữ liệu tồn dư giữa các đối tượng dùng RAM khác nhau. Khi sử dụng hình minh họa, cần lưu ý rằng khung cấp cho một trang tải từ bộ nhớ phụ sẽ được ghi bằng nội dung trang, còn khung cấp mới cho vùng nhớ ẩn danh có thể cần hiện thực hóa vùng zero theo cơ chế của hệ thống (Silberschatz et al., 2018, tr. 397).

Danh sách khung trống co lại khi có các yêu cầu cấp mới. Giáo trình nhận xét rằng tới một ngưỡng, danh sách phải được bổ sung trở lại; cách thực hiện có thể liên quan thu hồi, thay thế trang và các thuật toán thay thế trang được phân tích ở Phần 5 của nhóm. Báo cáo vì vậy chỉ xác lập quan hệ nguyên nhân: khi số khung trống giảm, việc phục vụ fault hợp lệ trở nên phụ thuộc mạnh hơn vào chính sách thu hồi tài nguyên. Tình trạng danh sách giảm tới zero không đồng nghĩa hệ thống đã thrashing; hệ thống có thể có cơ chế giải phóng khung thích hợp và fault vẫn xuất hiện với nhịp độ chấp nhận được (Silberschatz et al., 2018, tr. 397).

Từ góc độ thiết kế, free-frame list cho phép tách quyết định “cần nạp trang nào” khỏi thao tác “cấp khung nào”. Lợi ích là đường xử lý page fault có thể thao tác với tài nguyên sẵn có thay vì tìm kiếm trực tiếp trên toàn RAM. Tuy nhiên, cơ chế đó không tự giải bài toán cấp bao nhiêu khung cho mỗi tiến trình theo thời gian. Việc một tiến trình đã được cấp frame không bảo đảm trang sẽ còn trong RAM ở lần truy cập tiếp theo nếu áp lực bộ nhớ buộc hệ điều hành thu hồi. Những hạn chế này giải thích lý do cần các tín hiệu EAT, working-set và PFF ở các mục sau.

Bảng 4.2. Minh họa trạng thái khung và trang trước – sau một yêu cầu cấp phát (giả định)

Thời điểm	Free-frame list	Trang P	Kết quả
Trước truy cập	[7, 97, 15, 126]	P hợp lệ, chưa thường trú	Phát sinh page fault
Sau khi cấp khung	[97, 15, 126]	P ↦ frame 7	Frame 7 được lấy ra khỏi danh sách
Sau đọc trang / cập nhật	[97, 15, 126]	P hợp lệ, thường trú	Lệnh được phép chạy tiếp

Thời điểm	Free-frame list	Trang P	Kết quả
Khi khung được thu hồi	Danh sách có thể thêm frame	Ánh xạ cũ phải được xử lý nhất quán	Hành vi cụ thể tùy chính sách thu hồi

Nguồn: Tác giả xây dựng tình huống giả định dựa trên Silberschatz et al. (2018, tr. 394–397).

Bảng 4.2 làm rõ sự khác nhau giữa một danh sách khung còn chỗ và trạng thái của một trang ảo cụ thể. Cần thêm một quy ước: tại bước "khung được thu hồi", bảng không mô tả cách chọn trang nạn nhân, vì đó là nội dung phần thuật toán thay thế trang của thành viên khác. Điểm cần lưu lại là tính nhất quán: không được vừa công bố khung là "trống" vừa để ánh xạ cũ cho phép truy cập nội dung của khung đó mà không có kiểm soát.

Tiểu kết: free-frame list là hạ tầng cấp phát cần thiết để phản hồi một fault hợp lệ, nhưng không phải thước đo trực tiếp của hiệu năng. Muốn đánh giá thời gian mất do cơ chế nạp trễ, phải xét cả tần suất sự kiện fault và chi phí phục vụ mỗi sự kiện. Mô hình thời gian truy cập hiệu dụng được khảo sát ngay sau đây thực hiện chính phép nối đó.

4.4. HIỆU NĂNG PHÂN TRANG THEO YÊU CẦU VÀ EAT (giáo trình: 10.2.3)

4.4.1. Khái niệm và thành phần chi phí

Thời gian truy cập hiệu dụng (effective access time – EAT) là giá trị trung bình kỳ vọng của một lần truy cập bộ nhớ khi kết hợp hai loại tình huống: truy cập bình thường tới trang đã thường trú, và truy cập gây page fault cần phục vụ bởi hệ điều hành. Đây là mô hình xác suất đơn giản, cho phép chuyển nhận định định tính "lỗi trang tốn kém" thành một biểu thức có biến và đơn vị rõ ràng. Trong mô hình giáo trình, ta gọi *ma* là thời gian truy cập bộ nhớ bình thường, p là xác suất có lỗi trang, còn T_{pf} là thời gian xử lý một sự kiện page fault theo giả định đã chọn (Silberschatz et al., 2018, tr. 397–398).

$$\mathbf{EAT} = (1 - p) \times ma + p \times T_{pf} \quad \textbf{(4.1)}$$

Trong đó: EAT, *ma* và T_{pf} đo bằng ns; p là xác suất lỗi trang (không có đơn vị).

Phép trung bình ở (4.1) phải được hiểu cùng với các điều kiện của nó: mỗi tham chiếu được quy vào một trong hai trường hợp và các thời gian trong biểu thức được đặt trên cùng đơn vị. Đại lượng p nhận giá trị từ 0 tới 1. T_{pf} là thời gian phục vụ fault được xác định cho tình huống nghiên cứu; không nên lấy riêng thời gian truyền dữ liệu của thiết bị lưu trữ làm toàn bộ T_{pf}. Giáo trình liệt kê một chuỗi công việc: trap sang hệ điều hành, lưu trạng thái, kiểm tra tính hợp lệ, xác định vị trí trang, phát lệnh đọc, chờ thiết bị, nhận ngắt hoàn tất, cập nhật bảng trang, chờ CPU, khôi phục trạng thái và tiếp tục lệnh (Silberschatz et al., 2018, tr. 397–398).

Chi phí fault có thể thay đổi theo bối cảnh thực thi. Nếu CPU chuyển sang một tiến trình khác trong thời gian chờ I/O thì việc dùng CPU tổng thể có thể tốt hơn so với đứng yên, nhưng tiến trình ban đầu vẫn phải chịu độ trễ cho tới khi nó được lập lịch trở lại. Nếu hàng đợi thiết bị lưu trữ dài, thời gian phục vụ fault tăng dù tốc độ đọc vật lý của thiết bị không đổi. Nếu cần giải phóng một khung trước khi nạp, có thể phát sinh những tác vụ ngoài mô hình đơn giản. Do đó một trị số T_{pf} dùng trong bảng tính chỉ mang ý nghĩa khi nêu rõ phạm vi và giả định.



Các thành phần cộng vào thời gian phục vụ page fault; thời gian chờ hàng đợi I/O có thể tăng mạnh.

Hình 4.5. Ba cụm chi phí thường được xét khi phục vụ page fault

Nguồn: Tác giả hệ thống hóa từ Silberschatz et al. (2018, tr. 397–398).

4.4.2. Tái tính ví dụ định lượng từ giáo trình chính

Một ví dụ trong sách chọn *ma* = 200 nanosecond (ns) và thời gian phục vụ lỗi trang trung bình T_{pf} = 8 millisecond (ms) trong bối cảnh thiết bị lưu trữ HDD. Đây là giả định của ví dụ trong tài liệu năm 2018, không phải hằng số áp dụng cho mọi máy tính hiện nay. Để tính đúng thứ nguyên, 8 ms được đổi thành 8.000.000 ns. Thế vào (4.1) thu được kết quả dưới đây (Silberschatz et al., 2018, tr. 398).

$$\mathbf{EAT(ns) = (1 - p) \times 200 + p \times 8.000.000 = 200 + 7.999.800p} \quad \mathbf{(4.2)}$$

Giá trị 200 ns và 8 ms được lấy theo ví dụ của Silberschatz et al. (2018, tr. 398).

Nếu p = 0,001 (trung bình một fault cho một nghìn tham chiếu), EAT = 8.199,8 ns, hay khoảng 8,2 microsecond (µs), so với 0,2 µs khi không có page fault. Theo số liệu và giả định trong ví dụ này, tỉ lệ chậm đi xấp xỉ 41 lần. Con số phụ thuộc mạnh vào cặp thông số *ma* và T_{pf}; vì vậy, không được đưa nó lên slide như một kết quả đo tốc độ phổ quát của demand paging.

Tiếp theo, nếu muốn EAT thấp hơn mức 220 ns – nghĩa là phần tăng thêm nhỏ hơn 10% so với *ma* = 200 ns – ta giải bất đẳng thức 200 + 7.999.800p < 220, suy ra p < 20/7.999.800 ≈ 2,50006 × 10^{-6}. Điều này tương đương cần ít hơn khoảng một fault cho 400.000 tham chiếu, trong điều kiện giữ nguyên T_{pf} theo ví dụ. Ý nghĩa không nằm ở việc mọi hệ thống đều phải đạt tỷ lệ này, mà ở độ nhạy của trung bình EAT trước một sự kiện chậm hàng triệu nanosecond (Silberschatz et al., 2018, tr. 398).

$$\mathbf{p < 20 / 7.999.800 \approx 2{,}50006 \times 10^{-6}} \quad \mathbf{(4.3)}$$

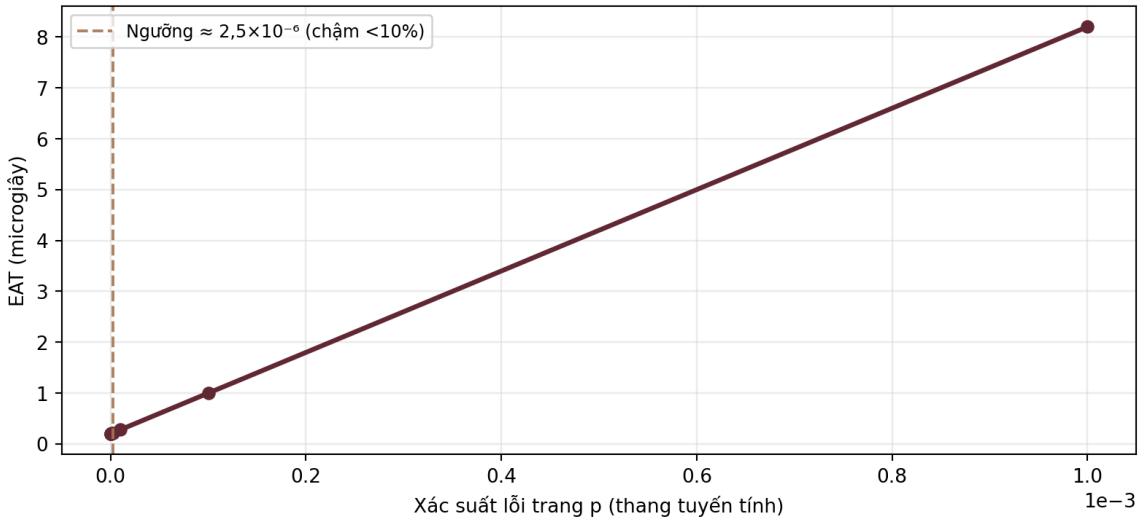
Điều kiện p nhỏ hơn ngưỡng áp dụng khi yêu cầu EAT tăng dưới 10% và giữ nguyên các tham số giả định.

Bảng 4.3. Độ nhạy của EAT theo p với ma = 200 ns, T_pf = 8 ms (tính toán từ giả định của sách)

p	EAT (ns)	EAT (µs)	EAT / 200 ns
0	200,00	0,20000	1.00×
1e-07	200,80	0,20080	1.00×
5e-07	204,00	0,20400	1.02×
1e-06	208,00	0,20800	1.04×
2.5e-06	220,00	0,22000	1.10×
1e-05	280,00	0,28000	1.40×
0.0001	999,98	0,99998	5.00×

p	EAT (ns)	EAT (µs)	EAT / 200 ns
0.001	8.199,80	8,19980	41.00×

Nguồn: Tác giả tái tính theo công thức và thông số ví dụ của Silberschatz et al. (2018, tr. 398); đây là tính toán, không phải đo thực nghiệm.



Hình 4.6. Quan hệ EAT–p trên miền xác suất minh họa; ma và T_pf được giữ cố định

Nguồn: Tác giả tính toán từ ví dụ Silberschatz et al. (2018, tr. 398). Đồ thị mô hình, không phải số liệu đo.

4.4.3. Diễn giải độ nhạy và giới hạn của EAT

Nhìn vào (4.2), đạo hàm của EAT theo p trong mô hình tuyến tính là $T_{pf} - ma = 7.999.800$ ns. Vì đây là một hệ số rất lớn so với *ma*, một thay đổi nhỏ của p có thể tạo ra chênh lệch EAT đáng kể. Hình 4.6 cho thấy vùng gần p = 0 mới đặc biệt có ý nghĩa khi theo đuổi mức trễ tăng không quá 10%; trong khi đó các giá trị p lớn dẫn tới trễ kỳ vọng tăng nhanh. Không được diễn giải độ dốc của đồ thị như một số liệu đo trên CPU thực tế, bởi đường thẳng được tạo trực tiếp từ công thức với hai thông số cố định.

EAT là giá trị trung bình; mô hình không cho biết độ trễ cực đại hay tính phụ thuộc giữa các lần page fault. Trong những khoảng truy cập có tính cục bộ ổn định, xác suất fault có thể thấp; lúc dịch chuyển sang một locality mới, hàng loạt trang cần nạp có thể làm tỷ lệ fault tăng theo thời gian. Khi đó, một p cố định cho toàn bộ tiến trình có thể che khuất các giai đoạn mất ổn định. Mối quan hệ giữa thay đổi locality, chuỗi lỗi trang và trạng thái thrashing sẽ được khảo sát ở mục 4.7 và 10.6.2 (Silberschatz et al., 2018, tr. 421–423).

Một hạn chế khác là giả định T_{pf} cố định. Ngay trong phần diễn giải, sách nhấn mạnh vai trò của thời gian chờ hàng đợi I/O, ngắt và lập lịch; khi áp lực phân trang tăng, một số thành phần chi phí có thể lớn hơn. Trong mô hình nghiên cứu, do đó, cần tách bạch "độ nhạy EAT theo p nếu T_{pf} không đổi" với "tác động hệ thống thực khi cả p và T_{pf} cùng biến động". Vế thứ hai có thể nghiêm trọng hơn, nhưng không thể suy ra một hệ số chính xác nếu chưa đo hoặc chưa có dữ liệu chi tiết của hệ thống xét tới.

Về mặt đánh giá, demand paging không phải cơ chế luôn làm máy chạy chậm: lợi ích tránh nạp trang không dùng tới và khả năng tăng số tiến trình cư trú là thực chất. Nó trở thành rủi ro hiệu năng khi tỷ lệ lỗi trang và thời gian phục vụ của chúng vượt quá mức cho phép đối với tải công việc. Tiểu kết của mục 4.4 là chính xác hóa biến trung gian quan trọng giữa quản lý khung

vật lý và chất lượng thực thi: xác suất page fault, cùng với chi phí phục vụ, quyết định phần lớn độ trễ bổ sung trong mô hình khảo sát.

4.4.4. Đối chiếu EAT giữa HDD và kịch bản SSD/NVMe giả định

Bản giáo trình xét thời gian truy cập bộ nhớ bình thường *ma* = 200 ns và thời gian phục vụ page fault trung bình T_{pf} = 8 ms trong ví dụ HDD. Để kiểm tra độ nhạy đại số, báo cáo giữ *ma* và p như cũ nhưng xét thêm hai giả định T_{pf} = 20 µs và 50 µs. Hai trị số sau KHÔNG phải kết quả đo SSD/NVMe từ các giáo trình được cung cấp; chúng chỉ có ý nghĩa nếu được giả định là toàn bộ thời gian phục vụ fault, không phải riêng độ trễ đọc thiết bị (Silberschatz et al., 2018, tr. 397–398).

$$\mathbf{EAT = (1 - p) \times ma + p \times T_{pf}}$$

Các đại lượng EAT, *ma* và T_{pf} phải cùng đơn vị (ns); p là xác suất không có đơn vị.

Bảng 4.4. EAT theo xác suất lỗi trang trong ba kịch bản phục vụ fault

p	HDD (8 ms) — EAT (ns)	Giả định (20 µs) — EAT (ns)	Giả định (50 µs) — EAT (ns)
0	200,000	200,000	200,000
1e-06	208,000	200,020	200,050
1e-05	279,998	200,198	200,498
0.0001	999,980	201,980	204,980
0.001	8 199,800	219,800	249,800

Nguồn: Tác giả tính từ công thức Silberschatz et al. (2018, tr. 398); 20–50 µs là giả định, không phải kết quả đo.

$$\mathbf{p < 0{,}1 \times ma / (T_{pf} - ma)} \quad \textbf{(với } \mathbf{T_{pf} > ma}\textbf{)} \quad \textbf{(4.4)}$$

Điều kiện để EAT thấp hơn 110% thời gian truy cập bình thường trong mô hình hai trạng thái.

Bảng 4.5. Ngưỡng xác suất lỗi trang dưới mức tăng EAT 10%

Kịch bản	Giới hạn p (<10% tăng EAT)	Diễn giải gần đúng
HDD 8 ms	2.50006e-06	1 lỗi / khoảng 399 990 tham chiếu
Giả định 20 µs	0.0010101	1 lỗi / khoảng 990 tham chiếu
Giả định 50 µs	0.000401606	1 lỗi / khoảng 2 490 tham chiếu

Nguồn: Tác giả giải bất đẳng thức từ EAT; giá trị không phải ngưỡng vận hành thực nghiệm.

Khi thời gian phục vụ fault giảm, cùng mức tăng EAT cho phép xác suất p lớn hơn. Tuy vậy, SSD/NVMe không xóa bỏ ảnh hưởng của chi phí xử lý trap, cấp khung, lập lịch và cạnh tranh I/O. Lỗi COW hoặc minor fault cũng không nên được gộp mặc định vào mô hình major fault 8 ms. Mô hình hai trạng thái không phản ánh đuôi phân bố độ trễ hay ảnh hưởng đồng thời giữa p và T_{pf} khi thrashing (Silberschatz et al., 2018, tr. 397–398, 416, 420–421).

4.5. SAO CHÉP KHI GHI – COPY-ON-WRITE (giáo trình: 10.3)

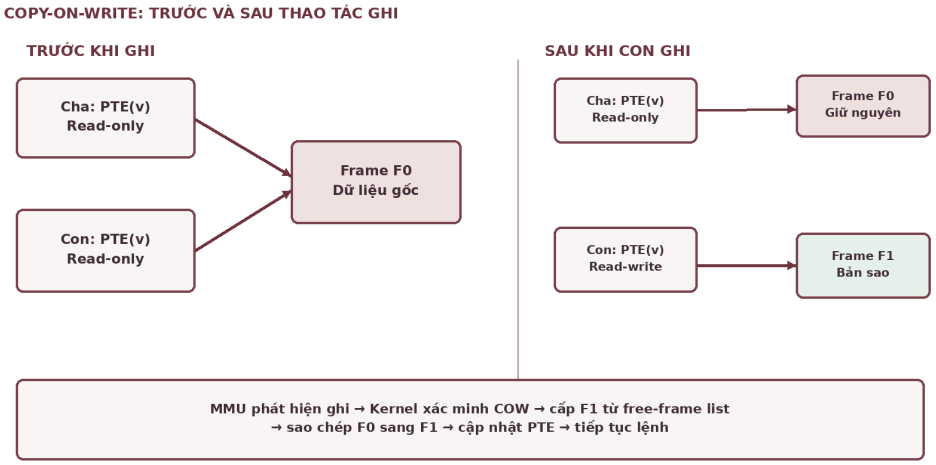
4.5.1. Vấn đề sao chép toàn bộ sau fork()

Trong mô hình tạo tiến trình truyền thống, fork() tạo một bản sao không gian địa chỉ của tiến trình cha cho tiến trình con. Nếu tiến trình con gọi exec() ngay sau đó để thay thế chương trình đang thực thi, phần lớn các trang mới sao chép có thể không được sử dụng; chi phí nhân bản trở thành lãng phí. Silberschatz trình bày Copy-on-Write (COW) như giải pháp cho tình huống này (Silberschatz et al., 2018, tr. 399; Arpaci-Dusseau & Arpaci-Dusseau, 2018, Chương 23, tr. 7).

4.5.2. Chia sẻ trang và xử lý ngoại lệ ghi

Với COW, cha và con ban đầu có các mục bảng trang trỏ đến cùng khung vật lý. Các ánh xạ dữ liệu cần bảo vệ được thiết lập để phát hiện ghi, thường dưới quyền Read-Only; nhân hệ điều hành duy trì thông tin phân biệt một trang COW hợp lệ với một trang vốn không được phép ghi. Khi con thực hiện ghi, MMU phát sinh trap do vi phạm quyền; hệ điều hành xác minh trang

COW, cấp một khung từ free-frame list, sao chép nội dung, cập nhật PTE của con sang khung mới với quyền Read-Write và cho lệnh tiếp tục (Silberschatz et al., 2018, tr. 399–400).



Hình 4.7. Sao chép khi ghi: khung F0 chia sẻ trước khi ghi và F1 riêng sau khi ghi

Nguồn: Tác giả dựng lại từ Silberschatz et al. (2018, tr. 399–400, Hình 10.7–10.8).

Năm giai đoạn cần phân biệt là: (1) trap phần cứng; (2) kiểm tra tính hợp lệ COW; (3) cấp khung mới; (4) sao chép và cập nhật ánh xạ/quyền; (5) tiếp tục lệnh ghi. Không phải mọi lỗi bảo vệ ghi đều được xử lý bằng sao chép: ghi vào một trang chỉ đọc không thuộc diện COW vẫn là vi phạm bảo vệ. Khi bản gốc đã nằm trong RAM, việc tạo bản sao thường không cần một lần đọc trang từ đĩa; vì vậy cần phân biệt lỗi COW với major page fault (Silberschatz et al., 2018, tr. 399–400, 416).

4.5.3. So sánh fork() truyền thống, COW fork() và vfork()

Bảng 4.6. So sánh đặc tính và giới hạn ba cơ chế tạo tiến trình

Tiêu chí	fork() sao chép toàn phần	fork() dùng COW	vfork()
Dữ liệu lúc tạo	Nhân bản các trang dữ liệu	Chia sẻ khung, tách khi ghi	Con tạm dùng không gian cha
Chi phí tạo	Cao hơn khi sao chép nhiều	Trì hoãn sao chép	Thấp trong trường hợp phù hợp
RAM phát sinh	Theo số trang nhân bản	Theo số trang thực sự bị sửa	Không tạo bản sao như COW
Ghi của con	Ghi trên bản sao riêng	Tạo F1 và cập nhật PTE	Có thể ảnh hưởng dữ liệu cha
Giới hạn	Lãng phí nếu exec() ngay	Trap và sao chép ở lần ghi	Cha bị tạm dừng, cần dùng thận trọng

Nguồn: Silberschatz et al. (2018, tr. 399–401). So sánh ở mức mô hình giáo trình.

COW giảm chi phí tạo tiến trình khi nhiều trang không bị sửa nhưng không loại bỏ chi phí tách trang về sau. vfork() được dành cho trường hợp con nhanh chóng gọi exec(); cần tránh mô tả nó như hai tiến trình tự do ghi đồng thời vào cùng vùng, vì giáo trình nêu cha bị tạm dừng trong giai đoạn này (Silberschatz et al., 2018, tr. 400–401).

4.6. PHÂN BỔ KHUNG TRANG (giáo trình: 10.5.1–10.5.4)

4.6.1. Số khung tối thiểu và ràng buộc kiến trúc

Số khung tối thiểu mà một tiến trình cần không chỉ phụ thuộc dung lượng chương trình mà còn phụ thuộc ISA và các chế độ định địa chỉ. Nếu một lệnh phải tham chiếu trang chỉ thị, toán

hạng và một hoặc nhiều cấp địa chỉ gián tiếp, các trang đang cần cho cùng lệnh có thể phải hiện diện đủ để lệnh được hoàn tất sau lỗi trang. Silberschatz minh họa trường hợp cần ba khung với định địa chỉ gián tiếp một cấp; một lệnh vắt qua ranh giới hai trang có thể cần 2 khung chứa mã lệnh; nếu có hai toán hạng định địa chỉ gián tiếp thì mỗi toán hạng có thể cần thêm 1 khung cho địa chỉ trung gian và 1 khung chứa dữ liệu đích. Trong kịch bản sáu trang khác nhau này, yêu cầu đồng thời có thể là 2 + 2 + 2 = 6 khung. Không có một con số tối thiểu phổ quát cho mọi kiến trúc (Silberschatz et al., 2018, tr. 413–414).

$$a_i \geq b_i; \quad \sum b_i \leq m \text{ (điều kiện khả thi cho n tiến trình)}$$

a_i: khung cấp cho tiến trình i; b_i: tối thiểu ISA của tiến trình i; m: tổng khung khả dụng (đơn vị: khung).

4.6.2. Phân bổ đều và theo tỷ lệ kích thước

Equal Allocation chia m khung cho n tiến trình với mức cơ sở floor(m/n); khung dư có thể để trong free-frame buffer pool. Ví dụ 93 khung chia cho 5 tiến trình cho 18 khung mỗi tiến trình, còn 3 khung dự phòng. Chính sách đơn giản nhưng có thể cấp thừa cho tiến trình nhỏ và thiếu cho tiến trình lớn. Proportional Allocation điều chỉnh lượng khung theo kích thước không gian ảo của các tiến trình (Silberschatz et al., 2018, tr. 414–415).

$$S = \sum s_i; \quad a_i = \lfloor (s_i / S) \times m \rfloor; \quad R = m - \sum a_i \quad (4.5)$$

s_i: kích thước tiến trình i (trang/byte); S: tổng theo cùng đơn vị; a_i: khung; R: khung dư.

Với hai tiến trình có 10 và 127 trang, m = 62, ta nhận $a_1^* \approx 4{,}526$ và $a_2^* \approx 57{,}474$; làm tròn xuống được 4 và 57, còn 1 khung chưa phân phối. Có thể giữ khung dư hoặc dùng một quy tắc bổ sung như phân phát theo phần thập phân lớn nhất; quy tắc xử lý dư này là giả định minh họa của báo cáo, không phải chính sách mặc định được giáo trình quy định. Kết quả phải thỏa số khung tối thiểu và tổng phân bổ không vượt m (Silberschatz et al., 2018, tr. 414–415).

4.6.3. Thay thế toàn cục – cục bộ và đường dẫn đến thrashing

Bảng 4.7. Đánh đổi giữa thay thế toàn cục và cục bộ

Tiêu chí	Global Replacement	Local Replacement
Phạm vi nạn nhân	Có thể chọn khung của tiến trình khác	Chỉ từ tập khung của chính tiến trình
Tính linh hoạt	Tận dụng tài nguyên trên phạm vi hệ thống	Giới hạn bởi phân bổ ban đầu
Phụ thuộc lẫn nhau	Một tiến trình có thể làm tiến trình khác mất trang	Hạn chế tác động thay thế trực tiếp
Thông lượng	Giáo trình nhận định thường lớn hơn	Có thể kém linh hoạt hơn
Rủi ro	Có thể truyền áp lực trang khi thiếu khung	Không bảo đảm tránh I/O thrashing

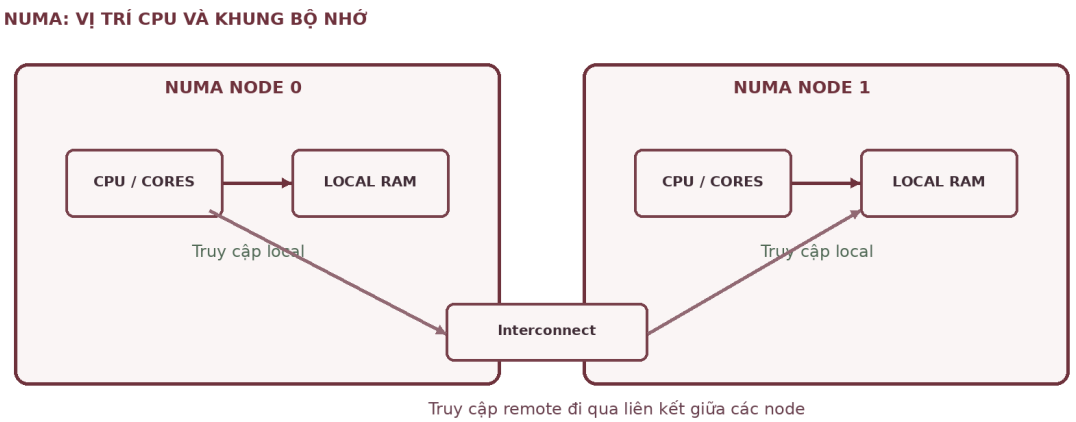
Nguồn: Silberschatz et al. (2018, tr. 415–416, 419–421).

Khi P1 thiếu khung và thay thế trang thuộc tập làm việc của P2, P2 có thể fault để nạp lại; lần nạp đó lại thay thế trang mà P3 còn cần. Nếu tổng nhu cầu các tập làm việc vượt số khung khả dụng và quá trình diễn ra lặp lại, hệ thống có thể bước vào trạng thái phân trang nhiều hơn thực thi. Tuy nhiên, Global Replacement không phải nguyên nhân đủ gây thrashing trong mọi trường hợp; điểm quyết định là áp lực bộ nhớ, locality và chính sách điều chỉnh mức đa chương (Silberschatz et al., 2018, tr. 415–416, 419–421).

4.6.4. Định vị khung trên kiến trúc NUMA

Trong kiến trúc NUMA, bộ nhớ vật lý được tổ chức theo các node gắn với nhóm CPU; một CPU có thể truy cập RAM cục bộ nhanh hơn RAM nằm ở node khác qua liên kết hệ thống. Vì vậy, việc phân bổ khung không chỉ hỏi "cấp bao nhiêu" mà còn hỏi "cấp ở đâu". Hệ điều hành

cần phối hợp chính sách cấp bộ nhớ với vị trí mà bộ điều phối CPU cho tiến trình thực thi (Silberschatz et al., 2018, tr. 417–419).



Hình 4.8. Hai NUMA node: truy cập RAM cục bộ và đường truy cập từ xa

Nguồn: Tác giả mô hình hóa từ Silberschatz et al. (2018, tr. 418–419, Sec. 10.5.4).

$$ma_{NUMA} = q \times t_{local} + (1 - q) \times t_{remote}$$

q: tỷ lệ truy cập local (0–1); t_{locals}, t_{remote}: độ trễ local/remote (ns).

$$EAT_{NUMA} = (1 - p) \times \text{maNUMA} + p \times \text{Tpf} \quad (4.6)$$

Mở rộng kỳ vọng có trọng số phục vụ minh họa; không phải công thức đo kiểm NUMA của sách.

Nếu bộ nhớ được cấp xa CPU chạy tiến trình, độ trễ truy cập RAM bình thường có thể tăng; nếu tiến trình thường xuyên di chuyển giữa node, định vị tối ưu cũng thay đổi. Mô hình EAT_{NUMA} giả định p và Tpf giữ nguyên để cô lập ảnh hưởng locality, do đó không mô tả đầy đủ tranh chấp, điều phối và hàng đợi trong triển khai cụ thể (Silberschatz et al., 2018, tr. 418–419).

Ví dụ số học giả định: chọn tlocal = 150 ns và tremote = 450 ns. Khi q = 1,00, thời gian truy cập trung bình maNUMA = 150 ns; khi q = 0,50, maNUMA = 0,5 × 150 + 0,5 × 450 = 300 ns. Như vậy, chỉ riêng tỷ lệ truy cập xa tăng đã làm thành phần truy cập RAM tăng gấp hai trong mô hình này. Nếu tiếp tục giả định $p = 10^{-5}$ và Tpf = 8.000.000 ns, công thức (4.6) cho EAT_{NUMA} lần lượt xấp xỉ 230 ns và 380 ns. Đây là phép tính minh họa, không phải số đo của một hệ thống NUMA thực tế; giả định Tpf không đổi chỉ nhằm cô lập tác động của q.

Chuyển tiếp đến mục 4.7: khi các tiến trình cùng không được duy trì đủ trang thuộc locality hiện hành, việc lựa chọn nạn nhân và cấp khung dẫn tới page fault lặp. Working-set và PFF sẽ cung cấp hai cách nhận diện và điều tiết áp lực này (Silberschatz et al., 2018, tr. 419–425).

4.7. NGUYÊN NHÂN HÌNH THÀNH THRASHING (giáo trình: 10.6.1)

4.7.1. Phân biệt page fault thông thường và thrashing

Một lỗi trang là sự kiện bình thường trong bộ nhớ ảo phân trang theo yêu cầu; bản thân nó không chứng minh hệ thống bị thrashing. Trang mới sử dụng lần đầu hoặc trang của locality mới có thể chưa nằm trong RAM, nên phát sinh fault rồi được nạp một lần. Chỉ khi tiến trình không thể giữ trong RAM đủ các trang đang được dùng tích cực, việc thay một trang lại nhanh chóng khiến nó cần nạp trở lại, và quá trình này lặp nhiều lần, hệ thống mới rơi vào hoạt động phân trang quá mức. Giáo trình mô tả trạng thái một tiến trình dành nhiều thời gian cho paging hơn cho executing là thrashing (Silberschatz et al., 2018, tr. 419).

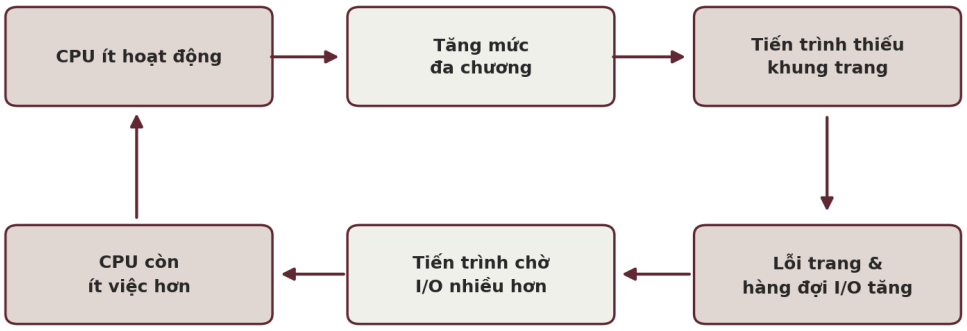
Khái niệm "đủ khung" cần được đặt trong bối cảnh nhu cầu làm việc hiện hành thay vì số khung tối thiểu kiến trúc. Một tiến trình có thể vẫn thực thi được về mặt nguyên tắc với ít khung, nhưng lại không giữ nổi nhiều trang cần truy cập liên tiếp. Kết quả là chuỗi page fault không chỉ xảy ra khi lần đầu chạm trang mà tái phát đối với những trang vừa bị loại khỏi RAM. Đây là lý do working-set – phép gần đúng những trang đang hoạt động – có vai trò giải thích nguyên nhân và phòng tránh thrashing hơn một phép đếm tổng số trang ảo của tiến trình.

Cần phân biệt ba mức: (i) fault đơn lẻ, có thể giải quyết bằng khung tự do; (ii) áp lực khung tạm thời, khiến tỷ lệ fault tăng nhưng chưa hình thành vòng lặp lặp lại kéo dài; và (iii) thrashing, khi phân trang chiếm ưu thế so với công việc tính toán có ích. Các mức này tạo thành một phổ vận hành; vì giáo trình không cung cấp một ngưỡng số duy nhất để kết luận tất cả hệ thống đều thrashing, việc dùng tỷ lệ lỗi trang đơn độc làm tiêu chí chẩn đoán tuyệt đối là thiếu cơ sở.

4.7.2. Vòng phản hồi của kịch bản đa chương và thay thế toàn cục

Tiểu mục 10.6.1 trong giáo trình xây dựng một kịch bản từ những hệ thống phân trang đời đầu. Trình quản lý quan sát CPU utilization: khi mức sử dụng CPU thấp, hệ điều hành tăng mức đa chương bằng cách đưa thêm tiến trình vào. Một tiến trình đang chạy chuyển sang giai đoạn làm việc mới và cần nhiều frame hơn. Do dùng thay thế toàn cục, tiến trình này có thể lấy khung đang chứa trang của tiến trình khác. Những tiến trình bị mất trang lập tức phát sinh fault để nạp lại. Hàng đợi thiết bị paging dài ra trong khi hàng đợi sẵn sàng sử dụng CPU giảm; CPU utilization tiếp tục giảm, kích hoạt quyết định tăng đa chương lần nữa. Vòng phản hồi này khiến throughput suy sụp (Silberschatz et al., 2018, tr. 419–420).

Đây không chỉ là một quan hệ cùng chiều giữa lỗi trang và độ trễ. Trình tự là chuỗi nguyên nhân: thiếu khung gây lỗi lặp; lỗi lặp làm I/O tăng; thời gian chờ I/O tăng khiến CPU nhàn rỗi; chính sách quan sát CPU thấp nhưng không phân biệt nguyên nhân lại đưa thêm tải vào RAM; tải mới làm thiếu khung nghiêm trọng hơn. Hình 4.9 diễn giải vòng phản hồi, trong đó mũi tên thể hiện tác động của cơ chế và chính sách điều độ trong kịch bản nguồn, không mô tả một vòng điều khiển bắt buộc tồn tại trên mọi hệ điều hành hiện đại.



Vòng phản hồi mô tả kịch bản lịch sử với cấp phát thiếu và thay thế toàn cục; không phải mọi hệ điều hành đều vận hành như vậy.

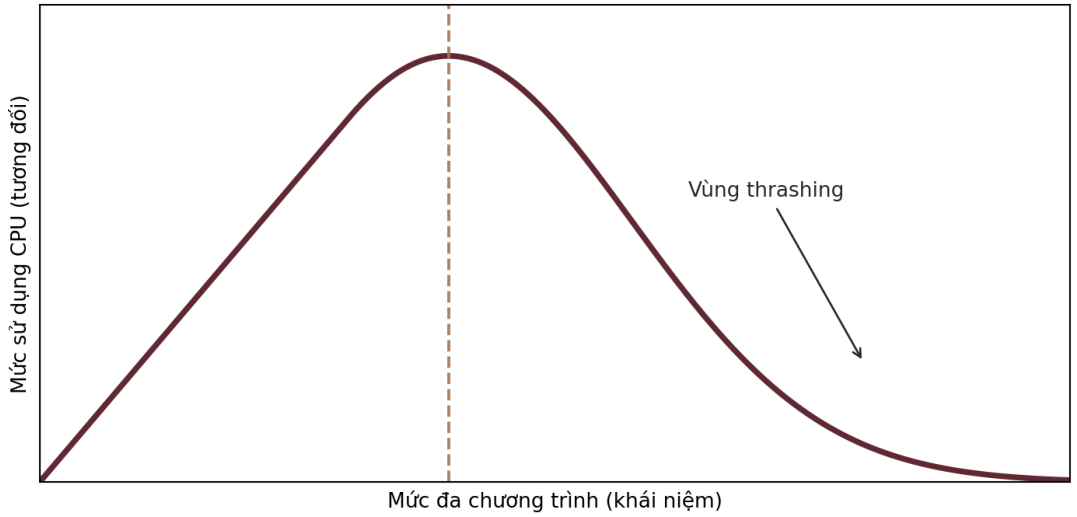
Hình 4.9. Vòng phản hồi gây thrashing theo kịch bản lịch sử của giáo trình

Nguồn: Tác giả chuyển thể mạch nhân quả từ Silberschatz et al. (2018, tr. 419–420).

Giáo trình cũng đề cập giới hạn của phương án thay thế cục bộ (local replacement). Việc giới hạn tiến trình trong tập khung của chính nó làm giảm nguy cơ một tiến trình hút mất khung của tiến trình khác; tuy nhiên nó không giải quyết triệt để nút thắt thiết bị phân trang. Khi nhiều tiến trình cùng thrashing, độ dài hàng đợi I/O tăng làm thời gian phục vụ fault của ngay cả tiến trình

không thrashing có thể lớn hơn. Điều này kết nối trực tiếp mục 4.7 với mô hình EAT ở mục 4.4: trong điều kiện xấu, hệ số T_{pf} không còn nên được xem bất biến (Silberschatz et al., 2018, tr. 420–421).

4.7.3. Đồ thị CPU utilization – mức đa chương



Hình 4.10. Xu hướng khái niệm của CPU utilization khi tăng mức đa chương

Nguồn: Tác giả phỏng theo ý nghĩa Hình 10.20 của Silberschatz et al. (2018, tr. 420); không phải số liệu đo.

Hình 10.20 của sách biểu diễn một đường cong khái niệm: lúc đầu, tăng mức đa chương giúp CPU có thêm việc để làm nên CPU utilization tăng; sau một điểm tối đa, việc nạp thêm tiến trình làm trầm trọng áp lực khung và phân trang, khiến mức sử dụng CPU giảm mạnh. Hình 4.10 diễn đạt lại cùng xu hướng mà không gắn giá trị số giả cho trục. Vì là sơ đồ khái niệm, không thể suy ra chính xác mức độ đa chương tối ưu hoặc xác suất fault tại điểm ngoặt từ hình này (Silberschatz et al., 2018, tr. 420).

Giải pháp lý thuyết mà giáo trình chỉ ra là giảm mức đa chương nếu hệ thống đang ở bên phải điểm cực đại, đồng thời cấp đủ khung cho các tiến trình còn hoạt động. Tuy nhiên, quyết định giảm tải phải dựa trên cách nhận diện nhu cầu trang thực sự, nếu không hệ điều hành có thể thu hồi khung của một tiến trình vẫn đang sử dụng các trang đó. Mô hình working-set trong tiểu mục kế tiếp cung cấp một đại lượng gần đúng cho nhu cầu này bằng tập trang được tham chiếu gần đây.

4.8. MÔ HÌNH TẬP LÀM VIỆC – WORKING-SET (giáo trình: 10.6.2)

4.8.1. Tính cục bộ và định nghĩa cửa sổ quan sát

Mô hình working-set xuất phát từ locality of reference: trong từng giai đoạn xử lý, chương trình thường tập trung truy cập một nhóm trang chứa mã lệnh và dữ liệu đang có ích. Khi chương trình chuyển hàm, thay đổi cấu trúc dữ liệu hoặc bước sang pha tính toán mới, nhóm trang được truy cập thường xuyên có thể thay đổi. Giáo trình minh họa hiện tượng này qua mô hình locality và chỉ ra rằng các nhóm trang của hai giai đoạn có thể giao nhau, chứ không nhất thiết tách biệt hoàn toàn (Silberschatz et al., 2018, tr. 421–422).

Để đo gần đúng nhóm trang đang hoạt động, mô hình working-set chọn một cửa sổ Δ tham chiếu bộ nhớ gần nhất. Tập hợp các số trang khác nhau có mặt trong cửa sổ này tại thời điểm t được ký hiệu ở đây là $W_i(t, \Delta)$ đối với tiến trình i. Kích thước $WSS_i(t, \Delta)$ bằng số phần tử của tập đó, không phải số lần xuất hiện trang trong cửa sổ và cũng không phải tổng số trang được khai báo trong không gian ảo. Nếu một trang tiếp tục xuất hiện trong cửa sổ, nó vẫn được tính một

phần tử; sau đủ Δ tham chiếu không được dùng, nó sẽ rời tập đang xét (Silberschatz et al., 2018, tr. 422–424).

$$\mathbf{WSS_i(t, \Delta) = |W_i(t, \Delta)|} \quad \textbf{(4.7)}$$

$WSS_i(t, \Delta)$ là số trang phân biệt trong cửa sổ Δ tham chiếu tại thời điểm t, đơn vị trang.

Hình 10.22 của sách trình bày chuỗi tham chiếu và hai vị trí cửa sổ Δ = 10. Ở một thời điểm, working-set gồm {1, 2, 5, 6, 7}; ở thời điểm về sau, nhóm trang chuyển sang {3, 4}. Ví dụ cho thấy working-set không bất biến trong suốt vòng đời tiến trình. Khi phân tích chính sách cấp phát khung, điều quan trọng là theo dõi biến thiên của tập làm việc, vì cấp đủ khung cho giai đoạn trước không bảo đảm đủ cho giai đoạn kế tiếp (Silberschatz et al., 2018, tr. 422).

Cửa sổ Δ phải được lựa chọn thận trọng. Nếu quá ngắn, nó có thể bỏ sót một số trang thuộc cùng locality nhưng không được tham chiếu trong khoảng rất gần; ước lượng nhu cầu sẽ thấp và khiến hệ điều hành dễ thu hồi quá mức. Nếu quá dài, cửa sổ có thể chồng lấp nhiều localities khác nhau; WSS bị tăng bởi các trang không còn thiết yếu ở giai đoạn hiện tại, làm giảm cơ hội sử dụng hiệu quả khung ở tiến trình khác. Không có một Δ chung luôn thích hợp cho mọi chương trình; đây là độ đánh đổi giữa độ nhạy và độ ổn định của phép ước lượng (Silberschatz et al., 2018, tr. 422–423).



Hình 4.11. Cửa sổ sáu tham chiếu gần nhất trên chuỗi truy cập minh họa

Nguồn: Tác giả xây dựng chuỗi và tự tính W(t); vận dụng định nghĩa trong Silberschatz et al. (2018, tr. 422).

4.8.2. Tính toán working-set và tổng nhu cầu khung

Để minh họa thao tác cửa sổ trượt, xét chuỗi giả định R = [1, 2, 1, 3, 2, 4, 5, 4, 5, 6, 5, 6, 7, 6, 7, 8, 7, 8] và Δ = 6 tham chiếu. Ở t = 6, sáu giá trị gần nhất là [1, 2, 1, 3, 2, 4], nên working-set là {1, 2, 3, 4}, WSS = 4. Ở t = 12, cửa sổ chứa [5, 4, 5, 6, 5, 6], suy ra W = {4, 5, 6}, WSS = 3. Đến t = 18, cửa sổ [7, 6, 7, 8, 7, 8] tạo W = {6, 7, 8}, WSS = 3. Tất cả các chuỗi trong ví dụ này là dữ liệu tự dựng; kết quả chỉ minh họa quy tắc tập hợp, không được hiểu như trace thu từ một tiến trình thực.

Bảng 4.8. Working-set với Δ = 6 trên chuỗi tham chiếu tự dựng (số liệu giả định)

t	Sáu tham chiếu gần nhất	W(t, 6)	WSS
6	1 2 1 3 2 4	{1, 2, 3, 4}	4
8	1 3 2 4 5 4	{1, 2, 3, 4, 5}	5
10	2 4 5 4 5 6	{2, 4, 5, 6}	4
12	5 4 5 6 5 6	{4, 5, 6}	3
14	5 6 5 6 7 6	{5, 6, 7}	3
16	5 6 7 6 7 8	{5, 6, 7, 8}	4

t	Sáu tham chiếu gần nhất	W(t, 6)	WSS
18	7 6 7 8 7 8	{6, 7, 8}	3

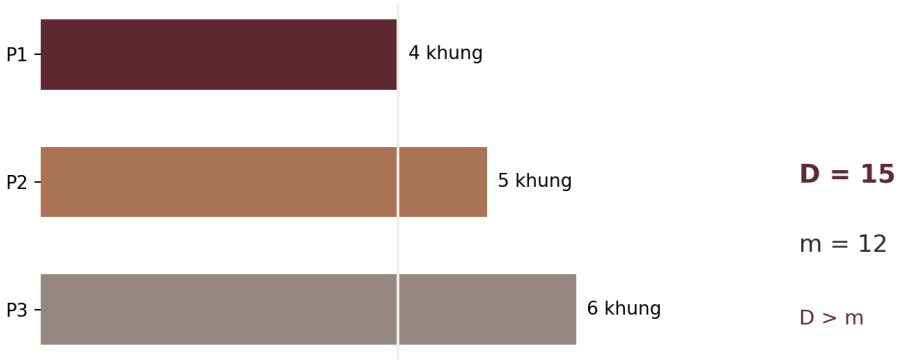
Nguồn: Tác giả tự dựng chuỗi và tính lại bằng phép loại trùng; dựa trên định nghĩa của Silberschatz et al. (2018, tr. 422–423).

Bảng 4.8 cho thấy khi cửa sổ dịch dần, có thời điểm tập làm việc chứa bốn trang nhưng ở các mốc sau chỉ còn ba trang. Đây không phải bằng chứng rằng RAM cần cấp giảm tuyến tính theo thời gian: diễn biến phụ thuộc trực tiếp vào chuỗi tham chiếu được quan sát và độ dài Δ. Đặc biệt, một trang có thể xuất hiện ba lần trong sáu tham chiếu nhưng vẫn chỉ được tính một trang trong WSS. Sự khác nhau giữa số lần truy cập và số trang phân biệt là yếu tố quan trọng khi thiết kế mô phỏng để tránh thổi phồng tổng nhu cầu khung.

Giả sử hệ thống có n tiến trình cùng hoạt động. Đặt D là tổng kích thước working-set của chúng tại thời điểm quan sát và m là số khung vật lý khả dụng cho các tiến trình này. Theo giáo trình, nếu D > m thì không thể đáp ứng đồng thời đầy đủ mọi working-set theo giả định mô hình và nguy cơ thrashing phát sinh do một số tiến trình thiếu trang đang dùng. Nếu D ≤ m, mô hình cho phép xem xét duy trì đồng thời các tiến trình; tuy nhiên điều kiện đó là điều kiện phục vụ nhu cầu tại một thời điểm, không phải sự bảo đảm tuyệt đối về hiệu năng sau khi các locality biến động (Silberschatz et al., 2018, tr. 422–423).

$$D(t) = \sum_i WSS_i(t, \Delta)\ ;\ \text{nếu } D(t) > m \text{ thì nhu cầu vượt số khung khả dụng.} \quad (4.8)$$

D là tổng nhu cầu khung của các tập làm việc; m là tổng khung khả dụng, cùng đơn vị khung.



Hình 4.12. Ví dụ tổng working-set D = 15 khung trong khi m = 12 khung

Nguồn: Tác giả xây dựng tình huống giả định để minh họa bất đẳng thức của Silberschatz et al. (2018, tr. 423).

Trong tình huống của Hình 4.12, giả định có ba tiến trình với WSS lần lượt là 4, 5 và 6; tổng nhu cầu là 15 khung trong khi chỉ có 12 khung có thể phân phối. Nếu buộc cả ba tiếp tục chạy mà không thay đổi nhu cầu của từng tiến trình, hệ điều hành không thể giữ toàn bộ tập làm việc của cả ba trong RAM. Một hướng xử lý trong mô hình sách là tạm dừng một tiến trình, giải phóng khung và phân phối lại cho các tiến trình còn chạy, rồi khôi phục tiến trình bị tạm dừng khi tài nguyên cho phép. Quyết định chọn tiến trình cụ thể sẽ phụ thuộc chính sách cấp phát mà báo cáo không tự đề xuất như một quy luật chung (Silberschatz et al., 2018, tr. 423).

4.8.3. Khả năng quan sát và chi phí xấp xỉ

Bài toán khó là working-set lý thuyết được định nghĩa trên mọi tham chiếu nhớ gần nhất, trong khi hệ điều hành không dễ ghi nhận chính xác từng lần truy cập mà không trả giá lớn. Giáo trình mô tả một cách xấp xỉ: dùng reference bit và ngắt đồng hồ định kỳ. Với ví dụ Δ = 10.000 tham chiếu và ngắt sau mỗi 5.000 tham chiếu, hệ điều hành lưu lại trạng thái bit sử dụng rồi xóa chúng, để về sau

xác định trang có khả năng từng được dùng trong cửa sổ gần đây. Phép xấp xỉ bị hạn chế về độ phân giải thời gian; tăng số bit lịch sử và tần suất ngắt giúp tăng độ chính xác nhưng cũng tăng chi phí phục vụ (Silberschatz et al., 2018, tr. 424).

Chi phí quan sát đặt ra một đánh đổi thiết kế: policy tốt trên mô hình toán học chưa chắc có thể thực hiện miễn phí. Cần phân biệt kết quả lý tưởng "working-set có bao nhiêu trang" với kết quả đo gần đúng trong hạt nhân hệ điều hành, phụ thuộc phần cứng hỗ trợ bit tham chiếu và chu kỳ lấy mẫu. Báo cáo không đồng nhất ví dụ xấp xỉ của sách với một thuật toán mặc định của Linux hay Windows hiện hành. Điều có thể rút ra là cách ước lượng locality cần vừa đủ tin cậy để cấp khung, vừa đủ tiết kiệm chi phí để bản thân thao tác đo không trở thành tải đáng kể.

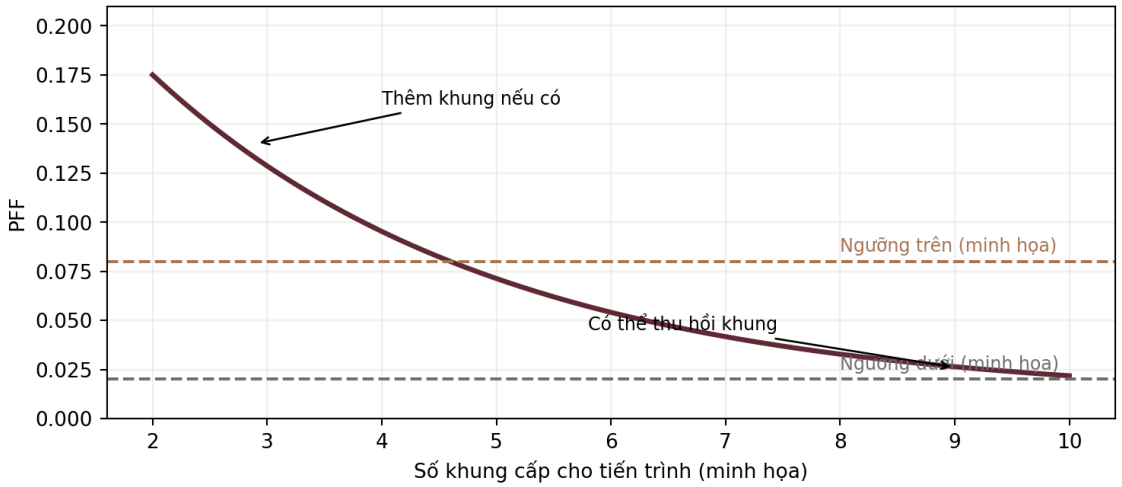
Tiểu kết: working-set trả lời câu hỏi "tiến trình đang cần khoảng bao nhiêu khung?" trên cơ sở các trang khác nhau xuất hiện trong cửa sổ tham chiếu gần nhất. Nó cung cấp một ước lượng mang ý nghĩa cấu trúc truy cập, nhưng phải chọn Δ và chấp nhận sai số quan sát. Phần kế tiếp giới thiệu hướng kiểm soát trực tiếp hơn qua đại lượng lỗi trang quan sát được, không cố xây dựng toàn bộ tập các trang đang hoạt động.

4.9. TẦN SUẤT LỖI TRANG – PAGE-FAULT FREQUENCY (giáo trình: 10.6.3)

4.9.1. Tín hiệu phản hồi và chính sách hai ngưỡng

Page-fault frequency (PFF) sử dụng một tín hiệu khác working-set. Thay vì theo dõi chính xác tập trang được sử dụng trong một cửa sổ, nó quan sát tần suất xuất hiện page fault của tiến trình. Tư tưởng là nếu fault xảy ra quá thường xuyên, tiến trình có thể đang thiếu khung so với locality hiện tại; nếu fault rất ít, tiến trình có thể đang nắm giữ nhiều khung hơn mức cần thiết. Giáo trình đưa ra chính sách thiết lập cận trên và cận dưới cho tần suất fault, rồi điều chỉnh số khung cấp theo hai ngưỡng này (Silberschatz et al., 2018, tr. 424–425; Nông Minh Ngọc & Nguyễn Văn Huy, 2016, tr. 146–147).

Cụ thể, khi tần suất quan sát vượt cận trên, hệ điều hành cấp thêm khung cho tiến trình nếu còn khả năng. Khi tần suất thấp hơn cận dưới, một khung có thể được thu hồi để tái phân phối. Nếu fault quá cao mà không có frame tự do, giáo trình mô tả việc có thể tạm dừng (swap out) một tiến trình, từ đó giải phóng khung cho các tiến trình đang thiếu. Đây là nguyên lý chính sách trên hệ điều hành khái niệm; tài liệu không đưa ra hai trị số ngưỡng số học phổ quát (Silberschatz et al., 2018, tr. 424–425).



Hình 4.13. Đường PFF theo số khung và hai mức phản hồi (trục minh họa)

Nguồn: Tác giả dựng lại logic hai ngưỡng từ Silberschatz et al. (2018, tr. 424–425, Hình 10.23); trị số ngưỡng giả định.

Hình 4.13 thể hiện logic "thêm khung khi thiếu, thu hồi khi dư" bằng hai đường ngang. Hình không chứng minh rằng PFF giảm theo đúng một đường cong cho mọi ứng dụng; đường cong chỉ

giúp hình dung xu hướng trong một tình huống giả định. Khi dùng trong báo cáo khoa học, cần chỉ ra đơn vị của trục tần suất và phương pháp lấy mẫu. Chẳng hạn, có thể tính tỷ lệ lỗi trang bằng số fault chia số lần tham chiếu trong khoảng quan sát; hoặc hệ thống có thể theo dõi fault trên đơn vị thời gian. Hai chỉ số này không tự tương đương khi cường độ truy cập nhớ thay đổi. Ở đây lựa chọn tỷ lệ fault trên số tham chiếu chỉ để minh họa tính toán, không gán nó cho một cơ chế sản phẩm cụ thể.

PFF□ = số page fault trong cửa sổ k / số tham chiếu trong cửa sổ k (4.9)

PFF_k là tỷ lệ lỗi trang trên số tham chiếu trong cửa sổ k, không có đơn vị.

4.9.2. Ví dụ điều tiết bằng số liệu giả định

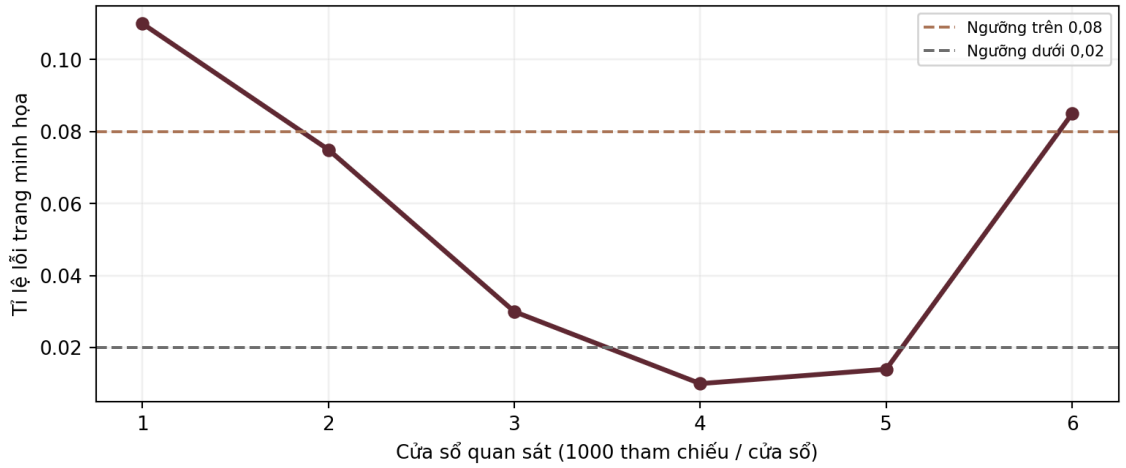
Giả sử một tiến trình được theo dõi trong sáu cửa sổ, mỗi cửa sổ có 1.000 tham chiếu trang. Đặt cận trên U = 0,08 và cận dưới L = 0,02, hoàn toàn do người viết chọn để minh họa. Số lần fault lần lượt là 110, 75, 30, 10, 14, 85, tương đương các tỷ lệ 0,110; 0,075; 0,030; 0,010; 0,014; 0,085. Theo chính sách cơ bản, cửa sổ 1 và 6 là trường hợp vượt ngưỡng U; cửa sổ 4 và 5 thấp hơn ngưỡng L. Các cửa sổ còn lại thuộc vùng tần suất được chấp nhận trong mô hình hai ngưỡng này.

Ví dụ phải được hiểu như mô phỏng phép ra quyết định, không phải quá trình điều chỉnh closed-loop thực tế. Sau khi cấp hoặc thu hồi một khung, số fault ở cửa sổ tiếp theo có thể thay đổi; trong chuỗi minh họa, các giá trị quan sát đã được cho sẵn để giải thích cách xác định điều kiện ngưỡng, không được suy diễn rằng thao tác cấp khung tạo ra chính xác những giá trị PFF tiếp theo. Nếu muốn kiểm chứng nhân quả đó cần mô phỏng đầy đủ reference string và chính sách quản lý khung, hoặc đo một hệ thống thực với điều kiện kiểm soát rõ ràng.

Bảng 4.9. Theo dõi PFF và phản hồi hai ngưỡng với U = 0,08; L = 0,02 (số liệu giả định)

Cửa sổ	Lỗi trang / 1000	PFF	Nhận định theo ngưỡng
1	110	0.110	Trên U: xét cấp thêm khung
2	75	0.075	Trong dải: theo dõi
3	30	0.030	Trong dải: theo dõi
4	10	0.010	Dưới L: xét thu hồi khung
5	14	0.014	Dưới L: xét thu hồi khung
6	85	0.085	Trên U: xét cấp thêm khung

Nguồn: Tác giả tự chọn 1.000 tham chiếu/cửa sổ, các số fault và ngưỡng; phép tính theo nguyên lý PFF của Silberschatz et al. (2018, tr. 424–425).



Hình 4.14. Diễn biến PFF theo cửa sổ quan sát và hai ngưỡng giả định

Nguồn: Tác giả tính từ Bảng 4.9; mô phỏng logic, không phải số liệu quan sát hệ thống.

Hình 4.14 cho thấy nguy cơ của việc quyết định từ một cửa sổ đơn lẻ: tại một khoảng, PFF giảm dưới L; nhưng ở khoảng cuối, nó lại tăng vượt U. Nếu chính sách vừa thu hồi một khung ngay sau khoảng thấp thì có thể khiến việc chuyển sang locality mới phát sinh nhiều fault hơn. Mô hình cơ bản trong giáo trình không ấn định cơ chế làm trơn, thời gian chờ giữa hai lần điều chỉnh hay ngưỡng bất đối xứng; đây là giới hạn khi chuyển nguyên lý sang hệ thống vận hành cần tránh phản ứng quá nhạy với nhiễu thống kê.

Không thể xếp hạng PFF và working-set nếu thiếu thực nghiệm so sánh. Working-set mô tả một tập trang theo cửa sổ tham chiếu; PFF đo hậu quả của cách phân phối khung thông qua lỗi trang. Working-set có thể cần chi phí quan sát tham chiếu; PFF có vẻ trực tiếp hơn nhưng cũng chỉ phản ánh tỷ lệ lỗi sau khi tình huống đã xảy ra. Cả hai phải chọn độ dài khoảng quan sát và đều chịu ảnh hưởng khi tiến trình chuyển locality.

4.9.3. Khi không còn khung tự do và giới hạn chính sách

Giả sử một tiến trình có PFF vượt cận trên nhưng danh sách free-frame đã cạn. Một chính sách chỉ biết "tăng khung khi fault cao" sẽ không thể thực hiện trực tiếp; sách mô tả giải pháp tạm dừng một tiến trình khác và phân phối lại khung được giải phóng. Điều này cho thấy PFF không đứng độc lập với điều tiết mức đa chương và khả năng reclaim. Khi tổng nhu cầu khung tăng đồng thời ở nhiều tiến trình, phản hồi cục bộ bằng cách lấy khung của tiến trình khác mà không xem xét hệ quả có thể tạo ra vòng lặp tương tự kịch bản ở mục 4.7 (Silberschatz et al., 2018, tr. 425).

Mặt khác, tỷ lệ fault rất thấp không nhất thiết chứng minh tất cả khung của tiến trình đều không cần thiết. Có thể trong khoảng quan sát tiến trình tạm ngừng hoặc chỉ thực hiện một pha tính toán cục bộ; sau đó các trang khác lại trở nên có ích. Khi diễn giải PFF, phải nói rõ "có thể dư khung" theo ngôn ngữ xác suất và điều kiện thời gian, thay vì coi cận dưới là chứng cứ tuyệt đối buộc thu hồi.

4.10. THỰC TIỄN XỬ LÝ THRASHING (giáo trình: 10.6.4)

4.10.1. Thông điệp thực tiễn được nguồn gốc xác nhận

Tiểu mục 10.6.4 của Operating System Concepts ấn bản 10 chỉ gồm một đoạn ngắn nhưng có ý nghĩa phương pháp luận rõ rệt. Tác giả nhấn mạnh thrashing và sự trao đổi trang kéo theo chi phí hiệu năng lớn; từ đó, trong điều kiện cho phép, thực tiễn thiết kế hệ thống nên trang bị đủ bộ nhớ vật lý để các working-set cùng tồn tại trong RAM, tránh paging và swapping quá mức trừ những thời điểm chịu

tải cực đoan. Sách nêu nguyên lý này cho dải thiết bị từ điện thoại thông minh tới máy chủ lớn (Silberschatz et al., 2018, tr. 425).

Đây là khuyến nghị theo bối cảnh giáo trình 2018, không phủ nhận giá trị của working-set hay PFF. Các mô hình tại các mục 4.8 và 4.9 vẫn giúp giải thích khi nào RAM thực tế không còn đủ, cách phân phối khung ảnh hưởng đến lỗi trang và vì sao giảm tải hoặc tăng bộ nhớ có thể cần thiết. Khuyến nghị của sách phải được đọc cùng mệnh đề điều kiện "whenever possible" – khi khả thi – bởi RAM là tài nguyên có chi phí và không có cấu hình nào tránh được mọi tình huống tải công việc biến động trong thực tiễn.

4.10.2. Từ khuyến nghị đến giới hạn ứng dụng

Xét trên bình diện vận hành, nguyên tắc "đủ RAM cho các working-set đồng thời" có thể được chuyển thành câu hỏi năng lực: ở một mức đa chương và một tập công việc nhất định, tổng nhu cầu khung đang hoạt động có thấp hơn khung vật lý khả dụng hay không? Nếu D > m kéo dài, yêu cầu giảm số tiến trình hoạt động hoặc bổ sung bộ nhớ có cơ sở theo mô hình. Nếu D không vượt m nhưng page fault vẫn tăng theo từng đợt, cần khảo sát chuyển đổi locality, trạng thái trang và chi phí xử lý lỗi trước khi gắn nhãn toàn hệ thống thrashing. Các đánh giá này là suy luận từ mô hình nguồn chứ không phải số đo cụ thể một hệ điều hành.

Năng lực bộ nhớ cần được diễn giải theo tải thay đổi theo thời gian. Đủ RAM ở một thời điểm chưa thể đảm bảo lúc khác vì working-set thay đổi theo nhịp truy cập. Trên hệ thống nhiều ứng dụng, có thể xảy ra những pha khởi động đồng thời, yêu cầu dữ liệu lớn hoặc biến thiên trong mức sử dụng, khiến cần theo dõi cả tỷ lệ page fault và thời gian đáp ứng. Nguồn gốc không cung cấp các ngưỡng cảnh báo thời gian thực, do đó báo cáo không đặt ra một mức RAM tối thiểu cố định hay phát biểu "hệ điều hành X luôn cần Y GB RAM".

Quan sát môi trường Linux — bộ nhớ hiện thời

```
$ free -h
               total        used        free      shared  buff/cache   available
Mem:           5.8Gi       1.3Gi       4.5Gi        12Ki       253Mi       4.5Gi
Swap:             0B          0B          0B

$ vmstat 1 2   # bản ghi thời điểm sau cùng
 0  0      0 4765656   6028 253808    0    0     0     0  131  102  0  0 100  0  0  0
```

Hình 4.15. Thông tin bộ nhớ và hoạt động hệ thống quan sát bằng free -h, vmstat

Nguồn: Tác giả thực thi các lệnh free -h và vmstat trên môi trường Linux kiểm thử; hình trình bày lại đầu ra lệnh bằng phông chữ đơn cách (quan sát ngày 16/10/2026).

Dữ liệu cho thấy kết quả quan sát trạng thái bộ nhớ tại một thời điểm, ví dụ dung lượng RAM hiện có và việc môi trường kiểm tra không cấu hình swap. Đây không phải hệ thống của người học, cũng không phải phép đo thrashing. Cách quan sát chỉ có ý nghĩa khi gắn với điều kiện tải và chuỗi thời gian như lý thuyết tại các mục 4.7–10.6.3.

4.11. TỔNG HỢP, PHẢN BIỆN VÀ KẾT LUẬN PHẦN 4

4.11.1. Tổng hợp quan hệ giữa cơ chế, chỉ báo và chính sách

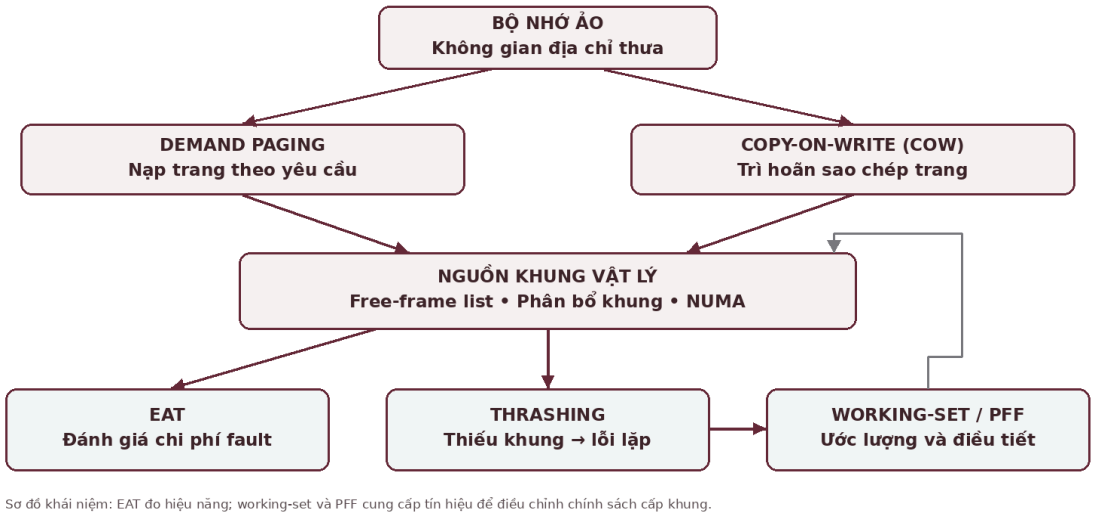
Các tiểu mục tạo thành chuỗi cơ chế: bộ nhớ ảo cho phép biểu diễn không gian địa chỉ thưa; demand paging trì hoãn nạp trang; COW trì hoãn sao chép; free-frame list và chính sách phân bổ quyết định cách sử dụng RAM. EAT lượng hóa độ trễ do lỗi trang; working-set và PFF mô tả nhu cầu và tín hiệu điều tiết khi áp lực khung tăng (Silberschatz et al., 2018, tr. 389–400, 413–425).

Working-set và PFF tiếp cận cùng bài toán bằng hai tín hiệu khác nhau. Working-set dùng các tham chiếu gần nhất để ước lượng những trang cần giữ; PFF dùng hậu quả của tình trạng thiếu trang – số lỗi phát sinh trên khoảng quan sát – để điều chỉnh khung. Cả hai không loại bỏ việc phải ra quyết định phân phối RAM hữu hạn giữa nhiều tiến trình. Trong mô hình, chúng có thể giúp kiểm soát số khung và mức đa chương, nhưng vẫn cần xét độ trễ của tín hiệu, khả năng cấp khung thực tế và chi phí của cơ chế giám sát (Silberschatz et al., 2018, tr. 422–425).

Bảng 4.10. Vai trò chín thành phần trong chuỗi quản lý bộ nhớ ảo và thrashing

Thành phần	Tín hiệu/chức năng	Lợi ích phân tích	Giới hạn chính
Nền tảng bộ nhớ ảo	Không gian địa chỉ thưa, ánh xạ ảo–vật lý	Tách không gian logic khỏi RAM, hỗ trợ dùng chung	Địa chỉ ảo lớn không bảo đảm có đủ RAM
Demand paging	Trang được tham chiếu, trạng thái valid/invalid	Tránh nạp trang chưa dùng	Chi phí fault và khả năng khởi động lại lệnh
Free-frame list	Số khung vật lý sẵn sàng cấp	Đáp ứng yêu cầu nạp hoặc cấp trang	Không đo được tính cục bộ truy cập
Copy-on-Write (COW)	Bảo vệ ghi và ngoại lệ COW sau fork()	Trì hoãn sao chép; giảm nhu cầu khung ban đầu	Chi phí trap và tạo bản sao khi có ghi
Phân bổ khung và NUMA	Số khung tối thiểu; tỷ lệ chia; vị trí node RAM	Điều tiết khung; cải thiện locality bộ nhớ	Phụ thuộc chính sách thay thế và điều phối CPU
EAT	p, ma, T_{pf}	Lượng hóa chi phí truy cập kỳ vọng	Phụ thuộc giả định của thời gian phục vụ fault
Thrashing	Lỗi trang lặp, phân trang áp đảo thực thi	Xác định hệ quả thiếu khung đang dùng	Không có ngưỡng số tuyệt đối chung
Working-set	Tập trang trong cửa sổ Δ	Ước lượng nhu cầu khung tích cực	Chọn Δ và chi phí theo dõi
PFF	Lỗi trang trong một cửa sổ quan sát	Điều tiết khung theo tín hiệu phản hồi	Nhiễu quan sát và cách chọn ngưỡng

Nguồn: Tác giả tổng hợp từ Silberschatz et al. (2018, tr. 389–401, 413–425) và Nông Minh Ngọc & Nguyễn Văn Huy (2016, tr. 128–132, 143–147).



Hình 4.16. Chuỗi cơ chế, giới hạn tài nguyên và tín hiệu kiểm soát thrashing

Nguồn: Tác giả mô hình hóa từ Silberschatz et al. (2018, các mục 10.1–10.3 và 10.5–10.6, tr. 389–425); sơ đồ tổng hợp lý thuyết.

4.11.2. Những điều mô hình chứng minh được và chưa chứng minh được

Thứ nhất, từ mô hình EAT, có thể chứng minh theo đại số rằng nếu giữ *ma* và T_{pf} cố định, EAT tăng tuyến tính với p và độ dốc bằng $T_{pf} - ma$. Ví dụ 200 ns và 8 ms trong sách cho thấy độ dốc lớn và xác suất fault phải rất nhỏ mới duy trì thời gian truy cập gần mức không fault. Tuy

nhiên, kết quả chỉ là kết luận điều kiện của mô hình; không thể suy ra rằng máy tính cụ thể sẽ chậm đi cùng hệ số trong mọi tải. Những số liệu thực nghiệm đòi hỏi thông tin về kiến trúc, thiết bị lưu trữ, cấu hình kernel, tải đồng thời và phép đo thực tế.

Thứ hai, working-set cung cấp một đại lượng xác định về mặt toán học trên trace tham chiếu: WSS là số trang phân biệt trong cửa sổ Δ. Tổng D cho phép so sánh nhu cầu đồng thời với số khung m. D > m là tín hiệu không thể giữ tất cả working-set trong RAM theo giả định đang dùng; đây là giải thích có sức nặng cho nguy cơ thrashing. Nhưng D ≤ m không bảo đảm page fault bằng zero vì trang có thể chưa được nạp hoặc các tập làm việc có thể đổi sau khi tính. Sự khác biệt giữa điều kiện năng lực bộ nhớ và trạng thái vận hành tại thời điểm cụ thể cần được giữ rõ trong diễn giải.

Thứ ba, chính sách PFF hai ngưỡng đưa ra mối liên hệ phản hồi hợp lý giữa lỗi trang và cấp khung. Mô phỏng của Bảng 4.9 minh họa cách áp dụng điều kiện ngưỡng; nó không chứng minh tốc độ hội tụ hoặc độ ổn định của một bộ điều khiển thật. Nếu cửa sổ đo quá ngắn, các quyết định có thể bị kích hoạt bởi nhiễu; nếu quá dài, phản ứng có thể chậm khi locality thay đổi. Vì giáo trình không cung cấp thuật toán tinh chỉnh các tham số đó hay tập số đo thực tế, báo cáo không tuyên bố đã tối ưu hóa ngưỡng cho một hệ điều hành cụ thể.

Thứ tư, công thức phân bổ theo tỷ lệ và mô hình EAT_{NUMA} định lượng được hai khía cạnh khác nhau: số khung chia theo kích thước không gian ảo và chi phí truy cập phụ thuộc vị trí node nhớ. Tuy nhiên, kích thước không gian ảo si không nhất thiết tỷ lệ thuận với tập làm việc WSSi tại mọi thời điểm. Global Replacement có thể cải thiện thông lượng nhờ sử dụng linh hoạt nguồn khung, nhưng khi áp lực bộ nhớ lớn, việc lấy khung của tiến trình khác có thể làm tăng lỗi trang dây chuyền. Với NUMA, lợi ích độ trễ local còn phụ thuộc vào việc điều phối CPU và vị trí khung; các số liệu minh họa ở mục 4.6.4 chưa phải phép đo kiểm chứng thực tế (Silberschatz et al., 2018, tr. 414–419).

4.11.3. Trả lời câu hỏi nghiên cứu và hướng kiểm chứng tiếp theo

Các câu hỏi nghiên cứu được trả lời theo chuỗi: (1) bộ nhớ ảo tách địa chỉ logic khỏi vị trí vật lý và chỉ hiện thực hóa trang khi phù hợp; (2) demand paging phục vụ trang vắng mặt, còn COW tách trang đang chia sẻ khi ghi; (3) EAT tăng theo xác suất và chi phí fault trong mô hình giả định; (4) mức phân bổ khung cùng hành vi locality quyết định nguy cơ lỗi lặp và thrashing. Cơ sở chính: Silberschatz et al. (2018, tr. 389–400, 413–425).

Hướng nghiên cứu tiếp theo là thu thập trace và số liệu page fault trên một máy có cấu hình xác định; thay đổi có kiểm soát tải đồng thời, RAM khả dụng, cửa sổ working-set và ngưỡng PFF để đối chiếu dự đoán EAT với số đo. Những phép kiểm chứng này chưa được thực hiện trong báo cáo.

Tóm lại, demand paging tiết kiệm RAM nhờ nạp trang theo nhu cầu, nhưng lợi ích phụ thuộc vào việc duy trì đủ khung cho những trang đang hoạt động. Khi áp lực bộ nhớ kéo dài, chi phí page fault và thrashing làm giảm hiệu quả thực thi. Vì vậy, phải đánh giá đồng thời nhu cầu locality, xác suất fault, chi phí I/O và khả năng cấp thêm RAM hoặc giảm tải (Silberschatz et al., 2018, tr. 425).

TÀI LIỆU THAM KHẢO

Arpaci-Dusseau, R. H., & Arpaci-Dusseau, A. C. (2018). Operating systems: Three easy pieces (Version 1.00). Arpaci-Dusseau Books.

Nông Minh Ngọc, & Nguyễn Văn Huy. (2016). Giáo trình nguyên lý hệ điều hành. Nhà xuất bản Đại học Thái Nguyên.

Silberschatz, A., Galvin, P. B., & Gagne, G. (2018). Operating system concepts (10th ed.). John Wiley & Sons.

Phần 5: Thay thế trang

MỤC	NỘI DUNG
10.4.1	Thay thế trang cơ bản
10.4.2–10.4.4	FIFO, OPT, LRU, Belady's anomaly
10.4.5	Xấp xỉ LRU: Second-Chance (Clock)
10.4.6	Thuật toán dựa trên đếm (LFU, MFU)
10.4.7	Thuật toán đệm trang (page buffering)
10.4.8	Ứng dụng tự quản lý bộ nhớ (ví dụ cơ sở dữ liệu)

10.4 Page Replacement

Phân trang theo yêu cầu cho phép tiến trình thực thi mà không cần nạp toàn bộ các trang vào bộ nhớ chính. Nhờ chỉ lưu giữ những trang cần thiết, hệ điều hành có thể tiết kiệm bộ nhớ và tăng số tiến trình hoạt động đồng thời. Tuy nhiên, nhu cầu sử dụng bộ nhớ có thể thay đổi trong quá trình thực thi. Chẳng hạn, hệ thống có 40 khung có thể đáp ứng sáu tiến trình nếu mỗi tiến trình đang sử dụng 5 trang, nhưng sẽ thiếu bộ nhớ nếu cả sáu cùng cần đến 10 trang. Bên cạnh đó, các bộ đệm vào/ra cũng chiếm một phần bộ nhớ, làm giảm số khung có thể cấp cho tiến trình. Vì vậy, khi xảy ra lỗi trang, hệ điều hành có thể xác định được trang cần nạp trên thiết bị lưu trữ nhưng không còn khung trống để chứa trang đó.

Trong tình huống này, chấm dứt tiến trình không phù hợp với mục tiêu duy trì hoạt động của hệ thống, còn hoán đổi toàn bộ tiến trình có thể gây chi phí lớn do phải di chuyển nhiều dữ liệu. Thay thế trang cung cấp cách xử lý ở phạm vi nhỏ hơn: lựa chọn một trang đang thường trú để loại bỏ, bảo toàn nội dung nếu cần và sử dụng khung được giải phóng để nạp trang yêu cầu. Tuy nhiên, trang bị loại bỏ vẫn có thể được tham chiếu trở lại, khiến hệ thống phát sinh thêm lỗi trang. Do đó, mỗi trang

không nhất thiết chỉ gây lỗi một lần khi được truy cập đầu tiên. Vấn đề cốt lõi của thay thế trang là lựa chọn trang nạn nhân hợp lý nhằm đáp ứng nhu cầu bộ nhớ hiện tại, đồng thời hạn chế việc phải nạp lại dữ liệu trong tương lai. Đây là cơ sở để xây dựng và đánh giá các thuật toán thay thế trang.

10.4.1. Basic Page Replacement

Thay thế trang cơ bản mở rộng quy trình xử lý lỗi trang để đáp ứng trường hợp bộ nhớ không còn khung trống. Trước hết, hệ điều hành xác định vị trí của trang được yêu cầu trên thiết bị lưu trữ thứ cấp và tìm một khung để nạp trang. Nếu có khung trống, hệ thống sử dụng trực tiếp khung đó; nếu không, thuật toán thay thế được thực hiện để lựa chọn một trang nạn nhân. Nội dung của trang nạn nhân được ghi ra thiết bị lưu trữ khi cần thiết, đồng thời bảng trang và bảng khung được cập nhật để phản ánh việc trang không còn thường trú. Sau đó, trang được yêu cầu được nạp vào khung vừa giải phóng, thông tin ánh xạ được điều chỉnh và tiến trình tiếp tục từ lệnh đã gây lỗi trang. Như vậy, cơ chế này thay đổi trang được lưu trong một khung vật lý nhưng vẫn bảo toàn nội dung và tính nhất quán của không gian địa chỉ mà tiến trình sử dụng.

Chi phí đáng kể của quá trình thay thế nằm ở các thao tác vào/ra. Nếu phải ghi trang nạn nhân ra ngoài rồi đọc trang mới vào, việc xử lý lỗi trang cần hai lần chuyển trang thay vì chỉ một lần đọc. Để tránh thao tác ghi không cần thiết, hệ thống sử dụng **bit sửa đổi**, còn gọi là **bit bẩn** (*modify bit* hoặc *dirty bit*). Bit này được phần cứng đặt khi nội dung trang bị thay đổi bởi thao tác ghi. Nếu trang nạn nhân đã được sửa đổi, hệ điều hành phải bảo toàn những thay đổi chưa được lưu trước khi tái sử dụng khung. Ngược lại, nếu trang chưa bị sửa đổi và đã có bản sao hợp lệ trên thiết bị lưu trữ, hệ thống có thể loại bỏ bản trong bộ nhớ mà không cần ghi lại. Các trang chỉ đọc, chẳng hạn trang chứa mã chương trình, cũng có thể được loại bỏ theo cách này. Việc xét bit sửa đổi giúp giảm chi phí xử lý lỗi trang, cho thấy hiệu năng phụ thuộc cả vào số lỗi trang lẫn lượng dữ liệu phải ghi ra ngoài.

Cơ chế thay thế hoàn thiện khả năng tách biệt giữa không gian địa chỉ ảo và bộ nhớ vật lý. Một tiến trình gồm 20 trang vẫn có thể thực thi với 10 khung bằng cách luân phiên nạp những trang cần thiết; trang đã bị loại bỏ sẽ được đưa trở lại khi có tham

chiếu tiếp theo. Tuy nhiên, để cơ chế hoạt động hiệu quả, hệ điều hành phải giải quyết hai bài toán liên quan: phân bổ bao nhiêu khung cho mỗi tiến trình và lựa chọn trang nào để thay thế khi cần khung. Chất lượng của thuật toán thay thế thường được đánh giá bằng số lỗi trang phát sinh trên một **chuỗi tham chiếu trang** với số khung và trạng thái ban đầu xác định. Cách đánh giá này cho phép so sánh các chính sách trong cùng điều kiện, từ đó xác định mức độ phù hợp của quyết định lựa chọn trang nạn nhân đối với hành vi truy cập của tiến trình.

Ví dụ minh hoạ:

Giả sử một tiến trình được cấp ba khung trang, hiện chứa các trang **A, B và C**. Khi tiến trình yêu cầu truy cập trang **D** chưa có trong bộ nhớ, lỗi trang xảy ra. Do không còn khung trống, hệ điều hành phải lựa chọn một trang để thay thế. Giả sử thuật toán chọn **B** làm trang nạn nhân và B đã bị sửa đổi, hệ thống phải ghi nội dung B ra thiết bị lưu trữ trước khi nạp D vào khung của B. Sau đó, bảng trang được cập nhật: B được đánh dấu không còn hiện diện trong bộ nhớ, còn D được ánh xạ đến khung vừa tiếp nhận nó. Tiến trình tiếp tục thực hiện lệnh đã gây lỗi trang. Nếu B chưa bị sửa đổi và đã có bản sao hợp lệ trên thiết bị lưu trữ, thao tác ghi B có thể được bỏ qua.

Khung trang	**Trước khi thay thế**	**Sau khi thay thế**
Khung 0	Trang A	Trang A
Khung 1	Trang B — nạn nhân	Trang D — mới nạp
Khung 2	Trang C	Trang C

Bảng minh họa: Trang D được nạp vào khung của trang B; các trang A và C tiếp tục được giữ trong bộ nhớ.

Ví dụ cho thấy một lần thay thế không làm tăng số khung được cấp cho tiến trình mà chỉ thay đổi trang được lưu trong khung. Việc chọn B ở đây là giả định để minh họa quy trình; tiêu chí lựa chọn cụ thể sẽ phụ thuộc vào thuật toán thay thế. Nếu tiến trình truy cập B trở lại, một lỗi trang mới sẽ xảy ra và hệ điều hành phải nạp lại B.

Sơ đồ:

10.4.2. FIFO Page Replacement

FIFO (*First-In, First-Out*) là thuật toán thay thế trang lựa chọn trang đã được đưa vào bộ nhớ sớm nhất làm trang nạn nhân. Mỗi trang được xét dựa trên thời điểm nạp vào bộ nhớ, không phụ thuộc vào số lần truy cập hoặc thời điểm sử dụng gần nhất. Khi cần giải phóng một khung, hệ điều hành loại bỏ trang có thời gian thường trú lâu nhất. Cách tiếp cận này tạo ra một quy tắc lựa chọn đơn giản, nhưng cũng đặt ra giới hạn: một trang tồn tại lâu trong bộ nhớ chưa chắc đã ít cần thiết đối với tiến trình.

Nguyên tắc hoạt động và cách tổ chức hàng đợi

FIFO có thể được triển khai bằng một hàng đợi chứa các trang đang thường trú, sắp xếp theo thứ tự nạp. Trang được đưa vào bộ nhớ sớm nhất nằm ở đầu hàng, còn trang mới nạp được thêm vào cuối hàng. Khi xảy ra lỗi trang, nếu còn khung trống, hệ điều hành nạp trang vào khung đó và bổ sung trang vào cuối hàng. Nếu các khung đã đầy, trang ở đầu hàng bị loại bỏ; trang được yêu cầu được nạp vào khung vừa giải phóng và đưa xuống cuối hàng. Nhờ cách tổ chức này, hệ thống không cần lưu thời điểm nạp cụ thể của từng trang mà vẫn xác định được trang đã thường trú lâu nhất.

Đặc điểm quan trọng nhất là **một lần truy cập trúng không làm thay đổi thứ tự hàng đợi**. Khi tiến trình truy cập một trang đã có trong bộ nhớ, FIFO không chuyển trang đó xuống cuối hàng hoặc làm mới thời điểm nạp. Vì vậy, một trang dù vừa được sử dụng vẫn có thể bị thay thế ngay sau đó nếu đang nằm ở đầu hàng. Ngoài ra, nếu một trang đã bị loại bỏ rồi được nạp lại, nó được xem như một lần nạp mới và được đưa vào cuối hàng; FIFO không tiếp tục tính tuổi thường trú từ lần nạp trước.

Phân tích ví dụ với ba khung trang

Xét chuỗi tham chiếu được sử dụng trong giáo trình:

7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2, 1, 2, 0, 1, 7, 0, 1

Giả sử tiến trình được cấp **ba khung trang, ban đầu đều trống**. Trong bảng dưới đây, hàng đợi được trình bày từ trái sang phải theo thứ tự **cũ nhất → mới nhất**. Vị trí trong hàng đợi biểu diễn thứ tự nạp, không phải số hiệu khung vật lý.

Lần tham chiếu	**Trang yêu cầu**	**Kết quả**	**Trang bị thay thế**	**Hàng đợi sau khi xử lý**
1	7	Lỗi trang	—	7
2	0	Lỗi trang	—	7, 0
3	1	Lỗi trang	—	7, 0, 1
4	2	Lỗi trang	7	0, 1, 2
5	0	Trúng trang	—	0, 1, 2
6	3	Lỗi trang	0	1, 2, 3
7	0	Lỗi trang	1	2, 3, 0
8	4	Lỗi trang	2	3, 0, 4
9	2	Lỗi trang	3	0, 4, 2
10	3	Lỗi trang	0	4, 2, 3
11	0	Lỗi trang	4	2, 3, 0
12	3	Trúng trang	—	2, 3, 0

13	2	Trúng trang	—	2, 3, 0
14	1	Lỗi trang	2	3, 0, 1
15	2	Lỗi trang	3	0, 1, 2
16	0	Trúng trang	—	0, 1, 2
17	1	Trúng trang	—	0, 1, 2
18	7	Lỗi trang	0	1, 2, 7
19	0	Lỗi trang	1	2, 7, 0
20	1	Lỗi trang	2	7, 0, 1

Ba tham chiếu đầu tiên đến 7, 0 và 1 đều gây lỗi trang nhưng chưa cần thay thế, vì bộ nhớ vẫn còn khung trống. Đến tham chiếu thứ 4, trang 2 chưa có trong bộ nhớ và cả ba khung đã được sử dụng, nên FIFO loại bỏ trang 7 ở đầu hàng. Sau thao tác này, trang 0 trở thành trang được nạp sớm nhất trong số các trang còn thường trú.

Diễn biến ở các tham chiếu thứ 5, 6 và 7 thể hiện rõ hạn chế của FIFO. Tại lần thứ 5, trang 0 được truy cập thành công, nhưng hàng đợi vẫn giữ nguyên thứ tự 0, 1, 2. Khi trang 3 được yêu cầu ở lần thứ 6, FIFO loại bỏ trang 0 dù trang này vừa được sử dụng. Ngay ở lần thứ 7, tiến trình lại cần trang 0, khiến hệ thống phải nạp lại nó và loại bỏ trang 1. Như vậy, thuật toán không khai thác thông tin vừa xuất hiện về nhu cầu sử dụng trang 0. Quyết định dựa hoàn toàn vào thứ tự nạp có thể làm phát sinh thêm hoạt động chuyển trang khi tiến trình tiếp tục sử dụng các trang đã thường trú từ sớm.

Kết thúc chuỗi, FIFO tạo ra **15 lỗi trang và 5 lần truy cập trúng**, tương ứng tỷ lệ lỗi trang (15/20=75%). Trong 15 lỗi trang, ba lỗi đầu sử dụng khung trống; 12 lỗi còn lại dẫn đến thay thế. Cần phân biệt các đại lượng này với số lần ghi ra thiết bị lưu trữ:

không phải mọi trang bị thay thế đều phải được ghi lại, vì thao tác ghi còn phụ thuộc vào trạng thái sửa đổi của trang. Đồng thời, tỷ lệ 75% chỉ phản ánh kết quả của chuỗi tham chiếu và số khung đang xét, không phải tỷ lệ lỗi cố định của FIFO.

Ưu điểm và hạn chế của thuật toán

Ưu điểm nổi bật của FIFO là dễ hiểu và có chi phí quản lý thấp. Hệ thống chỉ cần duy trì thứ tự nạp trang, không phải cập nhật lịch sử sử dụng sau mỗi lần tham chiếu. Với hàng đợi được tổ chức thích hợp, việc lấy trang ở đầu hàng và thêm trang mới vào cuối hàng có thể được thực hiện trong thời gian hằng số. Tuy nhiên, lợi ích này chỉ liên quan đến thao tác quản lý hàng đợi; thời gian xử lý lỗi trang vẫn chịu ảnh hưởng lớn của các hoạt động đọc và ghi trên thiết bị lưu trữ.

Giới hạn căn bản của FIFO nằm ở việc sử dụng **tuổi thường trú thay cho mức độ cần thiết**. Trong một số trường hợp, trang được nạp sớm chứa mã khởi tạo đã hoàn thành nhiệm vụ, nên loại bỏ nó là hợp lý. Nhưng trong trường hợp khác, trang được nạp sớm có thể chứa biến hoặc dữ liệu được truy cập liên tục. FIFO xử lý hai tình huống này theo cùng một quy tắc, vì không xem xét lịch sử sử dụng sau khi trang được nạp. Do đó, thời gian tồn tại lâu trong bộ nhớ không đủ để kết luận một trang là ứng viên thay thế tốt.

Một lựa chọn không phù hợp không làm sai kết quả thực thi nếu hệ điều hành vẫn bảo toàn dữ liệu và xử lý lỗi trang đúng cách. Trang bị loại bỏ sẽ được nạp lại khi cần thiết. Hệ quả chủ yếu là suy giảm hiệu năng: tiến trình phải chờ thêm, số thao tác vào/ra tăng và các khung có thể liên tục được sử dụng để nạp lại những trang vừa bị loại bỏ. Vì vậy, tính đơn giản trong triển khai của FIFO không đồng nghĩa với hiệu quả sử dụng bộ nhớ cao.

Nghịch lý Belady

Một tính chất đáng chú ý của FIFO là khả năng xuất hiện **nghịch lý Belady (*Belady's anomaly*)**: khi tăng số khung được cấp, số lỗi trang có thể tăng thay vì giảm. Thông thường, thêm khung được kỳ vọng giúp giữ lại nhiều trang hơn và giảm nhu cầu nạp

lại. Tuy nhiên, đối với FIFO, thay đổi số khung còn làm thay đổi diễn biến hàng đợi và thứ tự loại bỏ trang, nên kết quả không nhất thiết cải thiện.

Xét chuỗi tham chiếu:

1, 2, 3, 4, 1, 2, 5, 1, 2, 3, 4, 5

Với các khung ban đầu trống, FIFO tạo ra **9 lỗi trang khi có ba khung**, nhưng **10 lỗi trang khi có bốn khung**. Bảng dưới đây đối chiếu diễn biến của hai cấu hình; thứ tự trang vẫn được ghi từ cũ nhất đến mới nhất.

Lần	**Trang yêu cầu**	**Hàng đợi với 3 khung**	**Kết quả**	**Hàng đợi với 4 khung**	**Kết quả**
1	1	1	Lỗi	1	Lỗi
2	2	1, 2	Lỗi	1, 2	Lỗi
3	3	1, 2, 3	Lỗi	1, 2, 3	Lỗi
4	4	2, 3, 4	Lỗi	1, 2, 3, 4	Lỗi
5	1	3, 4, 1	Lỗi	1, 2, 3, 4	Trúng
6	2	4, 1, 2	Lỗi	1, 2, 3, 4	Trúng
7	5	1, 2, 5	Lỗi	2, 3, 4, 5	Lỗi
8	1	1, 2, 5	Trúng	3, 4, 5, 1	Lỗi
9	2	1, 2, 5	Trúng	4, 5, 1, 2	Lỗi
10	3	2, 5, 3	Lỗi	5, 1, 2, 3	Lỗi
11	4	5, 3, 4	Lỗi	1, 2, 3, 4	Lỗi
12	5	5, 3, 4	Trúng	2, 3, 4, 5	Lỗi

Ở cấu hình ba khung, các trang 1 và 2 bị loại bỏ trong giai đoạn đầu, sau đó được nạp lại ở lần thứ 5 và thứ 6. Việc nạp lại đưa chúng về cuối hàng, nên khi trang 5 xuất hiện ở lần thứ 7, trang 4 bị loại bỏ và bộ nhớ giữ lại 1, 2, 5. Nhờ vậy, hai tham chiếu tiếp theo đến 1 và 2 đều trúng trang. Trong cấu hình bốn khung, 1 và 2 vẫn có sẵn ở lần thứ 5 và thứ 6, nhưng các lần truy cập này không làm mới thứ tự FIFO. Khi trang 5 được yêu cầu, trang 1 bị loại bỏ; yêu cầu tiếp theo đến 1 lại làm trang 2 bị loại bỏ. Từ đó, cấu hình bốn khung liên tục gặp lỗi trang trong phần còn lại của chuỗi.

Nguyên nhân sâu hơn là FIFO không bảo đảm **tính bao hàm giữa các tập trang thường trú** khi số khung tăng. Sau tham chiếu thứ 7, cấu hình ba khung giữ tập ({1,2,5}), trong khi cấu hình bốn khung giữ tập ({2,3,4,5}). Mặc dù có nhiều khung hơn, cấu hình thứ hai không chứa trang 1 mà cấu hình thứ nhất đang giữ. Do đó, nhiều khung hơn không bảo đảm có sẵn mọi trang mà cấu hình ít khung hơn có thể đáp ứng. Nghịch lý Belady không khẳng định rằng tăng khung luôn làm FIFO kém hiệu quả; nó chứng minh rằng thuật toán này không bảo đảm số lỗi trang giảm hoặc giữ nguyên khi được cấp thêm khung.

Qua hai ví dụ, có thể thấy FIFO giải quyết bài toán lựa chọn nạn nhân bằng một tiêu chí rõ ràng và ít tốn công theo dõi, nhưng tiêu chí đó chưa phản ánh đầy đủ hành vi truy cập của tiến trình. Việc bỏ qua các lần sử dụng sau khi nạp khiến thuật toán có thể loại bỏ trang đang hoạt động tích cực, đồng thời dẫn đến những kết quả bất lợi như nghịch lý Belady. Đây là cơ sở để xem xét các thuật toán sử dụng thông tin về tương lai hoặc lịch sử truy cập nhằm đưa ra quyết định thay thế phù hợp hơn.

10.4.3. Optimal Page Replacement

Thuật toán thay thế trang tối ưu, thường được gọi là **OPT** hoặc **MIN**, lựa chọn thay thế trang có lần sử dụng tiếp theo xa nhất trong tương lai. Nếu một trang không còn được tham chiếu trong phần còn lại của quá trình thực thi, trang đó là ứng viên thích hợp để loại bỏ. Khác với FIFO dựa vào thời điểm nạp, OPT dựa vào nhu cầu sử dụng sắp tới để đưa ra quyết định. Với một chuỗi tham chiếu và số khung xác định, thuật toán bảo đảm số lỗi trang thấp nhất có thể đạt được trong mô hình thay thế trang đang xét, đồng thời không chịu nghịch lý Belady.

Nguyên tắc lựa chọn trang nạn nhân

Khi xảy ra lỗi trang và không còn khung trống, OPT xem xét lần xuất hiện tiếp theo của từng trang đang thường trú. Trang nào được sử dụng muộn nhất sẽ bị loại bỏ, nhờ đó các trang sắp cần đến được tiếp tục giữ trong bộ nhớ. Nếu có nhiều trang không bao giờ được sử dụng lại, có thể chọn bất kỳ trang nào trong số đó mà không ảnh hưởng đến số lỗi trang còn lại.

Cần phân biệt **thời điểm sử dụng tiếp theo** với **tổng số lần sử dụng trong tương lai**. OPT không lựa chọn trang có ít lần truy cập nhất, mà lựa chọn trang có lần truy cập kế tiếp nằm xa nhất. Một trang có thể được sử dụng nhiều lần về sau nhưng vẫn bị thay thế nếu trước lần sử dụng tiếp theo của nó, các trang còn lại đều cần được truy cập sớm hơn. Quyết định này giúp ưu tiên những nhu cầu gần, trong khi việc nạp lại trang bị loại bỏ được trì hoãn lâu nhất có thể.

Có thể biểu diễn nguyên tắc lựa chọn bằng cách gọi (N(p)) là vị trí tham chiếu tiếp theo của trang (p) trong phần chuỗi chưa xử lý. Nếu trang không còn xuất hiện, quy ước (N(p)=+\infty). Khi cần thay thế, OPT chọn trang nạn nhân (p^*) thỏa mãn:

[
p^*=\underset{p\in M}{\arg\max};N(p)
]

Trong đó, (M) là tập trang đang thường trú. Biểu thức cho thấy quyết định chỉ được thực hiện giữa những trang hiện có trong bộ nhớ và phụ thuộc trực tiếp vào thông tin về tương lai.

Phân tích ví dụ với ba khung trang

Xét chuỗi tham chiếu trong giáo trình:

7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2, 1, 2, 0, 1, 7, 0, 1

Giả sử tiến trình được cấp **ba khung trang, ban đầu đều trống**. Bảng dưới đây ghi nhận trạng thái của từng khung sau mỗi lần tham chiếu. Khi có nhiều ứng viên không còn được sử dụng, bảng lựa chọn một phương án hợp lệ của OPT.

Lần tham chiếu	**Trang yêu cầu**	**Khung 0**	**Khung 1**	**Khung 2**	**Kết quả**	**Trang bị thay thế**
1	7	7	—	—	Lỗi trang	—
2	0	7	0	—	Lỗi trang	—
3	1	7	0	1	Lỗi trang	—
4	2	2	0	1	Lỗi trang	7
5	0	2	0	1	Trúng trang	—
6	3	2	0	3	Lỗi trang	1
7	0	2	0	3	Trúng trang	—
8	4	2	4	3	Lỗi trang	0
9	2	2	4	3	Trúng trang	—
10	3	2	4	3	Trúng trang	—
11	0	2	0	3	Lỗi trang	4
12	3	2	0	3	Trúng trang	—

13	2	2	0	3	Trúng trang	—
14	1	2	0	1	Lỗi trang	3
15	2	2	0	1	Trúng trang	—
16	0	2	0	1	Trúng trang	—
17	1	2	0	1	Trúng trang	—
18	7	7	0	1	Lỗi trang	2
19	0	7	0	1	Trúng trang	—
20	1	7	0	1	Trúng trang	—

Ba tham chiếu đầu tiên lần lượt nạp 7, 0 và 1 vào các khung trống. Đến tham chiếu thứ 4, trang 2 được yêu cầu trong khi bộ nhớ đã đầy. OPT so sánh lần sử dụng tiếp theo của ba trang hiện có: trang 0 sẽ được dùng ở vị trí 5, trang 1 ở vị trí 14 và trang 7 ở vị trí 18. Vì trang 7 có lần sử dụng tiếp theo xa nhất, nó bị thay thế bởi trang 2. Quyết định này giữ lại trang 0 để đáp ứng ngay tham chiếu kế tiếp, đồng thời tiếp tục lưu trang 1 vì nó sẽ được cần đến trước trang 7.

Tại tham chiếu thứ 6, bộ nhớ chứa 2, 0 và 1, còn trang được yêu cầu là 3. Lần sử dụng tiếp theo của 0 nằm ở vị trí 7, của 2 ở vị trí 9 và của 1 ở vị trí 14. OPT vì thế loại bỏ trang 1. Đến vị trí 8, trang 4 được yêu cầu và bộ nhớ đang chứa 2, 0, 3. Thuật toán giữ lại 2 và 3 vì chúng sẽ được sử dụng ở hai vị trí kế tiếp, đồng thời loại bỏ 0 vì trang này đến vị trí 11 mới được tham chiếu lại. Diễn biến này thể hiện rõ cách OPT ưu tiên

những trang cần thiết sớm hơn, thay vì giữ trang theo tuổi thường trú hoặc theo mức độ sử dụng trong quá khứ.

Ở vị trí 11, trang 0 được nạp lại và thay thế trang 4 vì 4 không còn xuất hiện trong phần chuỗi còn lại. Tương tự, trang 3 bị loại bỏ ở vị trí 14 và trang 2 bị loại bỏ ở vị trí 18, bởi tại thời điểm lựa chọn, các trang này đều không còn được sử dụng. Kết thúc chuỗi, OPT tạo ra **9 lỗi trang và 11 lần truy cập trúng**, tương ứng tỷ lệ lỗi trang (9/20=45%). Trong đó, ba lỗi đầu dùng khung trống và sáu lỗi còn lại dẫn đến thay thế.

Ý nghĩa của kết quả tối ưu

Với cùng chuỗi tham chiếu và ba khung, FIFO tạo ra 15 lỗi trang, trong khi OPT chỉ tạo ra 9 lỗi. Như vậy, OPT giảm được 6 lỗi, tương đương **40% tổng số lỗi trang so với FIFO** trong ví dụ này. Nếu chỉ xét những lỗi dẫn đến thay thế sau khi ba khung ban đầu đã được lấp đầy, FIFO có 12 lỗi còn OPT có 6 lỗi, tức giảm một nửa. Việc phân biệt hai cách tính giúp tránh diễn giải rằng OPT giảm một nửa tổng số lỗi trang hoặc luôn làm thời gian thực thi giảm một nửa.

Quan trọng hơn, 9 lỗi không chỉ là kết quả tốt hơn FIFO mà còn là **giới hạn thấp nhất đối với chuỗi và số khung đang xét**. Cơ sở của tính tối ưu nằm ở việc loại bỏ trang có nhu cầu sử dụng xa nhất. Giả sử một phương án khác loại bỏ trang sẽ được dùng sớm hơn và giữ lại trang được dùng muộn hơn. Có thể điều chỉnh phương án đó theo lựa chọn của OPT mà không làm tăng số lỗi trang: trước khi trang được dùng muộn hơn trở nên cần thiết, việc giữ lại trang được dùng sớm hơn có thể tránh được một yêu cầu nạp lại. Lập luận trao đổi này có thể được áp dụng qua các lần thay thế để chứng minh rằng lựa chọn của OPT đạt số lỗi trang tối thiểu. Đây là cách giải thích cơ sở lý thuyết của thuật toán, thay vì chỉ suy ra tính tối ưu từ một ví dụ cụ thể.

OPT cũng không chịu nghịch lý Belady. Khi có thêm khung, hệ thống có khả năng giữ lại nhiều trang hơn; xét trên mọi phương án thay thế, số lỗi tối thiểu không thể tăng vì cấu hình lớn hơn luôn có thể đáp ứng ít nhất những gì cấu hình nhỏ hơn thực hiện được. Do OPT đạt giới hạn tối thiểu ở mỗi mức số khung, việc tăng số khung không

thể khiến số lỗi của nó tăng. Tính chất này tạo ra sự khác biệt quan trọng với FIFO, vốn có thể thay đổi thứ tự loại bỏ theo hướng bất lợi khi được cấp thêm bộ nhớ.

Giới hạn triển khai và vai trò làm chuẩn đánh giá

Khó khăn căn bản của OPT là yêu cầu biết trước các tham chiếu trong tương lai. Trong quá trình thực thi thông thường, hệ điều hành không có đầy đủ thông tin này vì hành vi truy cập phụ thuộc vào dữ liệu đầu vào, các nhánh xử lý và những sự kiện chưa xảy ra. Việc biết địa chỉ của các trang hoặc lịch sử truy cập trước đó không đủ để xác định chắc chắn trang nào sẽ được sử dụng xa nhất về sau. Vì vậy, OPT chủ yếu là một thuật toán lý tưởng dùng trong nghiên cứu và mô phỏng, thay vì chính sách có thể áp dụng trực tiếp cho mọi tiến trình đang chạy.

Khi đã ghi lại được một chuỗi tham chiếu, thông tin tương lai trở nên sẵn có đối với quá trình mô phỏng. Lúc này, OPT được dùng để xác định số lỗi trang tối thiểu, từ đó đánh giá khoảng cách giữa một thuật toán khả thi và kết quả lý tưởng. Chẳng hạn, nếu một thuật toán tạo ra nhiều lỗi hơn OPT trên cùng chuỗi, khoảng chênh lệch cho thấy mức tổn thất về số lỗi do các quyết định thay thế của thuật toán đó. Tuy nhiên, việc so sánh phải giữ thống nhất số khung và trạng thái ban đầu; kết quả trên một chuỗi cũng chưa đủ để khẳng định chất lượng đối với mọi loại chương trình.

Cuối cùng, tính tối ưu của OPT cần được hiểu đúng phạm vi: thuật toán tối thiểu hóa **số lỗi trang**, không trực tiếp tối thiểu hóa mọi chi phí của hệ thống. Thời gian thực thi còn chịu ảnh hưởng của trạng thái trang bẩn, hoạt động vào/ra và chi phí quản lý. Dù có giới hạn triển khai, OPT vẫn giữ vai trò quan trọng vì xác lập một chuẩn rõ ràng cho bài toán thay thế trang. Từ chuẩn này, các thuật toán thực tế tìm cách sử dụng những thông tin có thể thu thập được, đặc biệt là lịch sử truy cập, để tiến gần hơn đến quyết định lý tưởng.

10.4.4. LRU Page Replacement

LRU (*Least Recently Used*) là thuật toán thay thế trang lựa chọn trang đã lâu không được sử dụng nhất làm trang nạn nhân. Thuật toán xuất phát từ việc tìm kiếm một phương án gần với Optimal nhưng không cần biết trước chuỗi tham chiếu tương lai.

Nếu Optimal loại bỏ trang có lần sử dụng tiếp theo xa nhất, LRU loại bỏ trang có lần sử dụng gần nhất xa nhất trong quá khứ. Cơ sở của cách lựa chọn này là tính cục bộ theo thời gian: những trang vừa được truy cập thường có khả năng tiếp tục được sử dụng trong tương lai gần. Vì vậy, lịch sử truy cập gần đây được sử dụng để dự đoán nhu cầu sắp tới của tiến trình.

Nguyên tắc lựa chọn trang nạn nhân

LRU theo dõi lần sử dụng gần nhất của các trang đang thường trú. Khi xảy ra lỗi trang và không còn khung trống, hệ thống so sánh thông tin này để chọn trang lâu không được tham chiếu nhất. Khác với FIFO, tiêu chí của LRU không phải thời điểm trang được đưa vào bộ nhớ. Một trang được nạp từ sớm nhưng liên tục được truy cập vẫn có thể được giữ lại, trong khi một trang được nạp muộn hơn nhưng đã lâu không được dùng có thể bị thay thế. Đồng thời, LRU xét **tính gần đây**, không xét tổng số lần sử dụng: một trang từng được truy cập nhiều vẫn có thể trở thành nạn nhân nếu lần truy cập cuối cùng của nó xa hơn các trang còn lại.

Đặc điểm quan trọng là **mọi lần tham chiếu, kể cả truy cập trúng, đều phải được phản ánh vào lịch sử sử dụng**. Khi một trang đang có trong bộ nhớ được truy cập, nó trở thành trang được sử dụng gần nhất và thứ tự ưu tiên thay thế phải thay đổi tương ứng. Đây là điểm giúp LRU thích ứng với hoạt động của tiến trình, nhưng cũng là nguyên nhân làm chi phí triển khai cao hơn FIFO. Thuật toán cần duy trì thông tin liên tục trong quá trình thực thi, thay vì chỉ điều chỉnh thứ tự khi nạp hoặc loại bỏ trang.

Phân tích ví dụ với ba khung trang

Xét chuỗi tham chiếu được sử dụng trong giáo trình:

7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2, 1, 2, 0, 1, 7, 0, 1

Giả sử tiến trình có **ba khung trang, ban đầu đều trống**. Trong bảng dưới đây, thứ tự sử dụng được trình bày từ trái sang phải theo chiều **lâu không được dùng nhất → vừa được dùng gần nhất**. Thứ tự này thể hiện lịch sử truy cập, không phải vị trí của các trang trong bộ nhớ vật lý.

Lần tham chiếu	**Trang yêu cầu**	**Kết quả**	**Trang bị thay thế**	**Thứ tự sử dụng sau khi xử lý**
1	7	Lỗi trang	—	7
2	0	Lỗi trang	—	7, 0
3	1	Lỗi trang	—	7, 0, 1
4	2	Lỗi trang	7	0, 1, 2
5	0	Trúng trang	—	1, 2, 0
6	3	Lỗi trang	1	2, 0, 3
7	0	Trúng trang	—	2, 3, 0
8	4	Lỗi trang	2	3, 0, 4
9	2	Lỗi trang	3	0, 4, 2
10	3	Lỗi trang	0	4, 2, 3
11	0	Lỗi trang	4	2, 3, 0
12	3	Trúng trang	—	2, 0, 3
13	2	Trúng trang	—	0, 3, 2
14	1	Lỗi trang	0	3, 2, 1
15	2	Trúng trang	—	3, 1, 2

16	0	Lỗi trang	3	1, 2, 0
17	1	Trúng trang	—	2, 0, 1
18	7	Lỗi trang	2	0, 1, 7
19	0	Trúng trang	—	1, 7, 0
20	1	Trúng trang	—	7, 0, 1

Ba tham chiếu đầu tiên nạp 7, 0 và 1 vào các khung trống. Khi trang 2 được yêu cầu ở vị trí thứ 4, LRU loại bỏ trang 7 vì đây là trang có lần sử dụng cuối cùng xa nhất. Đến tham chiếu thứ 5, trang 0 được truy cập thành công nên trở thành trang vừa được sử dụng gần nhất. Vì vậy, khi cần nạp trang 3 ở vị trí thứ 6, LRU loại bỏ trang 1, có lần sử dụng cuối ở vị trí thứ 3, thay vì loại bỏ trang 0 như FIFO. Lần truy cập tiếp theo đến trang 0 nhờ đó là một lần trúng trang. Diễn biến này cho thấy việc cập nhật lịch sử trên các lần truy cập trúng có thể giúp tránh loại bỏ trang đang được sử dụng.

Tuy nhiên, lịch sử gần không phải lúc nào cũng dự đoán đúng tương lai. Tại tham chiếu thứ 8, bộ nhớ chứa 2, 0 và 3, có lần sử dụng gần nhất lần lượt ở vị trí 4, 7 và 6. Khi trang 4 được yêu cầu, LRU loại bỏ trang 2 vì nó lâu không được dùng nhất. Nhưng tham chiếu thứ 9 lại chính là trang 2, khiến hệ thống phải nạp lại ngay trang vừa loại bỏ. Trong cùng tình huống, Optimal giữ trang 2 vì biết trước nó sẽ được sử dụng ở bước kế tiếp và lựa chọn trang 0 làm nạn nhân. Sự khác biệt này phản ánh giới hạn căn bản của LRU: quyết định có thể hợp lý theo quá khứ nhưng vẫn không phù hợp với những gì xảy ra tiếp theo.

Kết thúc chuỗi, LRU tạo ra **12 lỗi trang và 8 lần truy cập trúng**, tương ứng tỷ lệ lỗi trang (12/20=60%). Trong 12 lỗi, ba lỗi đầu sử dụng khung trống và chín lỗi còn lại dẫn đến thay thế. So với FIFO có 15 lỗi, LRU giảm được ba lỗi, tương đương 20% trong ví dụ này; tuy nhiên, kết quả vẫn cao hơn Optimal với chín lỗi. Các số liệu cho

thấy lợi ích của việc khai thác lịch sử truy cập trên chuỗi đang xét, nhưng không chứng minh rằng LRU luôn tốt hơn FIFO đối với mọi chương trình.

Triển khai bằng bộ đếm thời gian

Một phương pháp triển khai LRU là gắn mỗi mục bảng trang với một trường ghi nhận thời điểm sử dụng gần nhất. Hệ thống duy trì một đồng hồ logic hoặc bộ đếm tăng theo các lần tham chiếu bộ nhớ. Khi một trang được truy cập, giá trị hiện tại của bộ đếm được ghi vào trường thời gian tương ứng. Khi cần thay thế, trang có giá trị nhỏ nhất được chọn vì có lần sử dụng cuối cùng xa nhất.

Cách tổ chức này biểu diễn trực tiếp nguyên tắc LRU, nhưng có hai nguồn chi phí đáng chú ý. Trước hết, thông tin thời gian phải được cập nhật sau mỗi lần tham chiếu, tạo thêm công việc trong quá trình truy cập bộ nhớ. Tiếp đó, để xác định nạn nhân, hệ thống phải tìm giá trị nhỏ nhất trong các trang thuộc phạm vi lựa chọn nếu không có cấu trúc hỗ trợ khác. Việc triển khai còn phải xử lý tình trạng bộ đếm tràn và duy trì thông tin thời gian phù hợp khi hệ thống chuyển đổi giữa các tiến trình. Vì vậy, một quy tắc lựa chọn đơn giản về mặt khái niệm vẫn có thể đòi hỏi cơ chế quản lý phức tạp.

Triển khai bằng cấu trúc thứ tự sử dụng

Phương pháp thứ hai là duy trì một cấu trúc ghi lại thứ tự sử dụng của các trang. Khi một trang được tham chiếu, nó được lấy ra khỏi vị trí hiện tại và chuyển lên đầu; trang vừa được dùng luôn nằm ở đầu, còn trang lâu không được dùng nhất nằm ở cuối. Khi cần thay thế, hệ thống chỉ cần chọn trang cuối cùng mà không phải tìm kiếm theo thời gian. Lưu ý rằng cách trình bày này đặt trang mới dùng ở đầu, ngược chiều với bảng minh họa phía trên, nhưng nguyên tắc lựa chọn hoàn toàn giống nhau.

Giáo trình mô tả phương pháp này theo ý tưởng ngăn xếp, nhưng do cần lấy phần tử ra khỏi cả vị trí giữa, cấu trúc thích hợp để triển khai là **danh sách liên kết đôi có con trỏ đầu và cuối**. Nếu có thể xác định trực tiếp phần tử tương ứng với trang vừa được tham chiếu, việc chuyển nó lên đầu chỉ cần điều chỉnh một số liên kết. Theo cách tổ chức được trình bày trong sách, thao tác này có thể cần thay đổi tối đa sáu con trỏ. Đổi

lại, xác định trang nạn nhân trở nên trực tiếp vì con trỏ cuối luôn chỉ đến trang ít được sử dụng gần đây nhất.

Dù tránh được việc quét tìm nạn nhân, phương pháp danh sách vẫn phải cập nhật sau mỗi lần tham chiếu bộ nhớ. Đây là điểm chung quan trọng của cả hai cách triển khai: chi phí không chỉ phát sinh khi xảy ra lỗi trang mà hiện diện xuyên suốt quá trình thực thi. Với tần suất truy cập bộ nhớ rất lớn, ngay cả một lượng công việc bổ sung nhỏ trên mỗi tham chiếu cũng có thể tạo ra tổng chi phí đáng kể.

Tính chất ngăn xếp và khả năng tránh nghịch lý Belady

LRU thuộc nhóm **thuật toán ngăn xếp (*stack algorithms*)**, có tính chất bao hàm giữa các tập trang thường trú khi số khung tăng. Với cùng chuỗi tham chiếu và trạng thái ban đầu trống, các trang được giữ bởi LRU khi có (n) khung là (n) trang phân biệt được tham chiếu gần đây nhất, hoặc toàn bộ các trang đã xuất hiện nếu số lượng ít hơn (n). Khi tăng lên (n+1) khung, những trang đó vẫn được giữ lại, đồng thời có thể lưu thêm một trang. Gọi (S_n(t)) là tập trang thường trú sau tham chiếu thứ (t) với (n) khung, tính chất này được biểu diễn bằng:

[
S_n(t)\subseteq S_{n+1}(t)
]

Từ quan hệ bao hàm, nếu một tham chiếu trúng trang ở cấu hình ít khung hơn, nó cũng trúng ở cấu hình nhiều khung hơn. Do đó, thêm khung không thể làm tăng số lỗi trang của LRU và thuật toán không chịu nghịch lý Belady. Cần phân biệt tính chất ngăn xếp với việc sử dụng một cấu trúc dữ liệu dạng ngăn xếp để triển khai: tính chất này mô tả quan hệ giữa kết quả của thuật toán ở các mức số khung, nên vẫn đúng khi LRU được triển khai bằng bộ đếm thời gian.

Đánh giá hiệu quả và giới hạn thực tiễn

LRU có ưu điểm là sử dụng được thông tin quá khứ để thích ứng với nhu cầu truy cập, đồng thời bảo vệ những trang vừa được sử dụng. Tuy nhiên, hiệu quả của nó phụ

thuộc vào mức độ phù hợp của tính cục bộ theo thời gian với chương trình đang chạy. Chẳng hạn, với ba khung và chuỗi lặp **1, 2, 3, 4, 1, 2, 3, 4, …**, mỗi trang đã bị loại bỏ trước khi được sử dụng lại. Sau khi ba khung đầu tiên được lấp đầy, mọi tham chiếu tiếp theo đều gây lỗi trang. Ví dụ bổ sung này cho thấy LRU có thể hoạt động kém khi tiến trình quét lặp qua một tập trang lớn hơn khả năng chứa của bộ nhớ được cấp.

Khó khăn lớn trong thực tiễn là duy trì LRU chính xác với chi phí chấp nhận được. Cả bộ đếm lẫn danh sách thứ tự đều cần cập nhật thông tin trên mọi tham chiếu, nên đòi hỏi sự hỗ trợ phần cứng đáng kể. Nếu phần mềm phải nhận ngắt sau từng lần truy cập để thực hiện cập nhật, chi phí xử lý sẽ làm chậm đáng kể hoạt động của hệ thống. Vì vậy, giá trị của LRU không chỉ nằm ở kết quả thay thế mà còn phải được cân nhắc cùng chi phí thu thập thông tin. Sự đánh đổi này dẫn đến các thuật toán xấp xỉ LRU: sử dụng thông tin truy cập ít chi tiết hơn để duy trì phần nào lợi ích của LRU với chi phí triển khai thấp hơn.

10.4.5. LRU-Approximation Page Replacement

LRU lựa chọn trang nạn nhân dựa trên thời điểm sử dụng gần nhất, nhưng việc duy trì chính xác thứ tự truy cập của các trang đòi hỏi chi phí đáng kể. Dù sử dụng bộ đếm thời gian hay danh sách thứ tự, hệ thống vẫn phải cập nhật thông tin sau mỗi lần tham chiếu bộ nhớ. Nhiều hệ thống không có đủ hỗ trợ phần cứng để thực hiện công việc này một cách hiệu quả. Vì vậy, các thuật toán **xấp xỉ LRU** được xây dựng nhằm khai thác thông tin về lịch sử sử dụng ở mức đơn giản hơn, qua đó giảm chi phí quản lý mà vẫn có cơ sở để bảo vệ những trang vừa được truy cập.

Thông tin nền tảng của nhóm thuật toán này là **bit tham chiếu (*reference bit*)**, ký hiệu (R), gắn với mỗi mục trong bảng trang. Phần cứng đặt (R=1) khi có thao tác đọc hoặc ghi vào trang; hệ điều hành có thể xóa bit về 0 để bắt đầu một khoảng theo dõi mới. Khi kiểm tra, (R=0) cho biết trang chưa được tham chiếu kể từ lần xóa gần nhất, còn (R=1) cho biết trang đã được tham chiếu ít nhất một lần. Tuy nhiên, một bit không thể biểu diễn chính xác thời điểm, số lần hay thứ tự truy cập. Chẳng hạn, hai trang cùng có (R=1) vẫn có thể được sử dụng ở những thời điểm rất khác nhau. Do đó, các quyết

định dựa trên bit tham chiếu chỉ xấp xỉ nguyên tắc LRU, thay vì tái hiện đầy đủ thứ tự sử dụng.

10.4.5.1. Thuật toán các bit tham chiếu bổ sung (Additional-Reference-Bits Algorithm)

Thuật toán các bit tham chiếu bổ sung mở rộng lượng thông tin lịch sử bằng cách duy trì một thanh ghi cho mỗi trang, chẳng hạn thanh ghi 8 bit. Theo những khoảng thời gian đều đặn, hệ điều hành dịch nội dung thanh ghi sang phải một vị trí, đưa bit tham chiếu hiện tại vào vị trí cao nhất và loại bỏ bit thấp nhất. Sau đó, bit tham chiếu được xóa để thu thập thông tin cho khoảng tiếp theo. Với thanh ghi 8 bit, mỗi trang lưu được lịch sử sử dụng trong tám khoảng quan sát gần nhất; bit cao nhất phản ánh khoảng mới nhất, còn các bit thấp hơn phản ánh những khoảng cũ hơn.

Có thể biểu diễn thao tác cập nhật bằng:

[
H_{t+1}=(H_t\gg 1);|;(R_t\ll 7)
]

Trong đó, (H_t) là giá trị lịch sử trước khi cập nhật, (R_t) là bit tham chiếu của khoảng vừa kết thúc; (\gg), (\ll) và (|) lần lượt là phép dịch phải, dịch trái và OR theo bit. Ví dụ, nếu thanh ghi hiện tại là 01010000 và trang được tham chiếu trong khoảng vừa qua, thao tác cập nhật tạo ra 10101000: lịch sử cũ dịch sang phải và giá trị 1 được đưa vào đầu. Nếu trang không được tham chiếu trong khoảng tiếp theo, thanh ghi trở thành 01010100. Nhờ quá trình dịch, ảnh hưởng của các lần sử dụng cũ giảm dần và cuối cùng bị loại khỏi lịch sử.

Khi cần thay thế, các thanh ghi được xem như những số nguyên không dấu và trang có giá trị nhỏ nhất được chọn làm ứng viên. Giá trị 00000000 biểu thị trang không được tham chiếu trong tám khoảng vừa qua; giá trị 11111111 biểu thị trang được tham chiếu ít nhất một lần trong mỗi khoảng. So sánh hai trang có lịch sử 11000100 và 01110111, trang thứ nhất được ưu tiên giữ lại vì đã được sử dụng trong khoảng quan sát mới nhất. Mặc dù trang thứ hai có nhiều bit 1 hơn, những dấu hiệu sử dụng đó thuộc các

khoảng cũ hơn. Vì thế, thuật toán đặt trọng số lớn hơn cho hoạt động gần đây, thay vì chỉ đếm số khoảng có truy cập.

Giới hạn của phương pháp nằm ở độ phân giải của lịch sử. Trong cùng một khoảng quan sát, trang được sử dụng một lần và trang được sử dụng nhiều lần đều chỉ nhận một bit 1; thứ tự truy cập bên trong khoảng cũng không được ghi nhận. Hai trang vì vậy có thể có cùng giá trị lịch sử dù lần sử dụng cuối khác nhau. Khi cần chọn một nạn nhân trong các trang bằng điểm, hệ thống có thể áp dụng quy tắc phụ như FIFO. Số bit và độ dài khoảng quan sát cũng tạo ra sự đánh đổi: lịch sử dài hơn lưu được nhiều thông tin hơn, còn cập nhật thường xuyên hơn phản ánh biến động nhanh hơn nhưng làm tăng công việc quản lý. Do đó, trang có giá trị nhỏ nhất là ứng viên theo lịch sử đã lấy mẫu, không nhất thiết là trang LRU chính xác.

10.4.5.2. Thuật toán cơ hội thứ hai (Second-Chance Algorithm)

Thuật toán **cơ hội thứ hai** cải tiến FIFO bằng cách kiểm tra bit tham chiếu trước khi quyết định loại bỏ trang ở đầu hàng. Nếu trang có (R=0), nó được chọn để thay thế. Nếu có (R=1), thuật toán xóa bit về 0, chuyển trang về cuối hàng và tiếp tục kiểm tra trang kế tiếp. Như vậy, một trang đã được truy cập trong khoảng theo dõi được trao thêm cơ hội thường trú, thay vì bị loại bỏ chỉ vì được nạp từ sớm. Việc chuyển trang xuống cuối hàng chỉ thay đổi thứ tự xét thay thế, không yêu cầu di chuyển nội dung trang giữa các khung vật lý.

Cơ hội được trao không có nghĩa rằng trang được bảo vệ lâu dài. Sau khi (R) bị xóa, nếu trang không được truy cập lại trước lần kiểm tra tiếp theo, nó có thể bị loại bỏ. Ngược lại, nếu trang được sử dụng và phần cứng đặt lại (R=1), nó có thể tiếp tục nhận thêm cơ hội. Thuật toán vì thế sử dụng hoạt động truy cập để điều chỉnh thứ tự FIFO, nhưng vẫn không duy trì đầy đủ thứ tự sử dụng như LRU. Các trang có cùng bit tham chiếu không được phân biệt theo thời điểm truy cập chính xác.

Cách triển khai phổ biến là **thuật toán đồng hồ (*Clock*)**. Các khung được tổ chức thành một vòng tròn, còn một con trỏ đóng vai trò kim đồng hồ, chỉ vào khung sẽ được xét tiếp theo. Khi cần nạn nhân, kim kiểm tra bit tham chiếu của trang tại vị trí

hiện tại. Nếu (R=1), bit được xóa và kim chuyển sang khung kế tiếp; nếu (R=0), trang được thay thế ngay tại khung đó. Sau khi hoàn tất, vị trí xét được chuyển tiếp. Cách tổ chức này thực hiện ý tưởng cơ hội thứ hai mà không cần liên tục lấy phần tử ra và đưa lại vào cuối một hàng đợi.

Giả sử thứ tự vòng là A, B, C, D và kim đang chỉ vào A:

Trang theo chiều quét	**Bit tham chiếu ban đầu**	**Xử lý**
A	1	Xóa về 0, giữ lại và xét tiếp
B	1	Xóa về 0, giữ lại và xét tiếp
C	0	Chọn làm nạn nhân
D	1	Chưa cần xét trong lần thay thế này

Trong ví dụ, A và B được giữ lại vì có dấu hiệu đã được sử dụng, còn C bị thay thế. Nếu trang E được nạp vào khung của C để phục vụ tham chiếu gây lỗi, khi tham chiếu đó được thực hiện, bit tham chiếu của E được đặt thành 1. Kim tiếp tục từ D cho lần tìm kiếm sau. Kết quả cho thấy cơ hội thứ hai vẫn sử dụng thứ tự quét để lựa chọn, nhưng có bổ sung thông tin sử dụng nhằm tránh loại bỏ ngay những trang vừa được tham chiếu.

Nếu mọi trang đều có (R=1), kim phải đi qua toàn bộ vòng để xóa các bit. Trong điều kiện không có tham chiếu mới làm thay đổi chúng khi quét, trang đầu tiên được xét lại sẽ có (R=0) và bị thay thế. Khi đó, lựa chọn nạn nhân tương ứng với FIFO, dù phải thực hiện thêm một lượt quét. Điều này cho thấy Clock không bảo đảm tìm được nạn nhân chỉ bằng một lần kiểm tra; trong trường hợp bất lợi, nó phải xét toàn bộ các khung trước khi lựa chọn. Tuy nhiên, so với LRU chính xác, thuật toán giảm đáng kể yêu cầu theo dõi vì chỉ cần bit tham chiếu và vị trí kim, thay vì cập nhật thứ tự đầy đủ sau mỗi truy cập.

10.4.5.3. Thuật toán cơ hội thứ hai nâng cao (Enhanced Second-Chance Algorithm)

Thuật toán **cơ hội thứ hai nâng cao** xét thêm chi phí bảo toàn dữ liệu khi thay thế. Bên cạnh bit tham chiếu (R), nó sử dụng **bit sửa đổi (M)** để phân biệt trang sạch và trang bẩn. Bit (R) cung cấp dấu hiệu về nhu cầu sử dụng gần đây, còn bit (M) cho biết nội dung trang đã bị thay đổi hay chưa. Hai bit tạo thành bốn lớp với thứ tự ưu tiên thay thế như sau:

Lớp ((R,M))	**Ý nghĩa**	**Thứ tự ưu tiên thay thế**
((0,0))	Chưa được tham chiếu trong khoảng theo dõi, chưa sửa đổi	1
((0,1))	Chưa được tham chiếu trong khoảng theo dõi, đã sửa đổi	2
((1,0))	Đã được tham chiếu trong khoảng theo dõi, chưa sửa đổi	3
((1,1))	Đã được tham chiếu trong khoảng theo dõi, đã sửa đổi	4

Trang thuộc lớp ((0,0)) là ứng viên tốt nhất vì không có dấu hiệu được sử dụng gần đây và có thể được loại bỏ mà không phải ghi lại nếu đã có bản sao hợp lệ trên thiết bị lưu trữ. Trang ((0,1)) cũng chưa được tham chiếu trong khoảng theo dõi, nhưng việc loại bỏ đòi hỏi bảo toàn nội dung đã sửa đổi. Trang ((1,0)) được giữ lại lâu hơn vì có dấu hiệu đang được sử dụng, dù chi phí loại bỏ hiện tại thấp hơn trang bẩn. Cuối cùng, ((1,1)) được ưu tiên bảo vệ nhiều nhất do vừa được tham chiếu, vừa có thể phát sinh thao tác ghi khi bị thay thế.

Thứ tự này cho thấy thuật toán không đơn thuần chọn trang có chi phí ghi thấp nhất. Cụ thể, ((0,1)) được xét thay thế trước ((1,0)), nghĩa là dấu hiệu sử dụng gần đây vẫn

giữ vai trò ưu tiên. Trong những trang có cùng trạng thái tham chiếu, trang sạch được chọn trước trang bẩn để giảm thao tác vào/ra. Vì vậy, quyết định thay thế kết hợp hai mục tiêu: hạn chế loại bỏ trang có khả năng cần dùng lại sớm và tránh ghi dữ liệu không cần thiết.

Giả sử tại thời điểm phân loại, bốn trang A, B, C, D lần lượt thuộc các lớp ((1,0)), ((0,1)), ((0,0)), ((1,1)). Trang C được ưu tiên thay thế vì thuộc lớp tốt nhất. Nếu không có trang nào thuộc ((0,0)), thuật toán tìm ứng viên trong lớp ((0,1)), rồi mới đến các lớp còn lại. Khi kết hợp với cách tổ chức đồng hồ, hệ thống chọn trang đầu tiên gặp được trong lớp có mức ưu tiên cao nhất đang tồn tại. Quá trình này có thể cần nhiều lượt quét; các thao tác xóa bit tham chiếu cũng có thể làm thay đổi lớp của trang trong quá trình tìm kiếm. Do đó, cần phân biệt nguyên tắc ưu tiên bốn lớp với cách tổ chức cụ thể các lượt quét khi triển khai.

So với cơ hội thứ hai thông thường, phiên bản nâng cao có thêm khả năng phân biệt hai trang có mức độ sử dụng tương tự nhưng chi phí loại bỏ khác nhau. Chẳng hạn, giữa hai trang cùng có (R=0), lựa chọn trang sạch có thể tránh được một thao tác ghi mà vẫn giải phóng được khung. Tuy nhiên, ưu tiên giữ trang bẩn không có nghĩa rằng các trang này được miễn thay thế; khi không còn ứng viên phù hợp hơn, hệ thống vẫn phải lựa chọn và bảo toàn nội dung của chúng. Lợi ích của thuật toán nằm ở việc giảm những lần ghi có thể tránh được, chứ không loại bỏ hoàn toàn nhu cầu ghi trang bẩn.

Đánh giá chung

Ba phương pháp thể hiện những mức độ khai thác lịch sử khác nhau. Thuật toán các bit tham chiếu bổ sung lưu dấu sử dụng qua nhiều khoảng thời gian; cơ hội thứ hai dùng một bit để điều chỉnh lựa chọn FIFO; còn cơ hội thứ hai nâng cao bổ sung trạng thái sửa đổi nhằm cân nhắc chi phí vào/ra. Điểm chung là chúng giảm yêu cầu theo dõi so với LRU chính xác bằng cách chấp nhận mất bớt thông tin về thứ tự truy cập. Vì vậy, không thể mặc nhiên gán cho các thuật toán xấp xỉ mọi tính chất của LRU, chẳng hạn tính chất ngăn xếp hay khả năng tránh nghịch lý Belady. Giá trị thực tiễn của chúng nằm ở sự cân bằng giữa chất lượng lựa chọn trang nạn nhân, lượng thông tin cần duy trì và chi phí thực hiện trong hệ thống.

10.4.6. Counting-Based Page Replacement

10.4.6. Thay thế trang dựa trên bộ đếm (Counting-Based Page Replacement)

Các thuật toán thay thế trang dựa trên bộ đếm sử dụng số lần tham chiếu để đánh giá mức độ sử dụng của từng trang. Khác với LRU quan tâm đến lần truy cập gần nhất, nhóm này xem xét **tần suất truy cập** được ghi nhận trong bộ đếm. Hai cách lựa chọn tiêu biểu là **LFU (*Least Frequently Used*)**, thay thế trang có số lần tham chiếu nhỏ nhất, và **MFU (*Most Frequently Used*)**, thay thế trang có số lần tham chiếu lớn nhất. Mặc dù cùng sử dụng một loại thông tin, hai thuật toán dựa trên những giả định khác nhau về giá trị sử dụng tiếp theo của trang.

LFU cho rằng trang được sử dụng tích cực thường có số lần tham chiếu lớn, nên trang có số đếm thấp là ứng viên phù hợp để loại bỏ. Tuy nhiên, số đếm tích lũy có thể không phản ánh nhu cầu hiện tại. Chẳng hạn, một trang được truy cập nhiều trong giai đoạn khởi tạo có thể giữ số đếm cao dù không còn cần thiết ở giai đoạn xử lý tiếp theo. Trong khi đó, một trang mới bắt đầu được sử dụng thường xuyên lại có số đếm thấp và dễ bị thay thế. Hạn chế này xuất phát từ việc LFU có thể đánh giá quá cao lịch sử xa. Một biện pháp khắc phục là định kỳ dịch phải bộ đếm một bit, tương đương giảm xấp xỉ một nửa giá trị hiện có, để ảnh hưởng của những lần sử dụng cũ suy giảm theo thời gian. Nhờ đó, hoạt động truy cập mới có vai trò lớn hơn trong quyết định thay thế.

Ngược lại, MFU dựa trên lập luận rằng một trang có số đếm nhỏ có thể vừa được nạp vào và chưa có đủ thời gian để được sử dụng nhiều. Vì vậy, thuật toán lựa chọn trang có số đếm lớn nhất để thay thế, thay vì loại bỏ ngay những trang mới có ít lượt tham chiếu. Tuy nhiên, số đếm lớn cũng có thể biểu thị một trang vẫn đang được sử dụng thường xuyên, nên giả định của MFU không bảo đảm đúng trong mọi trường hợp. Theo đánh giá của giáo trình, cả LFU và MFU đều không phổ biến trong thay thế trang vì chi phí triển khai cao và khả năng xấp xỉ Optimal còn hạn chế. Điều này cho thấy số lần truy cập chỉ cung cấp một khía cạnh của hành vi sử dụng: trang từng được dùng nhiều chưa chắc còn cần thiết, còn trang ít được dùng chưa chắc sẽ không cần trong tương lai gần.

10.4.7. Các thuật toán bộ đệm trang (Page-Buffering Algorithms)

Các cơ chế bộ đệm trang thường được sử dụng bổ trợ cho thuật toán thay thế nhằm giảm thời gian chờ khi xử lý lỗi trang. Hệ thống duy trì một tập khung trống dự phòng; khi lỗi trang xảy ra, trang được yêu cầu có thể được đọc vào một khung trong tập này trước khi hoàn tất việc ghi trang nạn nhân ra thiết bị lưu trữ. Tiến trình nhờ đó có thể tiếp tục ngay khi trang cần thiết đã sẵn sàng, thay vì phải chờ lần lượt cả thao tác ghi trang cũ và đọc trang mới. Sau khi trang nạn nhân được ghi ra ngoài và khung của nó được thu hồi, khung đó được bổ sung vào tập dự phòng. Cơ chế này không nhất thiết làm giảm số lỗi trang, nhưng tách một phần công việc thu hồi khung khỏi thời gian chờ trực tiếp của tiến trình.

Một cách mở rộng là duy trì danh sách các trang đã sửa đổi và chủ động ghi chúng ra thiết bị lưu trữ khi thiết bị rảnh. Khi việc ghi hoàn tất và không có thay đổi mới cần bảo toàn, trang có thể được đánh dấu sạch. Nếu sau đó trang được chọn làm nạn nhân, hệ thống không phải ghi lại nội dung trước khi tái sử dụng khung. Như vậy, hoạt động ghi được chuyển sang thời điểm thuận lợi hơn, làm tăng khả năng xử lý nhanh khi cần thay thế. Hiệu quả của phương pháp đến từ việc tổ chức thời điểm thực hiện vào/ra, chứ không chỉ từ tiêu chí lựa chọn trang.

Một kỹ thuật khác là ghi nhớ trang từng nằm trong mỗi khung thuộc tập khung tự do. Việc thu hồi khung không nhất thiết xóa ngay nội dung cũ; nếu trang được yêu cầu trở lại trước khi khung bị ghi đè, hệ thống có thể sử dụng lại nội dung còn trong bộ nhớ mà không cần đọc từ thiết bị lưu trữ. Khi xử lý lỗi trang, hệ điều hành kiểm tra khả năng này trước khi thực hiện thao tác đọc. Kỹ thuật đó giúp giảm hậu quả của việc loại bỏ một trang rồi sớm cần đến nó, nhưng chỉ có tác dụng khi nội dung vẫn còn hợp lệ và khung chưa được tái sử dụng. Các cơ chế bộ đệm vì vậy cho thấy hiệu năng thay thế trang phụ thuộc cả vào cách lựa chọn nạn nhân lẫn cách tổ chức thu hồi, ghi và tái sử dụng khung.

10.4.8. Ứng dụng và thay thế trang (Applications and Page Replacement)

Một chính sách thay thế phù hợp với nhiều chương trình chưa chắc hiệu quả đối với mọi ứng dụng. Hệ điều hành thường đưa ra quyết định từ những thông tin truy cập có thể quan sát, trong khi ứng dụng chuyên biệt có thể hiểu rõ hơn về cấu trúc dữ liệu và trình tự xử lý của mình. Hệ quản trị cơ sở dữ liệu là ví dụ điển hình vì thường có cơ chế quản lý bộ nhớ và bộ đệm vào/ra riêng. Nếu cùng dữ liệu còn được lưu đệm ở cấp hệ điều hành, hiện tượng **lưu đệm kép** có thể xuất hiện, khiến bộ nhớ được sử dụng cho nhiều bản sao mà không tạo ra lợi ích tương xứng. Trong trường hợp này, sự chồng lặp giữa các lớp quản lý có thể làm giảm hiệu quả khai thác bộ nhớ.

Sự không phù hợp còn thể hiện trong những ứng dụng thực hiện các lượt đọc tuần tự lớn, chẳng hạn kho dữ liệu. LRU có xu hướng giữ các trang vừa được đọc ở cuối lượt quét và loại bỏ các trang đã đọc từ trước. Tuy nhiên, nếu lượt quét tiếp theo bắt đầu lại từ đầu, các trang cũ hơn lại là những trang cần được truy cập trước. Khi đó, lịch sử sử dụng gần nhất không phản ánh đúng thứ tự nhu cầu sắp tới. Giáo trình nêu rằng MFU có thể hiệu quả hơn LRU trong kiểu tình huống này. Nhận xét đó cho thấy hiệu quả của thuật toán phụ thuộc vào đặc điểm truy cập, không có nghĩa MFU luôn ưu việt hơn LRU đối với hoạt động đọc tuần tự hoặc mọi ứng dụng cơ sở dữ liệu.

Để hỗ trợ ứng dụng có yêu cầu đặc biệt, một số hệ điều hành cho phép sử dụng một phân vùng lưu trữ như một dãy khối logic, thường gọi là **raw disk**, với hoạt động truy cập gọi là **raw I/O**. Cách tiếp cận này bỏ qua các dịch vụ của hệ thống tệp như đặt tên tệp, tổ chức thư mục, cấp phát không gian, khóa tệp và đọc trước, tạo điều kiện để ứng dụng tự quản lý lưu trữ theo nhu cầu. Đổi lại, ứng dụng phải đảm nhiệm những chức năng cần thiết mà hệ thống tệp vốn cung cấp. Vì vậy, raw I/O chỉ có lợi khi cơ chế quản lý chuyên biệt mang lại hiệu quả đủ lớn để bù đắp trách nhiệm bổ sung. Đối với phần lớn ứng dụng, sử dụng các dịch vụ hệ thống tệp thông thường vẫn là lựa chọn hiệu quả hơn. Nội dung này nhấn mạnh rằng đánh giá thay thế trang cần gắn với cách ứng dụng sử dụng dữ liệu và sự phối hợp giữa các tầng quản lý bộ nhớ, thay vì chỉ so sánh thuật toán một cách độc lập.